

Artificial Potential Field and Extended Kalman Filter Methods

Equations and Source-Code Mapping

Contents

Abbreviations and full forms	3
1 Six-state vessel EKF	3
1.1 State vector and motion model	3
1.2 State-transition Jacobian	4
1.3 Prediction step	5
1.4 Pose and yaw-rate measurement models	5
1.5 Measurement update	6
2 Four-state obstacle constant-velocity EKF	6
2.1 State vector and constant-velocity prediction	6
2.2 Process noise	7
2.3 Position measurement update	7
2.4 Future position	8
3 APF obstacle geometry	8
3.1 Elliptical level set	8
3.2 Smooth elliptical repulsion	9
3.3 Observed dimensions in snapshot generation	10
4 APF attractive terms	10
4.1 Goal attraction	10
4.2 Path attraction	11
5 CPA-based dynamic-obstacle risk	12
6 COLREG encounter classification	13
7 Route-entry risk and virtual obstacle corridor	13

8	APF force composition and control output	14
8.1	Total force	14
8.2	Force direction and yaw-rate command	15
9	Processing chain	15
10	Implementation index	16
A	Complete implementation source chain	17
A.1	Behaviour-tree dispatch	17
A.2	Shared COLREG and APF geometry	18
A.3	Unified vessel, EKF, APF, and route-planning controller	21
A.4	Crossing encounter controller	112
A.5	Head-on encounter controller	131
A.6	Overtaking encounter controller	139

Abbreviations and full forms

Symbol or term	Meaning
N, E	Northing and easting coordinates.
G or ϕ	Vessel heading, in radians.
Δt	Discrete time step.
μ or $\mathbf{x}$	State estimate.
Σ or $\mathbf{P}$	State covariance matrix.
$\mathbf{Q}, \mathbf{R}$	Process-noise and measurement-noise covariance matrices.
$\mathbf{F}, \mathbf{H}$	Jacobians of the state-transition and measurement functions.
$\mathbf{z}$	Sensor measurement.
$\mathbf{K}$	Kalman gain.
Body frame	Vessel-fixed frame, with x forward and y to starboard.
Earth/NE frame	Earth-fixed north-east frame.
APF	Artificial Potential Field.
COLREGs	International Regulations for Preventing Collisions at Sea.
CPA	Closest point of approach.
DBSCAN	Density-Based Spatial Clustering of Applications with Noise.
TCPA, DCPA	Time to CPA and distance at CPA.
EKF	Extended Kalman Filter.
IMU	Inertial Measurement Unit.
LiDAR	Light Detection and Ranging.
PC	Principal Component.
PC1	First Principal Component.
PC2	Second Principal Component.
PCA	Principal Component Analysis.

Angular differences are normalised to $[-\pi, \pi)$ by `wrap_angle()`.

1 Six-state vessel EKF

In this chapter, N and E are northing and easting, G is the vessel heading, and dotted quantities are their time derivatives. Bold lower-case letters denote vectors and bold upper-case letters denote matrices. The subscripts b and e identify the body and earth frames, respectively; k is the discrete-time index. The symbols Δt , $\mathbf{u}$, $\mathbf{x}$, $\mathbf{p}$, $\mathbf{v}$, and $\mathbf{F}$ denote the sampling interval, thruster input, state, position, velocity, and force. The matrices $\mathbf{G}$, $\mathbf{R}_{be}(G)$, and $\mathbf{F}$ are, respectively, the thruster allocation matrix, the rotation from body to earth coordinates at heading G , and the state-transition Jacobian; the bold matrix $\mathbf{G}$ is distinguished from the scalar heading G by its typography. The functions f_{dyn} and f_{kin} are the vessel dynamics and rigid-body kinematics.

1.1 State vector and motion model

The vessel EKF uses the state vector

$$\mathbf{x} = \begin{bmatrix} N & E & G & \dot{N} & \dot{E} & \dot{G} \end{bmatrix}^T. \quad (1)$$

The thruster input is

$$\mathbf{u} = \begin{bmatrix} T_R & T_L \end{bmatrix}^T. \quad (2)$$

The allocation matrix maps the thruster forces to body-frame force and yaw moment,

$$\mathbf{F}_b = \mathbf{G}\mathbf{u}, \quad (3)$$

and the heading-dependent rotation maps this vector to the Earth-fixed frame,

$$\mathbf{F}_e = \mathbf{R}_{be}(G)\mathbf{F}_b. \quad (4)$$

The vessel dynamics produce the next Earth-frame velocity,

$$\mathbf{v}_{e,k+1} = f_{\text{dyn}}(\mathbf{v}_{e,k}, \mathbf{F}_e, \Delta t). \quad (5)$$

The Earth-frame velocity is transformed back to the body frame. Its surge and yaw-rate components form the kinematic input,

$$\mathbf{v} = v_b^x, \quad \omega = v_b^G. \quad (6)$$

The rigid-body kinematics then update the pose,

$$\mathbf{p}_{k+1} = f_{\text{kin}}(\mathbf{p}_k, [\mathbf{v}, \omega]^\top, \Delta t). \quad (7)$$

The implementation is located in `motion_model()`:

```
Fb = self.G @ control_input
H_eb = HomogeneousTransformation(state[N:E+1], state[G])
Fe = H_eb.H_R @ Fb

ve = self.robot.model(
    dynamics_translation_e,
    dynamics_rotation_e,
    Fe,
    state[DOTN:DOTG+1],
    dt,
)

vb = Inverse(H_eb.H_R) @ ve
u = Vector(2)
u[0, 0] = vb[0, 0]
u[1, 0] = vb[2, 0]

p = rigid_body_kinematics(state[N:G+1], u, dt)
p[2, 0] = p[2, 0] % (2 * np.pi)
```

1.2 State-transition Jacobian

The implementation uses a first-order discrete-time Jacobian. Here $\mathbf{F}_k$ is the Jacobian of the transition function at time k , and the unit entries preserve direct integration of northing, easting, and heading from their corresponding rates:

$$\mathbf{F} = \begin{bmatrix} 1 & 0 & 0 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 & \Delta t & 0 \\ 0 & 0 & 1 & 0 & 0 & \Delta t \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}. \quad (8)$$

```

F = Identity(6)
F[N, DOTN] = dt
F[E, DOTE] = dt
F[G, DOTG] = dt

```

1.3 Prediction step

The symbols $\hat{\mathbf{x}}_k$ and $\hat{\Sigma}_k$ denote the predicted state and covariance, while $\mathbf{x}_{k-1}$ and Σ_{k-1} are the previous corrected state and covariance. The transition function is f , $\mathbf{Q}_k$ is the process-noise covariance, and the superscript T denotes matrix transpose. The state and covariance predictions are

$$\hat{\mathbf{x}}_k = f(\mathbf{x}_{k-1}, \mathbf{u}_k, \Delta t), \quad (9)$$

$$\hat{\Sigma}_k = \mathbf{F}_k \Sigma_{k-1} \mathbf{F}_k^T + \mathbf{Q}_k. \quad (10)$$

```

def extended_kalman_filter_predict(mu, Sigma, u, f, Q, dt):
    pred_mu, F = f(mu, u, dt)
    pred_Sigma = F @ Sigma @ F.T + Q
    return pred_mu, pred_Sigma

```

1.4 Pose and yaw-rate measurement models

The measurement vector is $\mathbf{z}$, and h_{pose} is the pose measurement function. The matrix $\mathbf{H}$ is its measurement Jacobian. The scalar z_{gyro} is the measured yaw rate, and H_{gyro} is the corresponding Jacobian entry. The ArUco measurement contains northing, easting, and heading:

$$\mathbf{z}_{\text{pose}} = h_{\text{pose}}(\mathbf{x}) = \begin{bmatrix} N & E & G & 0 & 0 & 0 \end{bmatrix}^T. \quad (11)$$

Its Jacobian has unit entries at (N, N) , (E, E) , and (G, G) . The IMU model measures yaw rate:

$$z_{\text{gyro}} = \dot{G}, \quad H_{\text{gyro}}[\dot{G}, \dot{G}] = 1. \quad (12)$$

```

def h_pose_update(x):
    est_measurement = Vector(6)
    est_measurement[N] = x[N]
    est_measurement[E] = x[E]
    est_measurement[G] = x[G]
    H = Matrix(6, 6)
    H[N, N] = 1
    H[E, E] = 1
    H[G, G] = 1
    return est_measurement, H

def h_grate_update(x):
    est_measurement = Vector(6)
    est_measurement[DOTG] = x[DOTG]
    H = Matrix(6, 6)
    H[DOTG, DOTG] = 1
    return est_measurement, H

```

1.5 Measurement update

In this subsection, $\hat{z}$ is the predicted measurement, $\mathbf{y}$ is the innovation, $\mathbf{S}$ is the innovation covariance, and $\mathbf{K}$ is the Kalman gain. Superscripts $-$ and $+$ identify the predicted and corrected quantities. The matrix $\mathbf{I}$ is the identity matrix, and $\mathbf{R}$ is the measurement-noise covariance. The predicted measurement, innovation, innovation covariance, and Kalman gain are

$$\hat{z} = h(\hat{\mathbf{x}}), \quad (13)$$

$$\mathbf{y} = \mathbf{z} - \hat{z}, \quad (14)$$

$$\mathbf{S} = \mathbf{H}\mathbf{\Sigma}^-\mathbf{H}^\top + \mathbf{R}, \quad (15)$$

$$\mathbf{K} = \mathbf{\Sigma}^-\mathbf{H}^\top\mathbf{S}^{-1}. \quad (16)$$

The corrected state is

$$\mathbf{x}^+ = \mathbf{x}^- + \mathbf{K}\mathbf{y}. \quad (17)$$

The heading innovation is wrapped before correction:

$$y_G = \text{wrap}(z_G - \hat{z}_G). \quad (18)$$

The vessel EKF currently updates the covariance as

$$\mathbf{\Sigma}^+ = (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{\Sigma}^-. \quad (19)$$

```
def extended_kalman_filter_update(
    mu, Sigma, z, h, R, wrap_index=None
):
    pred_z, H = h(mu)
    K = Sigma @ H.T @ Inverse(H @ Sigma @ H.T + R)
    delta_z = z - pred_z
    if wrap_index is not None:
        delta_z[wrap_index] = (
            delta_z[wrap_index] + np.pi
        ) % (2 * np.pi) - np.pi
    cor_mu = mu + K @ delta_z
    cor_Sigma = (Identity(mu.shape[0]) - K @ H) @ Sigma
    return cor_mu, cor_Sigma
```

In the main loop, the update order is thruster-based prediction, ArUco pose correction, and IMU yaw-rate correction.

2 Four-state obstacle constant-velocity EKF

The obstacle EKF contains no yaw, heading, or turn-rate state. At prediction index k , it estimates northing p_N , easting p_E , and their corresponding earth-frame velocities v_N and v_E . The symbol Δt denotes the prediction interval; $\mathbf{F}$, $\mathbf{Q}$, $\mathbf{H}$, $\mathbf{R}$, $\mathbf{P}$, and $\mathbf{K}$ denote the state-transition, process-noise, measurement, measurement-noise, covariance, and Kalman-gain matrices respectively.

2.1 State vector and constant-velocity prediction

Each tracked obstacle uses the four-state vector

$$\mathbf{x} = \begin{bmatrix} p_N & p_E & v_N & v_E \end{bmatrix}^\top. \quad (20)$$

Its constant-velocity transition is

$$\mathbf{x}_{k+1} = \mathbf{F}\mathbf{x}_k, \quad \mathbf{F} = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (21)$$

```
state = np.asarray(state, dtype=float).reshape(4)
covariance = np.asarray(covariance, dtype=float).reshape(4, 4)
F = np.array([
    [1.0, 0.0, dt, 0.0],
    [0.0, 1.0, 0.0, dt],
    [0.0, 0.0, 1.0, 0.0],
    [0.0, 0.0, 0.0, 1.0],
], dtype=float)
predicted_state = F @ state
predicted_covariance = F @ covariance @ F.T + self.obstacle_ekf_process_noise(dt)
```

The direction used to orient a predicted APF ellipse is derived from the estimated velocity vector when its magnitude is non-zero; it is not stored or filtered as an additional yaw state.

2.2 Process noise

The process covariance assumes white acceleration with standard deviation σ_a . With $q = \sigma_a^2$, the implemented covariance is

$$\mathbf{Q} = q \begin{bmatrix} \frac{\Delta t^4}{4} & 0 & \frac{\Delta t^3}{2} & 0 \\ 0 & \frac{\Delta t^4}{4} & 0 & \frac{\Delta t^3}{2} \\ \frac{\Delta t^3}{2} & 0 & \Delta t^2 & 0 \\ 0 & \frac{\Delta t^3}{2} & 0 & \Delta t^2 \end{bmatrix}. \quad (22)$$

```
q = self.obstacle_ekf_accel_std_m_s2 ** 2
dt2, dt3, dt4 = dt * dt, dt * dt * dt, (dt * dt) ** 2
return q * np.array([
    [0.25 * dt4, 0.0, 0.5 * dt3, 0.0],
    [0.0, 0.25 * dt4, 0.0, 0.5 * dt3],
    [0.5 * dt3, 0.0, dt2, 0.0],
    [0.0, 0.5 * dt3, 0.0, dt2],
], dtype=float)
```

2.3 Position measurement update

LiDAR supplies the two-dimensional obstacle position,

$$\mathbf{z} = \begin{bmatrix} p_N^{\text{meas}} & p_E^{\text{meas}} \end{bmatrix}^\top, \quad \mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}. \quad (23)$$

The correction uses the innovation $\boldsymbol{\nu} = \mathbf{z} - \mathbf{H}\mathbf{x}^-$ and the Joseph covariance form,

$$\mathbf{S} = \mathbf{H}\mathbf{P}^- \mathbf{H}^\top + \mathbf{R}, \quad \mathbf{K} = \mathbf{P}^- \mathbf{H}^\top \mathbf{S}^{-1}, \quad (24)$$

$$\mathbf{x}^+ = \mathbf{x}^- + \mathbf{K}\boldsymbol{\nu}, \quad \mathbf{P}^+ = (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}^- (\mathbf{I} - \mathbf{K}\mathbf{H})^\top + \mathbf{K}\mathbf{R}\mathbf{K}^\top. \quad (25)$$

```

H = np.array([[1.0, 0.0, 0.0, 0.0], [0.0, 1.0, 0.0, 0.0]])
innovation = measurement_ne - H @ state
innovation_covariance = H @ covariance @ H.T + R_obs
try:
    kalman_gain = covariance @ H.T @ np.linalg.inv(innovation_covariance)
except np.linalg.LinAlgError:
    kalman_gain = covariance @ H.T @ np.linalg.pinv(innovation_covariance)
corrected_state = state + kalman_gain @ innovation
correction = np.eye(4) - kalman_gain @ H
corrected_covariance = correction @ covariance @ correction.T + kalman_gain @ R_obs
    @ kalman_gain.T

```

The vessel's own yaw uncertainty can enlarge $\mathbf{R}$ because it rotates a LiDAR measurement into the earth frame. This is a measurement-uncertainty term only; it does not introduce yaw into the four-state obstacle EKF.

2.4 Future position

For a horizon $\tau \geq 0$, the code propagates a track with

$$\mathbf{p}(\tau) = \begin{bmatrix} p_N & p_E \end{bmatrix}^T + \tau \begin{bmatrix} v_N & v_E \end{bmatrix}^T. \quad (26)$$

```

state = np.asarray(track.get("state", [np.nan] * 4), dtype=float).reshape(4)
position_ne = state[:2]
velocity_ne = state[2:4]
return position_ne + velocity_ne * max(float(dt_s), 0.0), velocity_ne.copy()

```

3 APF obstacle geometry

In this chapter, $\mathbf{r}$ is the relative position from the obstacle centre to the vessel, while $\mathbf{l}$ and $\mathbf{w}$ are orthogonal unit vectors along the obstacle length and width. The observed principal-component dimensions $pc1$ and $pc2$ define the corresponding semi-axes a_0 and b_0 . The scalars a and b are the projections of $\mathbf{r}$ onto these axes, ρ is the dimensionless elliptical level, $\mathbf{g}$ is the outward gradient, and $\hat{\mathbf{d}}$ is its unit direction. The scalar s is the outer field scale, u is the normalised decay coordinate, f_{fade} is the smooth fade factor, k_{rep} is the repulsive gain, and $\mathbf{F}_{\text{rep}}$ is the resulting repulsive force. A hat denotes a unit vector, $\|\cdot\|$ denotes Euclidean norm, and $\cdot$ denotes the inner product.

3.1 Elliptical level set

The measured obstacle dimensions $pc1$ and $pc2$ are obtained from the first and second principal components of the clustered LiDAR points. The axial projections are

$$a = \mathbf{r} \cdot \mathbf{l}, \quad b = \mathbf{r} \cdot \mathbf{w}. \quad (27)$$

The APF semi-axes are defined from the observed dimensions and enlarged by the own-vessel equivalent radius r_{own} :

$$a_0 = \frac{pc1}{2} + r_{\text{own}}, \quad b_0 = \frac{pc2}{2} + r_{\text{own}}. \quad (28)$$

Minimum PC dimensions are used only when an observation is invalid or unavailable. The elliptical level is

$$\rho = \sqrt{\left(\frac{a}{a_0}\right)^2 + \left(\frac{b}{b_0}\right)^2}. \quad (29)$$

The outward gradient and corresponding unit direction are

$$\mathbf{g} = \frac{a}{a_0^2} \mathbf{l} + \frac{b}{b_0^2} \mathbf{w}, \quad (30)$$

$$\hat{\mathbf{d}} = \frac{\mathbf{g}}{\|\mathbf{g}\|}. \quad (31)$$

The condition $\rho < 1$ indicates that the vessel has entered the observed obstacle boundary. The repulsive field is inactive when $\rho \geq s$.

```
pc1_m, pc2_m = self.obstacle_pc_dimensions(obstacle)
semi_length_m = 0.5 * pc1_m + self.apf_own_equivalent_radius_m
semi_width_m = 0.5 * pc2_m + self.apf_own_equivalent_radius_m

along = float(np.dot(offset, length_axis))
across = float(np.dot(offset, width_axis))
level = float(np.hypot(
    along / semi_length_m,
    across / semi_width_m,
))
gradient = (
    along / (semi_length_m * semi_length_m) * length_axis
    + across / (semi_width_m * semi_width_m) * width_axis
)
away = gradient / np.linalg.norm(gradient)
```

3.2 Smooth elliptical repulsion

For $s > 1$, the implementation applies a cubic smoothstep decay. The variable u clips the level interval $[1, s]$ to $[0, 1]$, so that the fade remains bounded both inside the obstacle and outside the influence region:

$$u = \text{clip}\left(\frac{\rho - 1}{s - 1}, 0, 1\right), \quad (32)$$

$$f_{\text{fade}} = 1 - (3u^2 - 2u^3), \quad (33)$$

$$\mathbf{F}_{\text{rep}} = k_{\text{rep}} f_{\text{fade}} \hat{\mathbf{d}}. \quad (34)$$

The force is maximal at and inside $\rho = 1$, and it decreases smoothly to zero at $\rho = s$. To retain the measured size information in the force magnitude, the implementation scales the gain by the square root of the PCA footprint, while clipping the multiplier to a bounded range:

$$k_{\text{eff}} = k_{\text{rep}} \text{clip}\left(\sqrt{pc1 pc2}, 0.5, 2.0\right). \quad (35)$$

```
u = np.clip(
    (level - 1.0) / (outer_scale - 1.0),
    0.0,
    1.0,
)
fade = 1.0 - (3.0 * u * u - 2.0 * u * u * u)
return gain * fade * np.asarray(away).reshape(2), True
```

The corresponding controller code evaluates the ellipse at the actual measured or virtual-field position. Hence DCPA remains a diagnostic quantity; it is not treated as a second force location.

```

level, away = ellipse_level_and_away(
    -centre_body, axis_body,
    0.5 * pc1_m + self.apf_own_equivalent_radius_m,
    0.5 * pc2_m + self.apf_own_equivalent_radius_m,
)
if level >= outer_scale:
    return np.zeros(2, dtype=float), False
size_gain = np.sqrt(max(pc1_m * pc2_m, 1e-6))
gain = self.apf_repulsive_gain * np.clip(size_gain, 0.5, 2.0)
return smooth_ellipse_repulsion(level, away, gain, outer_scale)

```

The project uses two fields with this form. The first is centred on the current LiDAR obstacle, while the second is centred on the EKF/CPA-predicted obstacle. Their outer scales are selected from encounter-specific profiles: head-on, overtaking, crossing, and static-obstacle cases. Predicted fields use a separate profile table from current-observation fields.

3.3 Observed dimensions in snapshot generation

The offline potential-field renderer uses the same observed $pc1$ and $pc2$ dimensions. The stored equivalent radius is not used to reconstruct the obstacle dimensions when these observations are present; fallback values are the configured minimum PC dimensions.

```

pc1_m = positive(obstacle.get("pc1_m"), minimum_pc1_m)
pc2_m = positive(obstacle.get("pc2_m"), minimum_pc2_m)
return max(pc1_m, minimum_pc1_m), max(
    min(pc2_m, pc1_m), minimum_pc2_m
)

```

4 APF attractive terms

In this chapter, all attractive-force vectors are expressed in the vessel body frame. The target vector is $\mathbf{t}$, its distance is d , k_g is the goal gain, and d_{sat} is the maximum distance used for saturating the goal-force magnitude. The path variables $\mathbf{s}$, $\mathbf{g}$, $\mathbf{p}$, d_{path} , L_{path} , λ , and $\mathbf{p}_{\text{path}}$ denote the start, goal, current position, route vector, route length, clipped projection coefficient, and selected route point. The transformation $T_{\text{earth} \rightarrow \text{body}}$ maps earth-frame vectors into body-frame vectors. The path error and its norm are $\mathbf{e}_{\text{path}}$ and d_{path} ; k_p is the path gain, and $d_{\text{threshold}}$ is the path dead-band threshold. Finally, $\mathbf{F}_{\text{goal}}$, $\mathbf{F}_{\text{path}}$, and $\mathbf{F}_{\text{att}}$ denote the goal, path, and total attractive forces.

4.1 Goal attraction

Let $\mathbf{t}$ be the target position in the body frame, with $d = \|\mathbf{t}\|$. The saturated goal-attraction term is

$$\mathbf{F}_{\text{goal}} = k_g \min(d, d_{\text{sat}}) \frac{\mathbf{t}}{d}. \quad (36)$$

```

distance_m = float(np.linalg.norm(target_body))
magnitude = self.apf_goal_gain * min(
    distance_m,

```

```

        self.apf_attraction_saturation_m,
    )
    goal_force = magnitude * target_body / distance_m

```

4.2 Path attraction

The path-attraction field keeps the vessel close to the straight route from the initial position to the goal. Let $\mathbf{s}$ and $\mathbf{g}$ denote the start and goal positions in the north-east frame, and let $\mathbf{p}$ be the current vessel position. The route vector and its squared length are

$$\begin{aligned} \mathbf{d}_{\text{path}} &= \mathbf{g} - \mathbf{s}, \\ L_{\text{path}}^2 &= \mathbf{d}_{\text{path}}^\top \mathbf{d}_{\text{path}}. \end{aligned} \quad (37)$$

If L_{path}^2 is smaller than a numerical tolerance, the start and goal are treated as coincident and the path-attraction force is set to zero.

For a valid route, the projection coefficient of the current position onto the start-to-goal segment is

$$\lambda^* = \frac{(\mathbf{p} - \mathbf{s})^\top \mathbf{d}_{\text{path}}}{L_{\text{path}}^2}. \quad (38)$$

The implementation clips this coefficient to $[0, 1]$:

$$\lambda = \text{clip}(\lambda^*, 0, 1). \quad (39)$$

Consequently, the reference point is on the finite route segment rather than on its infinite extension,

$$\mathbf{p}_{\text{path}} = \mathbf{s} + \lambda \mathbf{d}_{\text{path}}. \quad (40)$$

When the perpendicular projection falls before the start or beyond the goal, the start or goal endpoint is therefore used as the reference point.

The vector from the vessel to this reference point is first calculated in the north-east frame and then transformed into the vessel body frame. With $T_{\text{earth} \rightarrow \text{body}}$ denoting the rotation from the earth frame to the body frame,

$$\mathbf{e}_{\text{path}} = T_{\text{earth} \rightarrow \text{body}}(\mathbf{p}_{\text{path}} - \mathbf{p}). \quad (41)$$

The norm $d_{\text{path}} = \|\mathbf{e}_{\text{path}}\|$ is the current cross-track distance to the selected route point. The resulting path force is

$$\mathbf{F}_{\text{path}} = \begin{cases} \mathbf{0}, & d_{\text{path}} < d_{\text{threshold}}, \\ k_p \mathbf{e}_{\text{path}}, & d_{\text{path}} \geq d_{\text{threshold}}. \end{cases} \quad (42)$$

The dead-band threshold prevents small position-estimation fluctuations from causing continuous corrective commands while the vessel is already close to the route. Outside this band, the force magnitude is proportional to the cross-track error and its direction points toward the route.

```

path_vec = self.goal_ne - self.start_ne
path_len_sq = float(np.dot(path_vec, path_vec))
if path_len_sq < 1e-9:
    return np.zeros(2, dtype=float)

current_ne = np.array(
    [float(self.North), float(self.East)], dtype=float
)
ratio = float(np.clip(

```

```

    np.dot(current_ne - self.start_ne, path_vec)
    / path_len_sq,
    0.0,
    1.0,
))
closest_ne = self.start_ne + ratio * path_vec
to_path_body = self.earth_point_to_body(closest_ne)
distance_m = float(np.linalg.norm(to_path_body))

if distance_m < self.apf_path_threshold_m or distance_m < 1e-6:
    return np.zeros(2, dtype=float)
return self.apf_path_gain * to_path_body

```

The total attractive term is the vector sum of the two components:

$$\mathbf{F}_{\text{att}} = \mathbf{F}_{\text{goal}} + \mathbf{F}_{\text{path}}. \quad (43)$$

5 CPA-based dynamic-obstacle risk

In this chapter, $\mathbf{p}$ is the obstacle position relative to the vessel, $\mathbf{v}_{\text{obs}}$ and $\mathbf{v}_{\text{own}}$ are the obstacle and own- vessel velocities, and $\mathbf{v}_{\text{rel}}$ is their relative velocity. TCPA is the non-negative time to the closest point of approach, and DCPA is the distance at that time. The symbols $\mathbf{p}(t)$ and $\mathbf{v}_{\text{rel}}$ describe relative position and velocity, while $\|\cdot\|$ denotes vector magnitude. The zero-speed threshold is a numerical guard used when the relative-velocity norm is too small.

The relative velocity is

$$\mathbf{v}_{\text{rel}} = \mathbf{v}_{\text{obs}} - \mathbf{v}_{\text{own}}. \quad (44)$$

If $\|\mathbf{v}_{\text{rel}}\|^2$ is sufficiently small, the implementation returns

$$\text{TCPA} = \infty, \quad \text{DCPA} = \|\mathbf{p}\|. \quad (45)$$

Otherwise,

$$\text{TCPA} = \max\left(-\frac{\mathbf{p} \cdot \mathbf{v}_{\text{rel}}}{\|\mathbf{v}_{\text{rel}}\|^2}, 0\right), \quad (46)$$

$$\text{DCPA} = \|\mathbf{p} + \mathbf{v}_{\text{rel}} \text{TCPA}\|. \quad (47)$$

```

rel_vel = obs_vel_body - own_vel_body
rel_speed_sq = float(np.dot(rel_vel, rel_vel))

if rel_speed_sq < 1e-9:
    return np.inf, float(np.linalg.norm(obs_pos_body))

tcpa = max(
    -float(np.dot(obs_pos_body, rel_vel)) / rel_speed_sq,
    0.0,
)
dcpa = float(np.linalg.norm(
    obs_pos_body + rel_vel * tcpa
))
return tcpa, dcpa

```

6 COLREG encounter classification

The encounter classifier first uses the obstacle bearing in the vessel frame, then tests the obstacle course and the speed relation. The sectors are “ahead” ($[-67.5^\circ, 67.5^\circ]$), port, starboard, and astern. A head-on encounter is accepted only when an ahead obstacle has a reciprocal course. Overtaking is deliberately stricter: it requires a narrow ahead sector, an approximately common course, a verified stern-side geometry, and an own-vessel speed exceeding the obstacle speed. This avoids temporarily misclassifying a crossing vessel as an overtaking vessel when a track jitters near a sector boundary.

```
if head_on_ahead and head_on_reciprocal:
    return "head_on", -1.0, "COLREG Rule 14: both alter to starboard"

if ahead and (
    speed_known and overtaking_ahead and overtaking_same_course
    and own_in_obstacle_stern_sector
    and own_speed_m_s > obstacle_speed_m_s + 0.08
    and obstacle_lateral_speed_m_s <= 0.06
):
    return "overtaking", -1.0, "COLREG Rule 13: overtake on starboard side"

if on_starboard:
    return ("crossing_from_starboard", -1.0,
           "COLREG Rule 15: give way, pass astern")
```

For Rule 13, the controller uses a short-horizon EKF prediction only when the track is stable and its heading uncertainty is below the configured bound. Rule 14 may use the measured direction before this confirmation so that an approaching vessel is not missed during track initialisation.

7 Route-entry risk and virtual obstacle corridor

The controller does not create a single virtual field at DCPA. For every stable dynamic EKF track with $0 < \text{TCPA} \leq T_{\text{horizon}}$, it places the final virtual field at 2 TCPA and fills the interval from the current track position to that field with overlapping ellipses. The bridge turns the predicted motion into a continuous exclusion corridor, allowing route planning to respond before reaching the closest-approach point. The final field centre is

$$\mathbf{p}_{\text{virtual}} = \mathbf{p}_{\text{obs}} + 2 \text{ TCPA } \mathbf{v}_{\text{obs}}. \quad (48)$$

The route-entry check maps the line segment into the ellipse coordinate system. With $\mathbf{q}_0$ and $\mathbf{q}_1$ denoting the normalised start offset and direction, the route is unsafe when

$$\min_{0 \leq \lambda \leq 1} \|\mathbf{q}_0 + \lambda \mathbf{q}_1\| \leq 1. \quad (49)$$

```
final_ne = start_ne + velocity_ne * (2.0 * tcpa_s)
bridge_count = max(int(np.ceil(
    np.linalg.norm(final_ne - start_ne) / max_gap_m
)) - 1, 0)
for index in range(1, bridge_count + 1):
    self.apf_virtual_obstacles.append(field(
        start_ne + (final_ne - start_ne) * index / (bridge_count + 1),
        bridge=True,
    ))
```

```
self.apf_virtual_obstacles.append(field(final_ne))
```

```
axis /= axis_norm
lateral = np.array([-axis[1], axis[0]])
scale = np.array([half_long_m + margin_m, half_lateral_m + margin_m])
offset = np.array([np.dot(start-centre, axis), np.dot(start-centre, lateral)]) /
    scale
direction = np.array([np.dot(end-start, axis), np.dot(end-start, lateral)]) / scale
denominator = float(np.dot(direction, direction))
alpha = 0.0 if denominator < 1e-12 else float(np.clip(
    -np.dot(offset, direction) / denominator, 0.0, 1.0
))
return np.linalg.norm(offset + alpha * direction) <= 1.0
```

8 APF force composition and control output

In this chapter, $\mathbf{F}$ is the total body-frame guidance force, $\mathbf{F}_{\text{att}}$ is the attractive force, and $\mathbf{F}_{\text{rep}}$ is the accumulated repulsive force. When a COLREG side is locked, $\mathbf{F}_{\text{offset}}$ pulls the temporary target to that side of the main route. In the steering equations, F_x and F_y are the forward and lateral components of $\mathbf{F}$, θ_F is its steering angle, k_h is the heading gain, ω_c is the commanded yaw rate, and ω_{max} is its magnitude limit. The operator `atan2` returns the signed angle from the two force components, while `clip` limits a scalar to a prescribed interval. The final command contains surge speed v_c and yaw rate ω_c .

8.1 Total force

The controller sums the attractive force and the repulsive contributions:

$$\mathbf{F} = \mathbf{F}_{\text{att}} + \sum_i \mathbf{F}_{\text{rep},i}. \quad (50)$$

When a COLREG side is locked, the route look-ahead target is shifted by the required clearance in the selected route normal direction and contributes an additional attractive term,

$$\mathbf{F} \leftarrow \mathbf{F} + \mathbf{F}_{\text{offset}}. \quad (51)$$

If an active field would make the forward component non-positive, the steering vector is repaired with a small positive forward component and a lateral sign consistent with the locked avoidance side. This preserves turning authority without reversing the vessel.

```
force_body = attractive_force + repulsive_force
if any_repulsion and self.apf_side_lock_sign != 0.0:
    offset_target_ne = target_ne + (
        self.apf_side_lock_sign * clearance_offset_m * route_normal_left_ne
    )
    offset_target_body = self.earth_point_to_body(offset_target_ne)
    offset_force = self.apf_clearance_gain * (
        offset_target_body / max(np.linalg.norm(offset_target_body), 1e-6)
    )
    force_body += offset_force

if any_repulsion and force_body[0] <= 0.0:
    steering_force[1] = side_sign * max(abs(force_body[1]), 0.20)
    steering_force[0] = max(0.25 * abs(steering_force[1]), 0.05)
```

The controller also preserves the COLREGs-selected avoidance side. If the forward force becomes non-positive, it retains a lateral component and inserts a small positive forward component.

8.2 Force direction and yaw-rate command

The steering angle is

$$\theta_F = \text{atan2}(F_y, F_x). \quad (52)$$

After limiting this angle, the yaw-rate command is

$$\omega_c = \text{clip}\left(-\frac{k_h \theta_F}{\Delta t}, -\omega_{\max}, \omega_{\max}\right). \quad (53)$$

```
force_angle = np.arctan2(
    steering_force[1], steering_force[0]
)
force_angle = np.clip(
    force_angle,
    -self.apf_heading_step_limit_rad,
    self.apf_heading_step_limit_rad,
)
u_cmd[1, 0] = np.clip(
    -self.apf_heading_gain * force_angle
    / max(self.lasdt, 1e-3),
    -self.w_max,
    self.w_max,
)
```

During APF avoidance, the force magnitude does not directly determine surge speed. The active encounter profile supplies a conservative descent speed, and a close overtaking manoeuvre can further reduce it while retaining the minimum turning angle:

```
surge_speed = float(active_params.get(
    "constant_descent_speed_m_s",
    self.apf_constant_descent_speed_m_s,
))
force_angle, surge_speed = self.apf_overtaking_close_turn_command(
    primary_encounter,
    nearest_obstacle_distance_m,
    force_angle,
    surge_speed,
)
u_cmd[0, 0] = float(np.clip(surge_speed, 0.0, self.v_max))
```

The final APF control output is

$$\mathbf{u}_{\text{APF}} = \begin{bmatrix} v_c & \omega_c \end{bmatrix}^T. \quad (54)$$

9 Processing chain

The complete processing chain is:

1. cluster the LiDAR point cloud using DBSCAN and estimate obstacle dimensions using PCA;

2. associate detections with obstacle tracks;
3. predict and correct each obstacle state with the four-state constant-velocity EKF;
4. compute obstacle velocity, TCPA, DCPA, and future positions;
5. classify the encounter using the COLREG bearing, course, and speed tests;
6. construct the current field and the EKF $2 \times \text{TCPA}$ virtual-field corridor;
7. reject route segments entering an APF ellipse and combine APF attraction, repulsion, and the locked COLREG side constraint;
8. convert the resultant-force direction into surge and yaw-rate commands; and
9. allocate the commanded force and moment to the two thrusters.

The vessel EKF supplies the vessel position, heading, and velocity at the start of this chain. The obstacle EKF supplies the predicted dynamic-obstacle state used by the APF.

10 Implementation index

Function	Source-code entry point
Vessel motion model	<code>motion_model</code>
Vessel EKF prediction	<code>extended_kalman_filter_predict</code>
Vessel EKF update	<code>extended_kalman_filter_update</code>
Pose measurement model	<code>h_pose_update</code>
Yaw-rate measurement model	<code>h_grate_update</code>
Obstacle CV propagation	<code>obstacle_ekf_predict</code>
Obstacle EKF process noise	<code>obstacle_ekf_process_noise</code>
Obstacle EKF measurement update	<code>obstacle_ekf_update</code>
Elliptical geometry	<code>ellipse_level_and_away</code>
Smooth elliptical repulsion	<code>smooth_ellipse_repulsion</code>
Goal and path attraction	<code>apf_goal_attraction_body,</code> <code>apf_path_attraction_body</code>
CPA computation	<code>apf_cpa_metrics</code>
COLREG zone classification	<code>classify_colreg_zone,</code> <code>apf_classify_encounter</code>
Virtual-field corridor	<code>update_apf_virtual_obstacles</code>
Route-ellipse intersection	<code>segment_intersects_ellipse</code>
Obstacle repulsion composition	<code>apf_repulsion_for_obstacle</code>
APF control output	<code>compute_apf_control</code>

A Complete implementation source chain

The preceding code blocks identify the relevant operations next to their equations. This appendix embeds the complete current definitions without elision. Together, these files contain the implementation chain for every method indexed above: the behaviour-tree dispatch, COLREG classification, shared APF geometry, the unified controller, and the three encounter-specific controllers.

A.1 Behaviour-tree dispatch

```
class BTStatus:
    SUCCESS = "success"
    FAILURE = "failure"
    RUNNING = "running"

class BTNode:
    def tick(self, blackboard):
        raise NotImplementedError

class BTCondition(BTNode):
    def __init__(self, predicate):
        self._predicate = predicate

    def tick(self, blackboard):
        return BTStatus.SUCCESS if self._predicate(blackboard) else BTStatus.FAILURE

class BTACTION(BTNode):
    def __init__(self, action):
        self._action = action

    def tick(self, blackboard):
        return self._action(blackboard)

class BTSequence(BTNode):
    def __init__(self, *children):
        self._children = children

    def tick(self, blackboard):
        for child in self._children:
            status = child.tick(blackboard)
            if status != BTStatus.SUCCESS:
                return status
        return BTStatus.SUCCESS

class BTSelector(BTNode):
    def __init__(self, *children):
        self._children = children

    def tick(self, blackboard):
        for child in self._children:
            status = child.tick(blackboard)
```

```

        if status != BTStatus.FAILURE:
            return status
    return BTStatus.FAILURE

```

A.2 Shared COLREG and APF geometry

```

"""Pure COLREG zone classification and the single APF repulsion law."""

import numpy as np

def merge_collinear_cluster_labels(
    points,
    labels,
    max_gap_m=0.50,
    max_width_m=0.65,
    max_length_m=4.0,
    min_aspect_ratio=1.5,
):
    """Merge nearby DBSCAN fragments that form one elongated obstacle hull."""
    points = np.asarray(points, dtype=float)
    labels = np.asarray(labels, dtype=int)
    if points.ndim != 2 or points.shape[1] != 2 or len(points) != len(labels):
        raise ValueError("points must be Nx2 and labels must have length N")

    groups = [[int(label)] for label in sorted(set(labels) - {-1})]

    def group_points(group):
        return points[np.isin(labels, group)]

    def can_merge(left, right):
        left_points = group_points(left)
        right_points = group_points(right)
        gap_m = float(np.min(np.linalg.norm(
            left_points[:, None, :] - right_points[None, :, :], axis=2
        )))
        if gap_m > float(max_gap_m):
            return False

        combined = np.vstack((left_points, right_points))
        centred = combined - np.mean(combined, axis=0)
        covariance = centred.T @ centred / max(len(combined) - 1, 1)
        eigenvalues, eigenvectors = np.linalg.eigh(covariance)
        axis = eigenvectors[:, int(np.argmax(eigenvalues))]
        normal = np.array([-axis[1], axis[0]])
        length_m = float(np.ptp(centred @ axis))
        width_m = float(np.ptp(centred @ normal))
        aspect_ratio = length_m / max(width_m, 0.05)
        return (
            length_m <= float(max_length_m)
            and width_m <= float(max_width_m)
            and aspect_ratio >= float(min_aspect_ratio)
        )

    merged = True
    while merged:

```

```

merged = False
for left_index in range(len(groups)):
    for right_index in range(left_index + 1, len(groups)):
        if not can_merge(groups[left_index], groups[right_index]):
            continue
        groups[left_index].extend(groups.pop(right_index))
        merged = True
        break
    if merged:
        break

result = labels.copy()
for merged_label, group in enumerate(groups):
    result[np.isin(labels, group)] = merged_label
return result

def straight_line_cpa(own_position, own_velocity, obstacle_position,
obstacle_velocity, horizon_s=None):
    """Continuous-time CPA for two constant-velocity trajectories."""
    own_position = np.asarray(own_position, dtype=float).reshape(2)
    own_velocity = np.asarray(own_velocity, dtype=float).reshape(2)
    obstacle_position = np.asarray(obstacle_position, dtype=float).reshape(2)
    obstacle_velocity = np.asarray(obstacle_velocity, dtype=float).reshape(2)
    relative_position = obstacle_position - own_position
    relative_velocity = obstacle_velocity - own_velocity
    relative_speed_sq = float(np.dot(relative_velocity, relative_velocity))
    tcpa_s = 0.0 if relative_speed_sq < 1e-12 else max(
        -float(np.dot(relative_position, relative_velocity)) / relative_speed_sq,
        0.0,
    )
    if horizon_s is not None:
        tcpa_s = min(tcpa_s, max(float(horizon_s), 0.0))
    own_cpa = own_position + own_velocity * tcpa_s
    obstacle_cpa = obstacle_position + obstacle_velocity * tcpa_s
    return tcpa_s, float(np.linalg.norm(obstacle_cpa - own_cpa)), own_cpa,
    obstacle_cpa

def smooth_undirected_axis(previous_axis, measured_axis, alpha):
    """Smooth a PCA axis after resolving its arbitrary 180-degree sign."""
    previous = np.asarray(previous_axis, dtype=float).reshape(2)
    measured = np.asarray(measured_axis, dtype=float).reshape(2)
    previous /= max(float(np.linalg.norm(previous)), 1e-12)
    measured /= max(float(np.linalg.norm(measured)), 1e-12)
    if np.dot(previous, measured) < 0.0:
        measured = -measured
    blended = (1.0 - float(alpha)) * previous + float(alpha) * measured
    return blended / max(float(np.linalg.norm(blended)), 1e-12)

def constrain_velocity_to_axis(velocity, axis, max_gap_rad):
    """Keep motion direction within max_gap_rad of an undirected hull axis."""
    velocity = np.asarray(velocity, dtype=float).reshape(2)
    axis = np.asarray(axis, dtype=float).reshape(2)
    speed = float(np.linalg.norm(velocity))
    axis_norm = float(np.linalg.norm(axis))
    if speed < 1e-12 or axis_norm < 1e-12:

```

```

        return velocity.copy()
    direction = velocity / speed
    axis = axis / axis_norm
    if np.dot(direction, axis) < 0.0:
        axis = -axis
    gap = float(np.arctan2(direction[0] * axis[1] - direction[1] * axis[0], np.dot(
    direction, axis)))
    correction = np.sign(gap) * max(abs(gap) - max(float(max_gap_rad), 0.0), 0.0)
    c, s = np.cos(correction), np.sin(correction)
    return speed * np.array([c * direction[0] - s * direction[1], s * direction[0] +
    c * direction[1]])

def obstacle_stern_waypoint(centre_ne, velocity_ne, clearance_m):
    """Point behind the obstacle along its EKF motion direction."""
    centre = np.asarray(centre_ne, dtype=float).reshape(2)
    velocity = np.asarray(velocity_ne, dtype=float).reshape(2)
    speed = float(np.linalg.norm(velocity))
    if not np.isfinite(centre).all() or not np.isfinite(velocity).all() or speed < 1
    e-9:
        return None
    return centre - max(float(clearance_m), 0.0) * velocity / speed

def classify_colreg_zone(
    bearing_deg,
    relative_heading_deg,
    emergency=False,
    own_speed_m_s=np.nan,
    obstacle_speed_m_s=np.nan,
    own_bearing_from_obstacle_deg=np.nan,
):
    """Classify by the obstacle's robot-relative sector, then its course."""
    bearing = float(bearing_deg) % 360.0
    heading = float(relative_heading_deg) % 360.0
    ahead = bearing >= 292.5 or bearing < 67.5
    astern = 112.5 <= bearing < 247.5
    on_port = 247.5 <= bearing < 337.5
    on_starboard = 22.5 <= bearing < 112.5
    same_course = heading < 67.5 or heading >= 292.5
    # Rule 13 must be deliberately stricter than the general ahead/same-course
    # sectors. Otherwise a crossing vessel can briefly look like a vessel
    # ahead when its measured bearing or heading jitters near a sector edge.
    # Rule 14 uses the original COLREG capture sector.
    head_on_ahead = bearing >= 292.5 or bearing < 67.5
    head_on_reciprocal = 157.5 <= heading < 202.5
    overtaking_ahead = bearing >= 340.0 or bearing < 20.0
    overtaking_same_course = heading < 20.0 or heading >= 340.0
    own_bearing_from_obstacle_deg = float(own_bearing_from_obstacle_deg) % 360.0
    own_in_obstacle_stern_sector = (
        np.isfinite(own_bearing_from_obstacle_deg)
        and 150.0 <= own_bearing_from_obstacle_deg < 210.0
    )
    own_speed_m_s = float(own_speed_m_s)
    obstacle_speed_m_s = float(obstacle_speed_m_s)
    speed_known = np.isfinite(own_speed_m_s) and np.isfinite(obstacle_speed_m_s)
    obstacle_lateral_speed_m_s = abs(
        obstacle_speed_m_s * np.sin(np.deg2rad(heading))

```

```

) if np.isfinite(obstacle_speed_m_s) else np.inf

if head_on_ahead and head_on_reciprocal:
    # Planner side -1 is route-right/starboard. Keep this aligned
    # with Rule 13 so both dynamic manoeuvres always pass right.
    return "head_on", -1.0, "COLREG Rule 14: both alter to starboard"

if ahead:
    if (
        speed_known
        and overtaking_ahead
        and overtaking_same_course
        and own_in_obstacle_stern_sector
        and own_speed_m_s > obstacle_speed_m_s + 0.08
        and obstacle_lateral_speed_m_s <= 0.06
    ):
        return "overtaking", -1.0, "COLREG Rule 13: overtake on starboard side"
    if 67.5 <= heading < 157.5:
        return "crossing_from_port", 0.0, "COLREG Rule 15: stand on"
    if 202.5 <= heading < 292.5:
        return "crossing_from_starboard", -1.0, "COLREG Rule 15: give way, pass
astern"

if astern and same_course and (not speed_known or own_speed_m_s <
obstacle_speed_m_s):
    return "being_overtaken", 0.0, "COLREG Rule 13: keep course and speed"

if on_port:
    return (
        "crossing_from_port",
        -1.0 if emergency else 0.0,
        "COLREG Rule 17: emergency starboard action" if emergency else "COLREG
Rule 17: stand on",
    )

if on_starboard:
    return "crossing_from_starboard", -1.0, "COLREG Rule 15: give way, pass
astern"

return "static_obstacle", 0.0, "none"

def smooth_ellipse_repulsion(level, away, gain, outer_scale):
    """Maximum at/inside the hull, C1-smooth to zero at the size multiple."""
    level = float(level)
    outer_scale = float(outer_scale)
    if not np.isfinite(level) or outer_scale <= 1.0 or level >= outer_scale:
        return np.zeros(2, dtype=float), False
    u = np.clip((level - 1.0) / (outer_scale - 1.0), 0.0, 1.0)
    fade = 1.0 - (3.0 * u * u - 2.0 * u * u * u)
    return float(gain) * fade * np.asarray(away, dtype=float).reshape(2), True

```

A.3 Unified vessel, EKF, APF, and route-planning controller

```

"""
Copyright (c) 2025 The uos_sess6072_build Authors.

```

```

Authors: Blair Thornton, Alec O'Loughlin, Miquel Massot
All rights reserved.
Licensed under the BSD 3-Clause License.
See LICENSE.md file in the project root for full license information.
"""

import numpy as np
import json
import os
from datetime import datetime
import time
from pathlib import Path
import subprocess
import platform
import copy

from colreg_apf import constrain_velocity_to_axis, merge_collinear_cluster_labels,
    smooth_undirected_axis, straight_line_cpa

from zeroros import Publisher, Subscriber
from zeroros.messages import String, Vector3, Vector3Stamped, Pose, PoseStamped,
    RBLaserScan
from zeroros.datalogger import DataLogger

from drivers.aruco import ArUcoUDPDriver
from drivers.rpi import Console, Rate
from drivers import __version__
from scipy.spatial.transform import Rotation as R

# ----- LiDAR + DBSCAN imports -----
os.environ.setdefault("LOKY_MAX_CPU_COUNT", "1")
from sklearn.cluster import DBSCAN

from model_sess6072 import TAM, Vehicle2D_e, dynamics_translation_e,
    dynamics_rotation_e, TrajectoryGenerate, RangeAngleKinematics
from math_sess6072 import l2m, HomogeneousTransformation, Vector,
    HomogeneousTransformation, Matrix, Identity
from model_sess6072 import rigid_body_kinematics # tried to remove <existing
    libraries>
from math_sess6072 import Inverse, Vector # tried to remove <existing libraries>

# enter additional library imports here

# define global variables
N = 0
E = 1
G = 2
DOTN = 3
DOTE = 4
DOTG = 5

# Obstacle ship EKF tracking and CPA switch.
# True: enable EKF tracking, CPA, predicted trajectories, and virtual collision
    points.
# False: use current LiDAR obstacles only; disable EKF tracking, CPA, and prediction.

ENABLE_OBSTACLE_EKF_PREDICTION = True

```

```

# APF range is always derived from each DBSCAN cluster's PCA dimensions.
ENABLE_CLUSTER_BASED_APF_RANGE = True

# define global functions
def get_wifi_name():
    result = subprocess.run(["netsh", "wlan", "show", "interfaces"], capture_output=
    True, text=True)
    for line in result.stdout.split("\n"):
        if "SSID" in line and "BSSID" not in line:
            return line.split(":")[1].strip()
    return 0

def Vector(dim): return np.zeros((dim, 1), dtype=float)

def rpm2N(x, fwd_lim = 2000, rev_lim = -2000):
    tol = 10
    if fwd_lim is not None and x>fwd_lim: x=fwd_lim
    if rev_lim is not None and x<rev_lim: x=rev_lim
    if abs(x) <= tol: return 0
    elif x>tol: return 1.541571428571430076E-7*x**2+3.293357142857142252E-4*x
    -1.401428571428424679E-3
    else: return -7.3574999999999954E-8*x**2+1.71674999999999581E-4*x
    -1.054478382732365536E-16

def N2rpm(x, fwd_lim = 1.2753, rev_lim = -0.63765):
    tol = 10E-4
    if x>fwd_lim: x=fwd_lim
    if x<rev_lim: x=rev_lim
    if abs(x) <= tol: return 0
    elif x>tol: return -5.727416623043567370E2*x**2+2.268233085499708977E3*x
    +2.958718669408357371E1
    else: return 2.397948698765131667E3*x**2+4.665568885752373717E3*x -
    -6.685183692128883879E-14

def force_to_rpm_unbounded(force_n):
    """Invert the propeller calibration without its fitted-range clamp."""
    force_n = float(force_n)
    if abs(force_n) <= 1e-3:
        return 0.0

    if force_n > 0.0:
        a, b, c = (
            1.541571428571430076E-7,
            3.293357142857142252E-4,
            -1.401428571428424679E-3,
        )
    else:
        a, b, c = (
            -7.3574999999999954E-8,
            1.71674999999999581E-4,
            -1.054478382732365536E-16,
        )

    discriminant = b * b - 4.0 * a * (c - force_n)
    if discriminant < 0.0:
        return 0.0
    roots = (

```

```

        (-b + np.sqrt(discriminant)) / (2.0 * a),
        (-b - np.sqrt(discriminant)) / (2.0 * a),
    )
    matching_roots = [root for root in roots if np.sign(root) == np.sign(force_n)]
    return float(min(matching_roots, key=abs)) if matching_roots else 0.0

# Keep heading errors continuous for route tracking.
def wrap_angle(a):
    return (a + np.pi) % (2 * np.pi) - np.pi

def cluster_principal_dimensions(points, min_pc1_m, min_pc2_m):
    points = np.asarray(points, dtype=float).reshape(-1, 2)
    centre = np.mean(points, axis=0)
    relative = points - centre

    if len(points) < 2 or np.allclose(relative, 0.0):
        return centre, float(min_pc1_m), float(min_pc2_m), np.array([1.0, 0.0])

    covariance = relative.T @ relative / max(len(relative), 1)
    _, eigenvectors = np.linalg.eigh(covariance)
    length_axis = eigenvectors[:, -1]
    if length_axis[0] < 0.0:
        length_axis = -length_axis
    width_axis = np.array([-length_axis[1], length_axis[0]], dtype=float)

    pc1_m = max(float(np.ptp(relative @ length_axis)), float(min_pc1_m))
    pc2_m = max(float(np.ptp(relative @ width_axis)), float(min_pc2_m))
    if pc2_m > pc1_m:
        pc1_m, pc2_m = pc2_m, pc1_m
        length_axis = width_axis

    return centre, pc1_m, pc2_m, length_axis

def closest_point_on_segment(point, start, end):
    point = np.asarray(point, dtype=float).reshape(2)
    start = np.asarray(start, dtype=float).reshape(2)
    end = np.asarray(end, dtype=float).reshape(2)
    segment = end - start
    segment_length_sq = float(np.dot(segment, segment))
    if segment_length_sq < 1e-12:
        return start.copy()
    ratio = float(np.clip(np.dot(point - start, segment) / segment_length_sq, 0.0,
    1.0))
    return start + ratio * segment

def ellipse_level_and_away(offset, length_axis, semi_length_m, semi_width_m):
    offset = np.asarray(offset, dtype=float).reshape(2)
    length_axis = np.asarray(length_axis, dtype=float).reshape(2)
    axis_norm = float(np.linalg.norm(length_axis))
    if axis_norm < 1e-9:
        length_axis = np.array([1.0, 0.0], dtype=float)
    else:
        length_axis = length_axis / axis_norm
    width_axis = np.array([-length_axis[1], length_axis[0]], dtype=float)
    semi_length_m = max(float(semi_length_m), 1e-6)
    semi_width_m = max(float(semi_width_m), 1e-6)

    along = float(np.dot(offset, length_axis))

```



```

across = float(np.dot(offset, width_axis))
level = float(np.hypot(along / semi_length_m, across / semi_width_m))
gradient = (
    along / (semi_length_m * semi_length_m) * length_axis
    + across / (semi_width_m * semi_width_m) * width_axis
)
gradient_norm = float(np.linalg.norm(gradient))
if gradient_norm < 1e-9:
    offset_norm = float(np.linalg.norm(offset))
    away = offset / offset_norm if offset_norm >= 1e-9 else -length_axis
else:
    away = gradient / gradient_norm
return level, away

def extended_kalman_filter_predict(mu, Sigma, u, f, Q, dt):
    # (1) Project the state forward
    pred_mu, F = f(mu, u, dt)

    # (2) Project the error forward:
    pred_Sigma = F@Sigma@F.T+Q

    # Return the predicted state and the covariance
    return pred_mu, pred_Sigma

def extended_kalman_filter_update(mu, Sigma, z, h, R, wrap_index = None):

    # Prepare the estimated measurement
    pred_z, H = h(mu)

    # (3) Compute the Kalman gain
    K = Sigma@ H.T@ Inverse(H@Sigma@H.T + R)

    # (4) Compute the updated state estimate
    delta_z = z- pred_z
    if wrap_index != None: delta_z[wrap_index] = (delta_z[wrap_index] + np.pi) % (2
* np.pi) - np.pi
    cor_mu = mu + K@(delta_z)

    # (5) Compute the updated state covariance
    cor_Sigma = (Identity(mu.shape[0]) - K @ H) @ Sigma

    # Return the state and the covariance
    return cor_mu, cor_Sigma

def h_pose_update(x):
    est_measurement = Vector(6)
    est_measurement[N] = x[N]
    est_measurement[E] = x[E]
    est_measurement[G] = x[G]
    H = Matrix(6,6)
    H[N, N] = 1
    H[E, E] = 1
    H[G, G] = 1
    return est_measurement, H

def h_grate_update(x):
    est_measurement = Vector(6)
    est_measurement[DOTG] = x[DOTG]

```

```

H=Matrix(6,6)
H[DOTG,DOTG]=1
return est_measurement, H

# main class
class _ControllerCore:
    def __init__(self, OPERATING_MODE):

        ##### DEFINE ARUCO MARKER ID #####
        MARKER_ID = 24 # <<< CHANGE TO YOUR ROBOT'S ARUCO ID

        ##### SET NETWORK CONDITIONS #####
        if OPERATING_MODE != 2: # robot
            self.robot_ip = "192.168.10.1"
            self.robot_available = False
            self.sim_init = False
            aruco_params = {
                "port": 50001, # Port to listen to (DO NOT CHANGE)
                "marker_id": MARKER_ID, # Marker ID to listen to
            }
            if platform.system() == "Windows": wifi_name = get_wifi_name()
            else: wifi_name = "SmartCatXX"

        elif OPERATING_MODE == 2: # webots
            self.robot_ip = "127.0.0.1"
            aruco_params = {
                "port": 50000, # Port to listen to (DO NOT CHANGE)
                "marker_id": 0, # Override for WEBOTS (DO NOT CHANGE)
            }
            wifi_name = "WEBOTS"
            self.sim_init = True # Deal with webots timestamps

        self.sim_time_offset = 0.0

        Console.info("Connecting to:", self.robot_ip, "")
        if wifi_name:
            Console.info(f"You are connected to {wifi_name}")
        else:
            Console.info("No WiFi connection detected")

        # store operating mode
        self.OPERATING_MODE = OPERATING_MODE
        self.obstacle_ekf_prediction_enabled = ENABLE_OBSTACLE_EKF_PREDICTION
        Console.info(
            "Obstacle EKF prediction:",
            "enabled" if self.obstacle_ekf_prediction_enabled else "disabled",
        )

        ##### INITIALISE DATA LOGS #####
        filename_time = datetime.now().strftime("%Y%m%d_%H%M%S")
        self.run_dir = Path("logs/run_" + filename_time)
        self.run_dir.mkdir(parents=True, exist_ok=True)
        self.filename = self.run_dir / f"log_{filename_time}.csv"
        self.obstacle_log_dir = self.run_dir
        self.last_obstacle_snapshot_stamp_s = None
        self.webots_obstacle_truth = []
        self.webots_truth_track_gate_m = 2.5

```

```

self.webots_robot_truth_ne = None
self.webots_robot_truth_yaw_rad = None
self.webots_collision_detected = False
self.webots_collision_record_written = False

with self.filename.open('w') as f:
    f.write("EpochTime(s),TimeFromStart(s),right_prop_rate(rad/s),
left_prop_rate(rad/s),LastDT(s),Yaw(rad),North(m),East(m),IMUSensedYawRate(rad/s
),IMUIntegratedYaw(rad),IMUSensedTimeStamp(s),ARUCOSensedNorth(m),
ARUCOSensedEast(m),ARUCOSensedYaw(rad),ArucoSensedTimeStamp(s),DepthTimeStamp(s),
Depth(m),NavigationMode,APFEncounter,APFSide,APFD CPA(m),APFTCPA(s),APFForceX,
APFForceY,NearestObstacleNorth(m),NearestObstacleEast(m),NearestObstacleDistance
(m)\n")
    global file
    file = self.filename

##### ENTER WAYPOINT VARIABLES #####
# Start waypoint: (North, East) in metres
start_north, start_east = 0, 1

# Goal waypoint: (North, East) in metres
goal_north, goal_east = 15, 1

north_path = [start_north, goal_north]
east_path = [start_east, goal_east]

self.waypoints = []

for i in range(len(north_path)):
    waypoint = Vector3()

    waypoint.y = north_path[i]
    waypoint.x = east_path[i]

    self.waypoints.append(waypoint)
    print("WAYPOINTS: ", self.waypoints)
    print("WAYPOINTS TYPE: ", type(self.waypoints))

##### INITIALISE ROBOT VARIABLES #####
rate = 5.0 # Hz
self.r = Rate(rate)
self.lastdt = 1/rate
self.starttime = time.time()
self.timefromstart = None
self.prev_sensed = None # originally None

self.sensed_imu_yaw_rate_rad_s = None
self.sensed_imu_stamp_s = None
self.sensed_imu_prev_stamp_s = None
self.sensed_yaw_rate = None
self.integrated_yaw = 0

self.sensed_pos_northings_m = None
self.sensed_pos_eastings_m = None
self.sensed_pos_yaw_rad = None
self.sensed_pos_stamp_s = None
self.sensed_bottom_depth_m = None
self.sensed_bottom_depth_stamp_s = None

```

```

# ----- LiDAR definitions -----
self.lidar_data = None
self.lidar_data_rb = None
self.lidar_timestamp_s = None
self.latest_lidar_received_s = None
self.lidar_beam_stamps_s = None
self.lidar_scan_time_s = 0.0
self.lidar_reference_pose = None
self.lidar_pose_extrapolation_limit_s = 0.5
self.lidar_time_sync_std_s = 0.02
self.lidar_new = False
self.lidar_x_bl = 0.1
self.lidar_y_bl = 0.0
self.lidar_gamma_bl = 0.0
self.lidar = RangeAngleKinematics(self.lidar_x_bl, self.lidar_y_bl, self.
lidar_gamma_bl)

# ----- LiDAR DBSCAN parameters and outputs -----
self.lidar_dbscan_eps_m = 0.20
self.lidar_dbscan_min_points = 3
self.lidar_fragment_merge_gap_m = 0.50
self.lidar_fragment_max_width_m = 0.65
self.lidar_fragment_max_length_m = 4.0
self.lidar_points_body = np.empty((0, 2))
self.lidar_points_ne = np.empty((0, 2))
self.lidar_cluster_labels = np.array([], dtype=int)
self.lidar_obstacles = []
self.lidar_obstacle_centres_body = np.empty((0, 2))
self.lidar_obstacle_centres_ne = np.empty((0, 2))
self.lidar_obstacle_distances_m = np.array([])
self.lidar_obstacle_angles_rad = np.array([])
self.nearest_lidar_obstacle = None

# ----- LiDAR sector parameters -----
self.front_angle_limit = np.deg2rad(25)
self.front_block_threshold = 0.8
self.min_front_close_beams = 5
self.side_angle_min = np.deg2rad(45)
self.side_angle_max = np.deg2rad(120)

self.map_observation_class = "unknown"
self.front_clearance_m = np.inf
self.left_clearance_m = np.inf
self.right_clearance_m = np.inf

# ----- Navigation parameters -----
self.start_ne = np.array([start_north, start_east], dtype=float)
self.goal_ne = np.array([goal_north, goal_east], dtype=float)
self.goal_tolerance_m = 0.40
self.navigation_mode = "track"
self.goal_reached = False
self.route_path_vec_ne = self.goal_ne - self.start_ne
self.route_path_length_m = float(np.linalg.norm(self.route_path_vec_ne))
if self.route_path_length_m > 1e-9:
    self.route_path_unit_ne = self.route_path_vec_ne / self.
route_path_length_m
else:

```

```

        self.route_path_unit_ne = np.array([1.0, 0.0], dtype=float)
        self.route_heading_rad = float(np.arctan2(self.route_path_unit_ne[1], self.
route_path_unit_ne[0]))
        self.route_tracking_speed_m_s = 0.65 if self.OPERATING_MODE == 2 else 0.35
        self.route_tracking_lookahead_m = 1.0 if self.OPERATING_MODE == 2 else 0.7
        self.final_approach_distance_m = 1.0 if self.OPERATING_MODE == 2 else 0.7
        self.final_slowdown_distance_m = 1.0 if self.OPERATING_MODE == 2 else 0.8
        self.final_heading_slow_angle_rad = np.deg2rad(75.0)
        self.max_heading_deviation_rad = np.deg2rad(60.0)
        self.heading_deviation_guard_rad = np.deg2rad(55.0)
        self.heading_deviation_return_gain = 3.0

# ----- APF parameters -----
self.apf_cluster_range_enabled = ENABLE_CLUSTER_BASED_APF_RANGE
self.apf_risk_pc_scale = 2.0
self.apf_avoidance_pc_scale = 2.0
self.apf_direction_pc_scale = 2.0
self.apf_virtual_pc_scale = 3.0
self.apf_activation_front_half_angle_rad = np.deg2rad(150.0)
self.apf_priority_front_half_angle_rad = np.deg2rad(90.0)
self.apf_goal_gain = 5.5
self.apf_path_gain = 5.5
self.apf_repulsive_gain = 0.08
self.apf_attraction_saturation_m = 3.5
self.apf_path_threshold_m = 0.10
self.apf_route_lookahead_m = 2.1 if self.OPERATING_MODE == 2 else 1.8
self.apf_clearance_gain = 2.3
self.apf_collision_horizon_s = 6.0 if self.OPERATING_MODE != 2 else 10.0
# COLREG uses the short-horizon EKF direction only after its heading
# estimate is sufficiently consistent. This prevents a noisy LiDAR
# displacement from promoting a crossing vessel to an overtaking one.
self.apf_colreg_prediction_horizon_s = 1.0
self.apf_colreg_max_heading_std_rad = np.deg2rad(30.0)
self.apf_prediction_dt_s = 0.5
self.apf_heading_gain = 0.9
self.apf_heading_step_limit_rad = np.deg2rad(60.0)
self.apf_min_forward_speed = 0.06
self.apf_overtaking_close_distance_m = 3.0
self.apf_overtaking_close_turn_angle_rad = np.deg2rad(55.0)
self.apf_overtaking_turn_release_angle_rad = np.deg2rad(40.0)
self.apf_overtaking_close_turn_speed_m_s = 0.15 if self.OPERATING_MODE == 2
else 0.10
# Fixed-step APF descent: the potential-field gradient determines only
# the travel direction while avoidance uses a constant surge speed.
self.apf_constant_descent_speed_m_s = self.route_tracking_speed_m_s
self.apf_force_body = np.zeros(2, dtype=float)
self.apf_repulsive_force_body = np.zeros(2, dtype=float)
self.apf_attractive_force_body = np.zeros(2, dtype=float)
self.apf_steering_force_body = np.zeros(2, dtype=float)
self.apf_target_ne = np.array([np.nan, np.nan], dtype=float)
self.apf_encounter_mode = "none"
self.apf_colreg_rule = "none"
self.apf_avoidance_side_sign = 0.0
self.apf_colreg_dcpa_m = np.nan
self.apf_colreg_tcpa_s = np.nan
self.apf_colreg_active = False
# Preserve Rule 14 until the detected encounter has cleared.
self.apf_head_on_latched = False

```

```

# Rule 13 uses the same encounter persistence, but always fixes the
# manoeuvre to route-right/starboard.
self.apf_overtaking_latched = False
# A target is only treated as dynamic after its EKF motion estimate has
# remained coherent for several observations. The previous 0.03 m/s
# threshold allowed cluster jitter to change the COLREG encounter type.
self.apf_dynamic_speed_threshold_m_s = 0.06 if self.OPERATING_MODE == 2 else
0.05
self.apf_dynamic_exit_speed_threshold_m_s = 0.04 if self.OPERATING_MODE == 2
else 0.03
self.apf_track_association_m = 0.80 if self.OPERATING_MODE == 2 else 0.60
self.apf_track_timeout_s = 1.5 if self.OPERATING_MODE == 2 else 1.0
self.obstacle_track_confirmation_hits = 3
self.apf_next_track_id = 1
self.apf_obstacle_tracks = []
self.apf_obstacle_track_candidates = []
self.apf_virtual_obstacles = []
self.obstacle_ekf_measurement_std_m = 0.08 if self.OPERATING_MODE == 2 else
0.12
self.obstacle_ekf_accel_std_m_s2 = 0.20 if self.OPERATING_MODE == 2 else
0.35
self.obstacle_ekf_initial_position_std_m = 0.20
self.obstacle_ekf_initial_velocity_std_m_s = 0.35
self.obstacle_heading_hold_speed_m_s = 0.03
self.obstacle_axis_smoothing_alpha = 0.20
self.obstacle_axis_min_aspect_ratio = 1.4
self.obstacle_axis_max_velocity_gap_rad = np.deg2rad(30.0)
self.obstacle_prediction_horizon_s = 15.0
self.obstacle_prediction_step_s = 0.5
self.obstacle_history_len = 60
self.obstacle_stats_window_s = 2.5
self.obstacle_prediction_min_samples = 3
self.obstacle_prediction_min_hits = 3
self.obstacle_prediction_min_time_span_s = 0.8
self.obstacle_prediction_max_speed_std_m_s = 0.10
self.obstacle_prediction_max_heading_var_rad2 = np.deg2rad(35.0) ** 2
self.obstacle_prediction_accel_min_samples = 10
self.obstacle_prediction_max_accel_std_m_s2 = 0.25
self.obstacle_min_pc1_m = 0.36 if self.OPERATING_MODE == 2 else 0.30
self.obstacle_min_pc2_m = 0.20 if self.OPERATING_MODE == 2 else 0.16
self.obstacle_min_equivalent_radius_m = 0.5 * self.obstacle_min_pc1_m
self.obstacle_max_accel_m_s2 = 0.80 if self.OPERATING_MODE == 2 else 0.60
self.apf_own_equivalent_radius_m = 0.30 if self.OPERATING_MODE == 2 else
0.25
self.apf_virtual_repulsive_gain = 0.12
self.apf_side_lock_sign = 0.0
self.apf_side_lock_until_s = 0.0
self.apf_side_lock_s = 5.0 if self.OPERATING_MODE != 2 else 8.0
self.apf_side_lock_exit_level = 1.15
self.apf_side_lock_active = False
self.apf_visual_hold_s = 2.0
self.apf_visual_hold_until_s = 0.0
# Encounter-specific APF tuning profiles. Crossing keeps the original
# values exactly so its avoidance trajectory is unchanged; overtaking
# and head-on get independent parameter sets in this file for isolated
tuning.
self.apf_crossing_params = self.apf_build_encounter_params()
self.apf_overtaking_params = self.apf_build_encounter_params()

```

```

self.apf_head_on_params = self.apf_build_encounter_params()
self.apf_active_profile_name = "crossing"
Console.info(
    "APF influence range:",
    "PCA ellipse 2.5x current / 4x EKF-predicted",
)

self.initial_state = Vector(6)
self.initial_state[N] = start_north
self.initial_state[E] = start_east
self.initial_state[G] = 0
self.initial_state[DOTN] = 0
self.initial_state[DOTE] = 0
self.initial_state[DOTG] = 0

self.North = self.initial_state[N][0]
self.East = self.initial_state[E][0]
self.Yaw = self.initial_state[G][0]

self.right_rate = 0
self.left_rate = 0

##### MOTION MODEL VARIABLES #####
# Body-force model: positive force from either thruster acts forward.
# The right propeller command sign is handled at the RPM conversion.
phi=l2m([0,0])
x=l2m([0,0])
# Body-frame lateral offsets: right thruster is negative y, left is positive
y.
y=l2m([-0.09,0.09])

self.G=TAM(phi,x,y)
print('G = ',self.G)

# hull, water properties
rho = 1000 # density of water in kg/m3
draft = 0.07 #m
beam = 0.04 #m of the immersed hull section
length = 0.5 #m
width = 0.4 #m # of the whole hull

# from ESDU 71016. Fluid forces, pressures and moments on rectangular blocks.
ESDU 71016 ESDU International, London
CD = 7#1.5 # approximation for block from Newman (0.9 to 2.75)
A = 2*beam*draft #catamaran cross section in surge
k_drag = 0.5*rho*CD*A

# Added mass from Imlay 1961, Technical Report DTMB - assuming a prolate
spheroid
mass = 3
e = 1 - (beam/length)**2
alpha = (2*(1-e**2)/e**3)*(0.5*np.log((1+e)/(1-e))-e) #note np.log() = ln(),
np.log10()=log()

mass_add = 2*alpha*mass/(2-alpha) # kg of water pushed by hull with, note
this is for an infinite

m_tot = mass + mass_add

```

```

    # Added mass from Imlay 1961, Technical Report DTMB - assuming a prolate
    spheroid
    I_66 = mass*((length/2)**2+(width/2)**2)/4 # rough approximation as
    rectangle
    e = 1 - (beam/length)**2
    alpha = (2*(1-e**2)/e**3)*(0.5*np.log((1+e)/(1-e))-e) #note np.log() = ln(),
    np.log10()=log()
    beta = 1/e**2 - ((1-e**2)/(2*e**3)) * np.log((1+e)/(1-e)) # kg of water
    pushed by hull with, note this is for an infinite

    I66_add = 2*(1/5)*mass*((draft**2-length**2)**2*(alpha-beta)/(2*(draft**2-
    length**2)+(draft**2+length**2)/(beta-alpha)))

    I_tot = I_66+I66_add

    # drag B_66
    B_66 = 0.12#0.12

    self.initial_pose = True # Set false after pose is initialised

    # read these into our vehicle class
    self.robot = Vehicle2D_e(m_tot,I_tot,k_drag,B_66)
    self.robot.info()

    self.v_robot = Vector(3) # initially stationary velocity vector in e frame
    self.p_robot = Vector(3); self.p_robot[0] = start_north; self.p_robot[1] =
    start_east; self.p_robot[2] = np.deg2rad(0) # pose in the e frame

    ##### CONTROL VARIABLES #####
    # Setup control parameters
    #####
    tau_s = 2 #s to remove along track error # 0.5
    self.L = 0.3#m distance to remove normal and angular error
    self.ks = 1/tau_s
    self.kn = None
    self.kg = None

    self.v_max = 0.85 #fastest the robot can go # 0.2
    self.w_max = np.deg2rad(80) #fastest the robot can turn # 30
    self.prop_rate_limit_rad_s = 200.0
    # setup a contranor to store controls
    self.U = Vector(2).T
    #####
    # Setup trajectory
    #####
    v = self.route_tracking_speed_m_s
    a = 0.4 # 0.1
    self.s = TrajectoryGenerate(north_path,east_path)
    self.s.path_to_trajectory(v, a)

    # Generate turning arcs trajectory
    self.arc_radius = 0.02
    self.s.turning_arcs(self.arc_radius)
    self.s.wp_id = len(self.s.P_arc) - 1
    self.trajectory_duration_s = float(self.s.Tp_arc[-1][0])

```



```

##### EKF VARIABLES #####
# State x = [N, E, G, Ndot, Edot, Gdot]^T
self.mu = Vector(6)
self.mu[N] = self.initial_state[N]
self.mu[E] = self.initial_state[E]
self.mu[G] = self.initial_state[G]
self.mu[DOTN] = self.initial_state[DOTN]
self.mu[DOTE] = self.initial_state[DOTE]
self.mu[DOTG] = self.initial_state[DOTG]

# Initial covariance
self.Sigma = Identity(6)
# Position uncertainty (m^2)
self.Sigma[N, N] = 0.01 # 0.1 m std
self.Sigma[E, E] = 0.01
# Heading uncertainty (rad^2)
self.Sigma[G, G] = np.deg2rad(5.0)**2
# Velocity uncertainty ((m/s)^2 and (rad/s)^2)
self.Sigma[DOTN, DOTN] = 0.01
self.Sigma[DOTE, DOTE] = 0.01
self.Sigma[DOTG, DOTG] = np.deg2rad(10.0)**2

# Process noise Q (very simple diagonal)
self.Q = Identity(6)
q_pos = 1e-4
q_vel = 1e-3
self.Q[N, N] = q_pos
self.Q[E, E] = q_pos
self.Q[G, G] = 1e-5
self.Q[DOTN, DOTN] = q_vel
self.Q[DOTE, DOTE] = q_vel
self.Q[DOTG, DOTG] = 1e-4

# Measurement noise for ArUco pose (N, E, G)
self.R_pose = Identity(6)
self.R_pose[N, N] = 0.02**2 # 2 cm std
self.R_pose[E, E] = 0.02**2
self.R_pose[G, G] = np.deg2rad(2.0)**2 # 2 deg std

# Measurement noise for IMU yaw rate (Gdot)
self.R_grate = Identity(6)
self.R_grate[DOTG, DOTG] = np.deg2rad(1.0)**2

# Time bookkeeping for EKF (not strictly needed, but handy)
self.last_nav_t = self.starttime

##### DECLARE PUBLISHERS AND SUBSCRIBERS #####
self.control_pub = Publisher("/control", Vector3, ip=self.robot_ip)
self.config_pub = Publisher("/config", String, ip=self.robot_ip)
self.imu_sub = Subscriber("/imu", Vector3, self.imu_cb, ip=self.robot_ip)
self.sonar_sub = Subscriber("/sonar", Vector3, self.sonar_cb, ip=self.
robot_ip)
self.lidar_sub = Subscriber("/lidar", RBLaserScan, self.lidar_callback, ip=
self.robot_ip)
self.collission_sub = (
    Subscriber("/collission", Vector3, self.collission_cb, ip=self.robot_ip)
    if OPERATING_MODE == 2
    else None

```

```

)
self.console_sub = Subscriber("/command", String, self.command_cb, ip=self.
robot_ip)
self.aruco_driver = ArUcoUDPDriver(aruco_params, parent=self)
# a callback only used by WEBOTS to fake Aruco readings
self.groundtruth_sub = Subscriber("/groundtruth", PoseStamped, self.
groundtruth_callback, ip=self.robot_ip)
self.webots_obstacle_truth_sub = Subscriber(
    "/webots_obstacle_groundtruth",
    PoseStamped,
    self.webots_obstacle_groundtruth_callback,
    ip=self.robot_ip,
)
self.pseudo_aruco_counter = 0

##### CONNECT TO ROBOT #####
if OPERATING_MODE != 2: # not a simulation
    # waits for robot to respond to configure
    count = 0
    Console.info("Connecting to robot")
    while not self.robot_available:
        self.config_pub.publish(String("Configure"))
        time.sleep(1.0)
        count += 1
    time.sleep(5.0)
else: # WEBOTS create fake ARUCO logs
    self.sensed_imu_stamp_s = 0
    self.groundtruth_log = self.run_dir / f"log_{filename_time}_pseudo_aruco.
csv"
    with self.groundtruth_log.open('w') as f:
        f.write("epoch [s],elapsed [s],x [m],y [m],z [m],roll [deg],pitch [
deg],yaw [deg],broadcast\n")

##### INITIALISE THRUSTERS #####
for i in range(10): # rad/s
    self.control_pub.publish(Vector3())
    self.r.sleep()
    self.initialise_pose = True # Will set to false once the pose is
initialised

##### Setup EXIT key if show_laptop not used #####
if OPERATING_MODE == 0: # robot without show_laptop - stopped via <Ctrl+C>
    while True:
        try:
            self.loop()
        except KeyboardInterrupt:
            Console.info("Ctrl+C pressed. Stopping...")
            if self.OPERATING_MODE == 0:
                self.imu_sub.stop()
                self.sonar_sub.stop()
                self.lidar_sub.stop()
                if self.collision_sub is not None:
                    self.collision_sub.stop()
            break
        self.r.sleep()
##### END OF INITIALISATION #####

```

```

##### DEFINE FUNCTIONS HERE #####
def stopcommand(self):
    Console.info("Thrusters stopping")
    control_msg = Vector3() # initially 0
    for i in range(10):
        self.control_pub.publish(control_msg)
        self.imu_sub.stop()
        self.r.sleep()
    self.sonar_sub.stop()
    self.lidar_sub.stop()
    if self.collision_sub is not None:
        self.collision_sub.stop()
    self.webots_obstacle_truth_sub.stop()
    Console.info("Thrusters stopped")
    Console.info("Data saved in ",self.filename)
    self.r.sleep()

##### DEFINE CALLBACKS HERE #####
def imu_cb(self, msg: Vector3):
    self.sensed_imu_yaw_rate_rad_s = msg.z
    self.sensed_imu_stamp_s = time.time()
    self.robot_available = True

def sonar_cb(self,msg: Vector3):
    self.sensed_bottom_depth_m = msg.z/1000
    self.sensed_bottom_depth_stamp_s = time.time()
    self.robot_available = True

def collision_cb(self, msg: Vector3):
    detected = bool(msg.y)
    if self.webots_collision_record_written and detected == self.
webots_collision_detected:
        return

    self.webots_collision_detected = detected
    self.webots_collision_record_written = True
    payload = {
        "source": "webots_touch_sensor",
        "obstacle_model": "ShipObstacle",
        "sensor_available": True,
        "detected": detected,
        "first_contact_time_s": float(msg.z) if detected and msg.z >= 0.0 else
None,
    }
    with (self.run_dir / "webots_collision.json").open("w") as stream:
        json.dump(payload, stream, indent=2)

def webots_obstacle_groundtruth_callback(self, msg: PoseStamped):
    pose = msg.pose
    try:
        # LiDAR/obstacle tracking uses Webots x,-y as North,East.
        position_ne = [float(pose.position.x), -float(pose.position.y)]
        q = [pose.orientation.x, pose.orientation.y, pose.orientation.z, pose.
orientation.w]
        yaw_rad = float(R.from_quat(q).as_euler("xyz")[2])
    except (TypeError, ValueError):
        return
    references = []

```

```

for track in self.apf_obstacle_tracks:
    state = np.asarray(track.get("state", []), dtype=float).reshape(-1)
    if len(state) >= 2 and np.isfinite(state[:2]).all():
        references.append(state[:2])
if not references:
    references = [
        np.asarray(obstacle.get("centre_ne"), dtype=float).reshape(2)
        for obstacle in self.lidar_obstacles
        if obstacle.get("centre_ne") is not None
    ]
if references and min(
    float(np.linalg.norm(np.asarray(position_ne) - reference))
    for reference in references
) > self.webots_truth_track_gate_m:
    return
if self.webots_obstacle_truth:
    previous_ne = np.asarray(self.webots_obstacle_truth[-1]["position_ne"],
dtype=float)
    if np.linalg.norm(np.asarray(position_ne) - previous_ne) > 1.0:
        self.webots_obstacle_truth.clear()
self.webots_obstacle_truth.append({
    "t": float(self.timefromstart or 0.0),
    "position_ne": position_ne,
    "heading_rad": yaw_rad,
})

def command_cb(self, msg: String):
    Console.info(f"Response from robot: {msg.data}")

# ----- LiDAR callback -----
def lidar_callback(self, msg: RBLaserScan):
    if self.sim_init:
        self.sim_time_offset = time.time() - msg.header.stamp
        self.sim_init = False

    self.lidar_timestamp_s = msg.header.stamp + self.sim_time_offset
    self.latest_lidar_received_s = self.lidar_timestamp_s

    ranges = np.array(msg.ranges, dtype=float)
    angles = np.array(msg.angles, dtype=float)

    if len(ranges) != len(angles):
        count = min(len(ranges), len(angles))
        ranges = ranges[:count]
        angles = angles[:count]

    ranges = np.where((ranges > 0.0) & np.isfinite(ranges), ranges, np.nan)
    self.lidar_data_rb = np.column_stack([ranges, angles])

    time_increment_s = float(getattr(msg, "time_increment", 0.0) or 0.0)
    scan_time_s = float(getattr(msg, "scan_time", 0.0) or 0.0)
    if not np.isfinite(time_increment_s) or time_increment_s < 0.0:
        time_increment_s = 0.0
    if not np.isfinite(scan_time_s) or scan_time_s < 0.0:
        scan_time_s = 0.0
    if time_increment_s <= 0.0 and len(ranges) > 1 and scan_time_s > 0.0:
        time_increment_s = scan_time_s / (len(ranges) - 1)
    self.lidar_scan_time_s = max(

```

```

        scan_time_s,
        time_increment_s * max(len(ranges) - 1, 0),
    )
    self.lidar_beam_stamps_s = (
        self.lidar_timestamp_s
        + time_increment_s * np.arange(len(ranges), dtype=float)
    )

    self.update_lidar_obstacle_clusters()
    self.lidar_data = self.lidar_points_ne.copy()
    self.update_apf_obstacle_tracks(self.lidar_timestamp_s)
    self.update_lidar_sectors()
    self.lidar_new = True
    self.robot_available = True

# ----- LiDAR coordinate transforms -----
def robot_pose_at_time(self, stamp_s):
    # ponytail: constant-velocity deskew; add a pose-history interpolator
    # only if LiDAR latency exceeds this short extrapolation window.
    pose = np.asarray(self.p_robot, dtype=float).reshape(3).copy()
    velocity = np.asarray(
        getattr(self, "v_robot", np.zeros((3, 1))),
        dtype=float,
    ).reshape(3)
    state_stamp_s = float(getattr(self, "last_nav_t", stamp_s))
    max_dt_s = max(float(getattr(self, "lidar_pose_extrapolation_limit_s", 0.5)),
, 0.0)
    dt_s = float(np.clip(float(stamp_s) - state_stamp_s, -max_dt_s, max_dt_s))
    pose[0:2] += velocity[0:2] * dt_s
    pose[2] = wrap_angle(pose[2] + velocity[2] * dt_s)
    return pose

def earth_vector_to_body(self, vector_ne):
    # Body frame convention: x is forward, y is left, gamma is yaw in earth
    frame.
    pose = np.asarray(self.p_robot, dtype=float).reshape(-1)
    yaw = pose[2]
    c = np.cos(yaw)
    s = np.sin(yaw)
    vector_ne = np.asarray(vector_ne, dtype=float).reshape(2)

    return np.array([
        c * vector_ne[0] + s * vector_ne[1],
        -s * vector_ne[0] + c * vector_ne[1],
    ])

def earth_point_to_body(self, point_ne):
    # Point transform is a vector transform after subtracting the EKF robot
    position.
    pose = np.asarray(self.p_robot, dtype=float).reshape(-1)
    return self.earth_vector_to_body(np.asarray(point_ne, dtype=float).reshape
(2) - pose[0:2])

def body_vector_to_earth(self, vector_body):
    pose = np.asarray(self.p_robot, dtype=float).reshape(-1)
    yaw = pose[2]
    c = np.cos(yaw)
    s = np.sin(yaw)

```

```

vector_body = np.asarray(vector_body, dtype=float).reshape(2)

return np.array([
    c * vector_body[0] - s * vector_body[1],
    s * vector_body[0] + c * vector_body[1],
])

def body_point_to_earth(self, point_body):
    pose = np.asarray(self.p_robot, dtype=float).reshape(-1)
    yaw = pose[2]
    c = np.cos(yaw)
    s = np.sin(yaw)
    point_body = np.asarray(point_body, dtype=float).reshape(2)

    return np.array([
        pose[0] + c * point_body[0] - s * point_body[1],
        pose[1] + s * point_body[0] + c * point_body[1],
    ])

# ----- LiDAR DBSCAN clustering -----
def clear_lidar_obstacle_clusters(self):
    self.lidar_points_body = np.empty((0, 2))
    self.lidar_points_ne = np.empty((0, 2))
    self.lidar_cluster_labels = np.array([], dtype=int)
    self.lidar_obstacles = []
    self.lidar_obstacle_centres_body = np.empty((0, 2))
    self.lidar_obstacle_centres_ne = np.empty((0, 2))
    self.lidar_obstacle_distances_m = np.array([])
    self.lidar_obstacle_angles_rad = np.array([])
    self.nearest_lidar_obstacle = None

def update_lidar_obstacle_clusters(self):
    if self.lidar_data_rb is None:
        self.clear_lidar_obstacle_clusters()
        return

    ranges = self.lidar_data_rb[:, 0]
    angles = self.lidar_data_rb[:, 1]
    valid = np.isfinite(ranges) & np.isfinite(angles)

    if np.count_nonzero(valid) < self.lidar_dbscan_min_points:
        self.clear_lidar_obstacle_clusters()
        return

    valid_ranges = ranges[valid]
    valid_angles = angles[valid] + self.lidar_gamma_bl

    points_body_at_beam = np.column_stack([
        self.lidar_x_bl + valid_ranges * np.cos(valid_angles),
        self.lidar_y_bl + valid_ranges * np.sin(valid_angles),
    ])

    beam_stamps_s = getattr(self, "lidar_beam_stamps_s", None)
    if beam_stamps_s is None or len(beam_stamps_s) != len(ranges):
        lidar_timestamp_s = getattr(self, "lidar_timestamp_s", None)
        reference_stamp_s = float(
            lidar_timestamp_s
            if lidar_timestamp_s is not None

```

```

        else getattr(self, "last_nav_t", time.time())
    )
    valid_stamps_s = np.full(len(valid_ranges), reference_stamp_s, dtype=
float)
    else:
        valid_stamps_s = np.asarray(beam_stamps_s, dtype=float)[valid]
        reference_stamp_s = float(valid_stamps_s[0])

    reference_pose = self.robot_pose_at_time(reference_stamp_s)
    self.lidar_reference_pose = reference_pose.copy()
    beam_poses = np.asarray(
        [self.robot_pose_at_time(stamp_s) for stamp_s in valid_stamps_s],
        dtype=float,
    )
    beam_cos = np.cos(beam_poses[:, 2])
    beam_sin = np.sin(beam_poses[:, 2])
    self.lidar_points_ne = np.column_stack([
        beam_poses[:, 0]
        + beam_cos * points_body_at_beam[:, 0]
        - beam_sin * points_body_at_beam[:, 1],
        beam_poses[:, 1]
        + beam_sin * points_body_at_beam[:, 0]
        + beam_cos * points_body_at_beam[:, 1],
    ])

    reference_cos = np.cos(reference_pose[2])
    reference_sin = np.sin(reference_pose[2])
    relative_ne = self.lidar_points_ne - reference_pose[0:2]
    self.lidar_points_body = np.column_stack([
        reference_cos * relative_ne[:, 0] + reference_sin * relative_ne[:, 1],
        -reference_sin * relative_ne[:, 0] + reference_cos * relative_ne[:, 1],
    ])

    labels = DBSCAN(
        eps=self.lidar_dbscan_eps_m,
        min_samples=self.lidar_dbscan_min_points,
        n_jobs=1,
    ).fit_predict(self.lidar_points_body)
    labels = merge_collinear_cluster_labels(
        self.lidar_points_body,
        labels,
        max_gap_m=self.lidar_fragment_merge_gap_m,
        max_width_m=self.lidar_fragment_max_width_m,
        max_length_m=self.lidar_fragment_max_length_m,
    )
    self.lidar_cluster_labels = labels

    obstacles = []
    for label in sorted(set(labels)):
        if label == -1:
            continue

        cluster_points = self.lidar_points_body[labels == label]
        cluster_points_ne = self.lidar_points_ne[labels == label]
        centre_body, pc1_m, pc2_m, length_axis_body =
cluster_principal_dimensions(
            cluster_points,
            self.obstacle_min_pc1_m,

```

```

        self.obstacle_min_pc2_m,
    )
    relative_points = cluster_points - centre_body
    cluster_radius_m = float(np.max(np.linalg.norm(relative_points, axis=1)))
)

    cluster_extent_xy_m = np.ptp(cluster_points, axis=0)
    cluster_size_m = float(np.linalg.norm(cluster_extent_xy_m))
    equivalent_radius_m = 0.5 * max(pc1_m, pc2_m)
    # EKF measurements must stay in the fixed world frame so ownship
    # translation and yaw do not appear as obstacle motion.
    centre_ne = np.mean(cluster_points_ne, axis=0)
    length_axis_ne = np.array([
        reference_cos * length_axis_body[0] - reference_sin *
length_axis_body[1],
        reference_sin * length_axis_body[0] + reference_cos *
length_axis_body[1],
    ])
    centre_distance_m = float(np.linalg.norm(centre_body))
    centre_angle_rad = float(np.arctan2(centre_body[1], centre_body[0]))
    min_distance_m = float(np.min(np.linalg.norm(cluster_points, axis=1)))
    measurement_covariance = self.obstacle_measurement_covariance(
        centre_ne,
        reference_pose,
    )

    obstacles.append({
        "label": int(label),
        "point_count": int(len(cluster_points)),
        "centre_body": centre_body.tolist(),
        "centre_ne": centre_ne.tolist(),
        "distance_m": centre_distance_m,
        "angle_rad": centre_angle_rad,
        "angle_deg": float(np.rad2deg(centre_angle_rad)),
        "min_distance_m": min_distance_m,
        "cluster_radius_m": cluster_radius_m,
        "cluster_size_m": cluster_size_m,
        "pc1_m": pc1_m,
        "pc2_m": pc2_m,
        "length_axis_body": length_axis_body.tolist(),
        "length_axis_ne": length_axis_ne.tolist(),
        "equivalent_radius_m": equivalent_radius_m,
        "measurement_covariance": measurement_covariance.tolist(),
        "points_ne": np.asarray(
            [self.body_point_to_earth(point) for point in cluster_points],
            dtype=float,
        ).tolist(),
    })

    obstacles.sort(key=lambda obstacle: obstacle["distance_m"])
    self.lidar_obstacles = obstacles
    self.nearest_lidar_obstacle = obstacles[0] if obstacles else None

    if obstacles:
        self.lidar_obstacle_centres_body = np.array(
            [obstacle["centre_body"] for obstacle in obstacles],
            dtype=float,
        )
        self.lidar_obstacle_centres_ne = np.array(

```



```

        [obstacle["centre_ne"] for obstacle in obstacles],
        dtype=float,
    )
    self.lidar_obstacle_distances_m = np.array(
        [obstacle["distance_m"] for obstacle in obstacles],
        dtype=float,
    )
    self.lidar_obstacle_angles_rad = np.array(
        [obstacle["angle_rad"] for obstacle in obstacles],
        dtype=float,
    )
else:
    self.lidar_obstacle_centres_body = np.empty((0, 2))
    self.lidar_obstacle_centres_ne = np.empty((0, 2))
    self.lidar_obstacle_distances_m = np.array([])
    self.lidar_obstacle_angles_rad = np.array([])

# ----- LiDAR sector helpers -----
def sector_ranges(self, angle_min, angle_max):
    if self.lidar_data_rb is None:
        return np.array([])

    ranges = self.lidar_data_rb[:, 0]
    angles = self.lidar_data_rb[:, 1]

    mask = (
        (angles > angle_min)
        & (angles < angle_max)
        & np.isfinite(ranges)
    )

    return ranges[mask]

def sector_min_range(self, angle_min, angle_max):
    vals = self.sector_ranges(angle_min, angle_max)

    if len(vals) == 0:
        return np.inf

    val = np.nanmin(vals)

    if not np.isfinite(val):
        return np.inf

    return val

def update_lidar_sectors(self):
    self.front_clearance_m = self.sector_min_range(
        -self.front_angle_limit,
        self.front_angle_limit,
    )

    self.left_clearance_m = self.sector_min_range(
        self.side_angle_min,
        self.side_angle_max,
    )

    self.right_clearance_m = self.sector_min_range(

```

```

        -self.side_angle_max,
        -self.side_angle_min,
    )

    front_ranges = self.sector_ranges(
        -self.front_angle_limit,
        self.front_angle_limit,
    )

    if len(front_ranges) == 0:
        front_blocked = False
    else:
        front_blocked = np.sum(front_ranges < self.front_block_threshold) >=
self.min_front_close_beams

    left_open = self.left_clearance_m > self.front_block_threshold
    right_open = self.right_clearance_m > self.front_block_threshold

    if front_blocked and left_open and right_open:
        self.map_observation_class = "t_junction_or_end_wall"
    elif front_blocked and left_open:
        self.map_observation_class = "right_angle_left_turn"
    elif front_blocked and right_open:
        self.map_observation_class = "right_angle_right_turn"
    elif front_blocked:
        self.map_observation_class = "blocked_front"
    else:
        self.map_observation_class = "straight_section"

def front_blocked(self):
    self.update_lidar_sectors()

    front_ranges = self.sector_ranges(
        -self.front_angle_limit,
        self.front_angle_limit,
    )

    if len(front_ranges) == 0:
        return False

    close_count = int(np.sum(front_ranges < self.front_block_threshold))
    return close_count >= self.min_front_close_beams

def json_safe(self, value):
    if value is None:
        return None

    if isinstance(value, (str, bool)):
        return value

    if isinstance(value, np.ndarray):
        return self.json_safe(value.tolist())

    if isinstance(value, np.generic):
        return self.json_safe(value.item())

    if isinstance(value, int):
        return value

```

```

    if isinstance(value, float):
        if not np.isfinite(value):
            return None
        return value

    if isinstance(value, dict):
        return {str(key): self.json_safe(item) for key, item in value.items()}

    if isinstance(value, (list, tuple)):
        return [self.json_safe(item) for item in value]

    return value

def write_obstacle_snapshot(self):
    stamp_s = self.latest_lidar_received_s
    if stamp_s is None:
        return

    stamp_s = float(stamp_s)
    if self.last_obstacle_snapshot_stamp_s == stamp_s:
        return

    self.last_obstacle_snapshot_stamp_s = stamp_s
    robot_truth_ne = np.asarray(
        getattr(self, "webots_robot_truth_ne", [np.nan, np.nan]), dtype=float
    ).reshape(2)
    robot_pos = robot_truth_ne if np.isfinite(robot_truth_ne).all() else np.
array([self.North, self.East])
    robot_yaw_rad = getattr(self, "webots_robot_truth_yaw_rad", None)
    if robot_yaw_rad is None or not np.isfinite(robot_yaw_rad):
        robot_yaw_rad = self.Yaw
    payload = {
        "t": self.timefromstart,
        "timestamp_s": stamp_s,
        "robot_pos": robot_pos,
        "robot_yaw_rad": robot_yaw_rad,
        "robot_pos_source": "webots_groundtruth" if np.isfinite(robot_truth_ne).
all() else "ekf",
        "robot_velocity_ne": np.asarray(self.v_robot[0:2], dtype=float),
        "cloud": self.lidar_data if self.lidar_data is not None else [],
        "clusters": self.lidar_obstacles,
        "tracks": self.obstacle_track_visuals(),
        "webots_obstacle_truth": self.webots_obstacle_truth,
        "virtual_obstacles": self.apf_virtual_obstacles,
        "apf": {
            "navigation_mode": self.navigation_mode,
            "encounter": self.apf_encounter_mode,
            "active_profile": self.apf_active_profile_name,
            "colreg_rule": self.apf_colreg_rule,
            "side": self.apf_avoidance_side_sign,
            "dcpa_m": self.apf_colreg_dcpa_m,
            "tcpa_s": self.apf_colreg_tcpa_s,
            "force_body": self.apf_force_body,
            "repulsive_force_body": self.apf_repulsive_force_body,
            "attractive_force_body": self.apf_attractive_force_body,
            "target_ne": self.apf_target_ne,
        },
    },

```

```

        "apf_settings": {
            "own_equivalent_radius_m": self.apf_own_equivalent_radius_m,
            "collision_horizon_s": self.apf_collision_horizon_s,
            "prediction_dt_s": self.apf_prediction_dt_s,
            "constant_descent_speed_m_s": self.apf_constant_descent_speed_m_s,
            "obstacle_ekf_prediction_enabled": self.
obstacle_ekf_prediction_enabled,
            "obstacle_prediction_horizon_s": self.obstacle_prediction_horizon_s,
            "obstacle_prediction_step_s": self.obstacle_prediction_step_s,
            "dynamic_speed_enter_m_s": self.apf_dynamic_speed_threshold_m_s,
            "dynamic_speed_exit_m_s": self.apf_dynamic_exit_speed_threshold_m_s,
            "prediction_min_samples": self.obstacle_prediction_min_samples,
            "prediction_min_time_span_s": self.
obstacle_prediction_min_time_span_s,
            "prediction_max_speed_std_m_s": self.
obstacle_prediction_max_speed_std_m_s,
            "prediction_max_heading_var_rad2": self.
obstacle_prediction_max_heading_var_rad2,
            "cluster_range_enabled": self.apf_cluster_range_enabled,
            "risk_pc_scale": self.apf_risk_pc_scale,
            "avoidance_pc_scale": self.apf_avoidance_pc_scale,
            "direction_pc_scale": self.apf_direction_pc_scale,
            "virtual_pc_scale": self.apf_virtual_pc_scale,
            "crossing_profile": self.apf_crossing_params,
            "overtaking_profile": self.apf_overtaking_params,
            "head_on_profile": self.apf_head_on_params,
            "minimum_pc1_m": self.obstacle_min_pc1_m,
            "minimum_pc2_m": self.obstacle_min_pc2_m,
        },
        "dbscan": {
            "eps_m": self.lidar_dbscan_eps_m,
            "min_samples": self.lidar_dbscan_min_points,
            "fragment_merge_gap_m": self.lidar_fragment_merge_gap_m,
            "fragment_max_width_m": self.lidar_fragment_max_width_m,
            "fragment_max_length_m": self.lidar_fragment_max_length_m,
        },
    }

    filename = f"obstacle_{int(round(stamp_s * 1000.0))}.json"
    with (self.obstacle_log_dir / filename).open("w") as f:
        json.dump(self.json_safe(payload), f, indent=2)

# ----- COLREGS-compliant modified APF -----
def reset_apf_diagnostics(self, clear_visual=True):
    if clear_visual:
        self.apf_force_body = np.zeros(2, dtype=float)
        self.apf_repulsive_force_body = np.zeros(2, dtype=float)
        self.apf_attractive_force_body = np.zeros(2, dtype=float)
        self.apf_steering_force_body = np.zeros(2, dtype=float)
        self.apf_target_ne = np.array([np.nan, np.nan], dtype=float)
        self.apf_virtual_obstacles = []
        self.apf_visual_hold_until_s = 0.0

    self.apf_active_profile_name = "crossing"
    self.apf_encounter_mode = "none"
    self.apf_colreg_rule = "none"
    self.apf_avoidance_side_sign = 0.0
    self.apf_colreg_dcpa_m = np.nan

```

```

self.apf_colreg_tcpa_s = np.nan
self.apf_colreg_active = False

def apf_visual_hold_active(self):
    now_s = float(self.timefromstart) if self.timefromstart is not None else 0.0
    return now_s < self.apf_visual_hold_until_s

def limit_heading_deviation_command(self, yaw_rate_cmd):
    yaw_rate_cmd = float(yaw_rate_cmd)
    heading_offset = wrap_angle(float(self.Yaw) - self.route_heading_rad)
    max_offset = self.max_heading_deviation_rad
    guard_offset = self.heading_deviation_guard_rad
    dt = max(float(self.lstdt), 1e-3)
    offset_sign = float(np.sign(heading_offset))

    if abs(heading_offset) >= max_offset:
        return offset_sign * self.w_max

    if abs(heading_offset) >= guard_offset and yaw_rate_cmd * offset_sign < 0.0:
        return offset_sign * min(
            self.w_max,
            self.heading_deviation_return_gain * (abs(heading_offset) -
guard_offset),
        )

    predicted_offset = heading_offset - yaw_rate_cmd * dt
    if predicted_offset > max_offset:
        return 0.0

    if predicted_offset < -max_offset:
        return 0.0

    return yaw_rate_cmd

def current_velocity_body(self):
    vel_ne = np.asarray(self.v_robot[0:2], dtype=float).reshape(2)
    vel_body = self.earth_vector_to_body(vel_ne)

    if not np.isfinite(vel_body).all():
        return np.zeros(2, dtype=float)

    return vel_body

def obstacle_ekf_process_noise(self, dt):
    dt = max(float(dt), 1e-3)
    q = self.obstacle_ekf_accel_std_m_s2 ** 2
    dt2 = dt * dt
    dt3 = dt2 * dt
    dt4 = dt2 * dt2

    return q * np.array(
        [
            [0.25 * dt4, 0.0, 0.5 * dt3, 0.0],
            [0.0, 0.25 * dt4, 0.0, 0.5 * dt3],
            [0.5 * dt3, 0.0, dt2, 0.0],
            [0.0, 0.5 * dt3, 0.0, dt2],
        ],
        dtype=float,
    )

```

```

)

def obstacle_ekf_predict(self, state, covariance, dt):
    dt = max(float(dt), 0.0)
    state = np.asarray(state, dtype=float).reshape(4)
    covariance = np.asarray(covariance, dtype=float).reshape(4, 4)

    F = np.array(
        [
            [1.0, 0.0, dt, 0.0],
            [0.0, 1.0, 0.0, dt],
            [0.0, 0.0, 1.0, 0.0],
            [0.0, 0.0, 0.0, 1.0],
        ],
        dtype=float,
    )

    predicted_state = F @ state
    predicted_covariance = F @ covariance @ F.T + self.obstacle_ekf_process_noise(dt)
    return predicted_state, predicted_covariance

def obstacle_measurement_covariance(self, measurement_ne, reference_pose=None):
    measurement_ne = np.asarray(measurement_ne, dtype=float).reshape(2)
    if reference_pose is None:
        reference_pose = getattr(self, "lidar_reference_pose", None)
    if reference_pose is None:
        reference_pose = np.asarray(self.p_robot, dtype=float).reshape(3)
    else:
        reference_pose = np.asarray(reference_pose, dtype=float).reshape(3)

    base_variance = float(self.obstacle_ekf_measurement_std_m) ** 2
    yaw_variance = 0.0
    sigma = getattr(self, "Sigma", None)
    if sigma is not None:
        sigma = np.asarray(sigma, dtype=float)
        if sigma.ndim == 2 and sigma.shape[0] > G and sigma.shape[1] > G:
            yaw_variance = max(float(sigma[G, G]), 0.0)

    yaw_rate_rad_s = getattr(self, "sensed_imu_yaw_rate_rad_s", None)
    if yaw_rate_rad_s is None or not np.isfinite(yaw_rate_rad_s):
        velocity = np.asarray(getattr(self, "v_robot", np.zeros((3, 1))), dtype=float).reshape(-1)
        yaw_rate_rad_s = velocity[2] if len(velocity) > 2 else 0.0

    timing_std_s = max(
        float(getattr(self, "lidar_time_sync_std_s", 0.02)),
        float(getattr(self, "lidar_scan_time_s", 0.0)) / np.sqrt(12.0),
    )
    yaw_variance += (float(yaw_rate_rad_s) * timing_std_s) ** 2

    relative_ne = measurement_ne - reference_pose[0:2]
    yaw_jacobian = np.array([-relative_ne[1], relative_ne[0]], dtype=float)
    return (
        base_variance * np.eye(2, dtype=float)
        + yaw_variance * np.outer(yaw_jacobian, yaw_jacobian)
    )

```

```

def obstacle_ekf_update(
    self,
    state,
    covariance,
    measurement_ne,
    measurement_covariance=None,
):
    state = np.asarray(state, dtype=float).reshape(4)
    covariance = np.asarray(covariance, dtype=float).reshape(4, 4)
    measurement_ne = np.asarray(measurement_ne, dtype=float).reshape(2)

    H = np.array(
        [
            [1.0, 0.0, 0.0, 0.0],
            [0.0, 1.0, 0.0, 0.0],
        ],
        dtype=float,
    )
    R_obs = (
        self.obstacle_measurement_covariance(measurement_ne)
        if measurement_covariance is None
        else np.asarray(measurement_covariance, dtype=float).reshape(2, 2)
    )
    innovation = measurement_ne - H @ state
    innovation_covariance = H @ covariance @ H.T + R_obs

    try:
        kalman_gain = covariance @ H.T @ np.linalg.inv(innovation_covariance)
    except np.linalg.LinAlgError:
        kalman_gain = covariance @ H.T @ np.linalg.pinv(innovation_covariance)

    corrected_state = state + kalman_gain @ innovation
    identity = np.eye(4, dtype=float)
    correction = identity - kalman_gain @ H
    corrected_covariance = correction @ covariance @ correction.T + kalman_gain
    @ R_obs @ kalman_gain.T
    return corrected_state, corrected_covariance

def obstacle_track_motion_is_stable(self, track):
    sample_count = int(track.get("stats_sample_count", 0))
    hit_count = int(track.get("hit_count", 0))
    if (
        sample_count < self.obstacle_prediction_min_samples
        or hit_count < self.obstacle_prediction_min_hits
        or int(track.get("miss_count", 0)) > 0
    ):
        return False

    samples = track.get("motion_window", [])
    if len(samples) < 2:
        return False

    first_stamp = float(samples[0].get("stamp_s", 0.0))
    last_stamp = float(samples[-1].get("stamp_s", first_stamp))
    if last_stamp - first_stamp < self.obstacle_prediction_min_time_span_s:
        return False

    speed_m_s = float(track.get("speed_mean_m_s", 0.0))

```

```

was_stable = bool(track.get("motion_stable", False))
speed_threshold = (
    self.apf_dynamic_exit_speed_threshold_m_s
    if was_stable
    else self.apf_dynamic_speed_threshold_m_s
)
if not np.isfinite(speed_m_s) or speed_m_s < speed_threshold:
    return False

return True

def obstacle_track_prediction_accel_ne(self, track):
    if (
        not bool(track.get("motion_stable", False))
        or int(track.get("stats_sample_count", 0)) < self.
obstacle_prediction_accel_min_samples
    ):
        return np.zeros(2, dtype=float)

    accel_ne = np.asarray(track.get("accel_ne", [0.0, 0.0]), dtype=float).
reshape(2)
    accel_var_ne = np.asarray(track.get("accel_var_ne", [np.inf, np.inf]), dtype
=float).reshape(2)
    if not np.isfinite(accel_ne).all() or not np.isfinite(accel_var_ne).all():
        return np.zeros(2, dtype=float)

    accel_std_m_s2 = float(np.sqrt(max(float(np.max(accel_var_ne)), 0.0)))
    if accel_std_m_s2 > self.obstacle_prediction_max_accel_std_m_s2:
        return np.zeros(2, dtype=float)

    return accel_ne

def obstacle_track_prediction_ne(self, track):
    if not self.obstacle_ekf_prediction_enabled:
        return np.empty((0, 2), dtype=float)

    state = np.asarray(track.get("state", [np.nan, np.nan, 0.0, 0.0]), dtype=
float).reshape(4)
    if not np.isfinite(state).all():
        return np.empty((0, 2), dtype=float)

    velocity_ne = state[2:4].copy()

    accel_ne = self.obstacle_track_prediction_accel_ne(track)

    step_s = max(float(self.obstacle_prediction_step_s), 1e-3)
    horizon_s = float(self.obstacle_prediction_horizon_s)
    if not np.isfinite(horizon_s) or horizon_s <= 0.0:
        return np.empty((0, 2), dtype=float)
    times = np.arange(0.0, horizon_s, step_s, dtype=float)
    times = np.r_[times, horizon_s]

    return np.column_stack(
        [
            state[0] + velocity_ne[0] * times + 0.5 * accel_ne[0] * times *
times,
            state[1] + velocity_ne[1] * times + 0.5 * accel_ne[1] * times *
times,

```



```

    ]
)

def sync_obstacle_track_fields(self, track):
    state = np.asarray(track.get("state", [np.nan, np.nan, 0.0, 0.0]), dtype=
float).reshape(4)
    track["pos_ne"] = state[0:2].copy()
    track["vel_ne"] = state[2:4].copy()

    pc1_m = float(track.get("pc1_m", self.obstacle_min_pc1_m))
    pc2_m = float(track.get("pc2_m", self.obstacle_min_pc2_m))
    if not np.isfinite(pc1_m) or pc1_m <= 0.0:
        pc1_m = self.obstacle_min_pc1_m
    if not np.isfinite(pc2_m) or pc2_m <= 0.0:
        pc2_m = self.obstacle_min_pc2_m
    track["pc1_m"] = max(pc1_m, self.obstacle_min_pc1_m)
    track["pc2_m"] = max(min(pc2_m, track["pc1_m"]), self.obstacle_min_pc2_m)
    track["radius_m"] = 0.5 * track["pc1_m"]
    track["equivalent_radius_m"] = track["radius_m"]

    length_axis_ne = np.asarray(track.get("length_axis_ne", [1.0, 0.0]), dtype=
float).reshape(2)
    axis_norm = float(np.linalg.norm(length_axis_ne))
    track["length_axis_ne"] = (
        length_axis_ne / axis_norm
        if np.isfinite(length_axis_ne).all() and axis_norm >= 1e-6
        else np.array([1.0, 0.0], dtype=float)
    )
    if bool(track.get("motion_stable", False)) and track["pc1_m"] >= self.
obstacle_axis_min_aspect_ratio * track["pc2_m"]:
        state[2:4] = constrain_velocity_to_axis(
            state[2:4], track["length_axis_ne"], self.
obstacle_axis_max_velocity_gap_rad
        )
        track["state"] = state
        track["vel_ne"] = state[2:4].copy()

    ekf_velocity_ne = track["vel_ne"].copy()
    speed_m_s = float(np.linalg.norm(ekf_velocity_ne))
    track["speed_m_s"] = speed_m_s
    heading_hold_speed_m_s = float(
        getattr(self, "obstacle_heading_hold_speed_m_s", 0.03)
    )
    if speed_m_s >= heading_hold_speed_m_s:
        heading_rad = float(np.arctan2(ekf_velocity_ne[1], ekf_velocity_ne[0]))
        track["heading_rad"] = heading_rad
        track["heading_deg"] = float(np.rad2deg(heading_rad))
        track["heading_axis_ne"] = ekf_velocity_ne / speed_m_s
    else:
        previous_heading_rad = float(track.get("heading_rad", np.nan))
        if not np.isfinite(previous_heading_rad):
            previous_heading_rad = float(np.arctan2(
                track["length_axis_ne"][1],
                track["length_axis_ne"][0],
            ))
        track["heading_rad"] = previous_heading_rad
        track["heading_deg"] = float(np.rad2deg(previous_heading_rad))
        track["heading_axis_ne"] = np.array([

```

```

        np.cos(previous_heading_rad),
        np.sin(previous_heading_rad),
    ])

    if not self.obstacle_ekf_prediction_enabled:
        track["prediction_model"] = "disabled"
        track["prediction_ne"] = np.empty((0, 2), dtype=float)
        return

    if np.linalg.norm(self.obstacle_track_prediction_accel_ne(track)) > 0.0:
        track["prediction_model"] = "sliding_window_acceleration"
    elif bool(track.get("motion_stable", False)):
        track["prediction_model"] = "stable_constant_velocity"
    else:
        track["prediction_model"] = "ekf_constant_velocity"
    track["prediction_ne"] = self.obstacle_track_prediction_ne(track)

def make_obstacle_track(
    self,
    detection_ne,
    stamp_s,
    pc1_m=np.nan,
    pc2_m=np.nan,
    length_axis_ne=None,
):
    detection_ne = np.asarray(detection_ne, dtype=float).reshape(2)
    pc1_m = float(pc1_m) if np.isfinite(pc1_m) and pc1_m > 0.0 else self.obstacle_min_pc1_m
    pc2_m = float(pc2_m) if np.isfinite(pc2_m) and pc2_m > 0.0 else self.obstacle_min_pc2_m
    length_axis_ne = np.asarray(
        [1.0, 0.0] if length_axis_ne is None else length_axis_ne,
        dtype=float,
    ).reshape(2)
    covariance = np.diag(
        [
            self.obstacle_ekf_initial_position_std_m ** 2,
            self.obstacle_ekf_initial_position_std_m ** 2,
            self.obstacle_ekf_initial_velocity_std_m_s ** 2,
            self.obstacle_ekf_initial_velocity_std_m_s ** 2,
        ]
    )
    track = {
        "id": self.apf_next_track_id,
        "state": np.array([detection_ne[0], detection_ne[1], 0.0, 0.0], dtype=
float),
        "covariance": covariance,
        "stamp_s": float(stamp_s),
        "last_seen_s": float(stamp_s),
        "hit_count": 1,
        "miss_count": 0,
        "history_ne": [],
        "lidar_history_ne": [],
        "motion_window": [],
        "pc1_m": pc1_m,
        "pc2_m": pc2_m,
        "length_axis_ne": length_axis_ne,
    }

```

```

        self.sync_obstacle_track_fields(track)
        self.append_obstacle_track_history(
            track,
            pc1_m=pc1_m,
            pc2_m=pc2_m,
            length_axis_ne=length_axis_ne,
            detection_ne=detection_ne,
            stamp_s=stamp_s,
        )
        self.apf_next_track_id += 1
        return track

def make_obstacle_track_candidate(
    self,
    detection_ne,
    stamp_s,
    pc1_m=np.nan,
    pc2_m=np.nan,
    length_axis_ne=None,
):
    detection_ne = np.asarray(detection_ne, dtype=float).reshape(2)
    length_axis_ne = np.asarray(
        [1.0, 0.0] if length_axis_ne is None else length_axis_ne,
        dtype=float,
    ).reshape(2)
    pc1_m = float(pc1_m) if np.isfinite(pc1_m) and pc1_m > 0.0 else self.
obstacle_min_pc1_m
    pc2_m = float(pc2_m) if np.isfinite(pc2_m) and pc2_m > 0.0 else self.
obstacle_min_pc2_m
    return {
        "centre_ne": detection_ne.copy(),
        "stamp_s": float(stamp_s),
        "last_seen_s": float(stamp_s),
        "hit_count": 1,
        "pc1_m": pc1_m,
        "pc2_m": pc2_m,
        "length_axis_ne": length_axis_ne,
    }

def predict_obstacle_track_to_time(self, track, stamp_s):
    now = float(stamp_s)
    dt = max(now - float(track.get("stamp_s", now)), 0.0)
    state, covariance = self.obstacle_ekf_predict(
        track.get("state", np.r_[track.get("pos_ne", [np.nan, np.nan]), track.
get("vel_ne", [0.0, 0.0])]),
        track.get("covariance", np.eye(4, dtype=float)),
        dt,
    )
    track["state"] = state
    track["covariance"] = covariance
    track["stamp_s"] = now
    self.sync_obstacle_track_fields(track)

def append_obstacle_track_history(
    self,
    track,
    pc1_m=np.nan,
    pc2_m=np.nan,

```

```

length_axis_ne=None,
detection_ne=None,
stamp_s=None,
):
    stamp_s = float(stamp_s if stamp_s is not None else track.get("stamp_s",
time.time()))
    pos_ne = np.asarray(track["pos_ne"], dtype=float).reshape(2).copy()
    vel_ne = np.asarray(track.get("vel_ne", [0.0, 0.0]), dtype=float).reshape(2).
copy()
    if not np.isfinite(vel_ne).all():
        vel_ne = np.zeros(2, dtype=float)

    pc1_m = float(pc1_m) if np.isfinite(pc1_m) and pc1_m > 0.0 else float(track.
get("pc1_m", self.obstacle_min_pc1_m))
    pc2_m = float(pc2_m) if np.isfinite(pc2_m) and pc2_m > 0.0 else float(track.
get("pc2_m", self.obstacle_min_pc2_m))
    track["pc1_m"] = max(pc1_m, self.obstacle_min_pc1_m)
    track["pc2_m"] = max(min(pc2_m, track["pc1_m"]), self.obstacle_min_pc2_m)

    if length_axis_ne is not None:
        length_axis_ne = np.asarray(length_axis_ne, dtype=float).reshape(2)
        axis_norm = float(np.linalg.norm(length_axis_ne))
        if np.isfinite(length_axis_ne).all() and axis_norm >= 1e-6:
            length_axis_ne = length_axis_ne / axis_norm
            previous_axis = np.asarray(track.get("length_axis_ne",
length_axis_ne), dtype=float).reshape(2)
            if float(np.dot(length_axis_ne, previous_axis)) < 0.0:
                length_axis_ne = -length_axis_ne
            track["length_axis_ne"] = smooth_undirected_axis(
                previous_axis, length_axis_ne, self.
obstacle_axis_smoothing_alpha
            )

    history = track.setdefault("history_ne", [])
    history.append(pos_ne)
    if len(history) > self.obstacle_history_len:
        del history[:-self.obstacle_history_len]

    if detection_ne is not None:
        detection_ne = np.asarray(detection_ne, dtype=float).reshape(2)
        if np.isfinite(detection_ne).all():
            lidar_history = track.setdefault("lidar_history_ne", [])
            lidar_history.append(detection_ne.copy())
            if len(lidar_history) > self.obstacle_history_len:
                del lidar_history[:-self.obstacle_history_len]

    speed_m_s = float(np.linalg.norm(vel_ne))
    heading_rad = (
        float(np.arctan2(vel_ne[1], vel_ne[0]))
        if speed_m_s >= self.obstacle_heading_hold_speed_m_s
        else float(track.get("heading_rad", np.nan))
    )
    motion_window = track.setdefault("motion_window", [])
    motion_window.append({
        "stamp_s": stamp_s,
        "pos_ne": pos_ne,
        "vel_ne": vel_ne,
        "speed_m_s": speed_m_s,

```

```

        "heading_rad": heading_rad,
        "pc1_m": track["pc1_m"],
        "pc2_m": track["pc2_m"],
    })

    cutoff_s = stamp_s - max(float(self.obstacle_stats_window_s), 0.0)
    track["motion_window"] = [
        sample for sample in motion_window
        if float(sample.get("stamp_s", stamp_s)) >= cutoff_s
    ][-self.obstacle_history_len:]
    self.update_obstacle_track_statistics(track)

def update_obstacle_track_statistics(self, track):
    samples = track.get("motion_window", [])
    samples = [
        sample for sample in samples
        if np.isfinite(np.asarray(sample.get("pos_ne", [np.nan, np.nan]), dtype=
float)).all()
    ]
    track["stats_sample_count"] = int(len(samples))

    if not samples:
        track["velocity_mean_ne"] = np.asarray(track.get("vel_ne", [0.0, 0.0]),
dtype=float).reshape(2)
        track["velocity_var_ne"] = np.zeros(2, dtype=float)
        track["speed_mean_m_s"] = float(np.linalg.norm(track["velocity_mean_ne"
]))

        track["speed_var_m2_s2"] = 0.0
        track["heading_mean_rad"] = np.nan
        track["heading_var_rad2"] = np.nan
        track["heading_circular_variance"] = np.nan
        track["pc1_mean_m"] = track.get("pc1_m", self.obstacle_min_pc1_m)
        track["pc2_mean_m"] = track.get("pc2_m", self.obstacle_min_pc2_m)
        track["pc1_var_m2"] = 0.0
        track["pc2_var_m2"] = 0.0
        track["accel_ne"] = np.zeros(2, dtype=float)
        track["accel_var_ne"] = np.zeros(2, dtype=float)
        track["motion_stable"] = False
        self.sync_obstacle_track_fields(track)
        return

    velocities = np.asarray([sample["vel_ne"] for sample in samples], dtype=
float)
    finite_vel = np.isfinite(velocities).all(axis=1)
    velocities = velocities[finite_vel]
    if len(velocities) > 0:
        track["velocity_mean_ne"] = np.mean(velocities, axis=0)
    else:
        track["velocity_mean_ne"] = np.asarray(track.get("vel_ne", [0.0, 0.0]),
dtype=float).reshape(2)

    if len(velocities) > 0:
        track["velocity_var_ne"] = np.var(velocities, axis=0)
    else:
        track["velocity_var_ne"] = np.zeros(2, dtype=float)

    speeds = np.asarray([sample["speed_m_s"] for sample in samples], dtype=float
)

```

```

speeds = speeds[np.isfinite(speeds)]
fitted_speed_m_s = float(np.linalg.norm(track["velocity_mean_ne"]))
if len(speeds) > 0:
    track["speed_mean_m_s"] = fitted_speed_m_s
    track["speed_var_m2_s2"] = float(np.var(speeds))
else:
    track["speed_mean_m_s"] = fitted_speed_m_s
    track["speed_var_m2_s2"] = 0.0

headings = np.asarray([sample["heading_rad"] for sample in samples], dtype=
float)
headings = headings[np.isfinite(headings)]
if len(headings) > 0:
    sin_mean = float(np.mean(np.sin(headings)))
    cos_mean = float(np.mean(np.cos(headings)))
    heading_mean = float(np.arctan2(sin_mean, cos_mean))
    heading_error = wrap_angle(headings - heading_mean)
    resultant_length = float(np.hypot(sin_mean, cos_mean))
    track["heading_mean_rad"] = heading_mean
    track["heading_var_rad2"] = float(np.var(heading_error))
    track["heading_circular_variance"] = float(1.0 - np.clip(
resultant_length, 0.0, 1.0))
else:
    track["heading_mean_rad"] = np.nan
    track["heading_var_rad2"] = np.nan
    track["heading_circular_variance"] = np.nan

pc1_values = np.asarray([sample["pc1_m"] for sample in samples], dtype=float
)
pc2_values = np.asarray([sample["pc2_m"] for sample in samples], dtype=float
)
pc1_values = pc1_values[np.isfinite(pc1_values) & (pc1_values > 0.0)]
pc2_values = pc2_values[np.isfinite(pc2_values) & (pc2_values > 0.0)]
track["pc1_mean_m"] = (
    max(float(np.mean(pc1_values)), self.obstacle_min_pc1_m)
    if len(pc1_values) > 0
    else float(track.get("pc1_m", self.obstacle_min_pc1_m))
)
track["pc2_mean_m"] = (
    max(float(np.mean(pc2_values)), self.obstacle_min_pc2_m)
    if len(pc2_values) > 0
    else float(track.get("pc2_m", self.obstacle_min_pc2_m))
)
track["pc2_mean_m"] = min(track["pc2_mean_m"], track["pc1_mean_m"])
track["pc1_var_m2"] = float(np.var(pc1_values)) if len(pc1_values) > 0 else
0.0
track["pc2_var_m2"] = float(np.var(pc2_values)) if len(pc2_values) > 0 else
0.0
track["pc1_m"] = track["pc1_mean_m"]
track["pc2_m"] = track["pc2_mean_m"]

velocity_samples = [
    sample for sample in samples
    if np.isfinite(np.asarray(sample.get("vel_ne", [np.nan, np.nan]), dtype=
float)).all()
]
velocity_samples.sort(key=lambda sample: float(sample.get("stamp_s", 0.0)))
accels = []

```

```

    for prev_sample, next_sample in zip(velocity_samples[:-1], velocity_samples
[1:]):
        dt = float(next_sample.get("stamp_s", 0.0)) - float(prev_sample.get("
stamp_s", 0.0))
        if dt <= 1e-3:
            continue
        prev_vel = np.asarray(prev_sample["vel_ne"], dtype=float).reshape(2)
        next_vel = np.asarray(next_sample["vel_ne"], dtype=float).reshape(2)
        accels.append((next_vel - prev_vel) / dt)

    if accels:
        accel_ne = np.mean(np.asarray(accels, dtype=float), axis=0)
        accel_norm = float(np.linalg.norm(accel_ne))
        if accel_norm > self.obstacle_max_accel_m_s2:
            accel_ne *= self.obstacle_max_accel_m_s2 / max(accel_norm, 1e-6)
        track["accel_ne"] = accel_ne
        track["accel_var_ne"] = np.var(np.asarray(accels, dtype=float), axis=0)
    else:
        track["accel_ne"] = np.zeros(2, dtype=float)
        track["accel_var_ne"] = np.zeros(2, dtype=float)

    track["motion_stable"] = self.obstacle_track_motion_is_stable(track)
    self.sync_obstacle_track_fields(track)

def annotate_lidar_obstacle_with_track(self, obstacle, track):
    prediction_ne = np.asarray(track.get("prediction_ne", np.empty((0, 2))),
dtype=float)
    obstacle["track_id"] = int(track["id"])
    obstacle["velocity_ne"] = np.asarray(track["vel_ne"], dtype=float).reshape
(2).tolist()
    obstacle["velocity_mean_ne"] = np.asarray(track.get("velocity_mean_ne",
track["vel_ne"]), dtype=float).reshape(2).tolist()
    obstacle["velocity_var_ne"] = np.asarray(track.get("velocity_var_ne", [0.0,
0.0]), dtype=float).reshape(2).tolist()
    obstacle["accel_ne"] = np.asarray(track.get("accel_ne", [0.0, 0.0]), dtype=
float).reshape(2).tolist()
    obstacle["speed_m_s"] = float(track.get("speed_m_s", 0.0))
    obstacle["speed_mean_m_s"] = float(track.get("speed_mean_m_s", obstacle["
speed_m_s"]))
    obstacle["speed_var_m2_s2"] = float(track.get("speed_var_m2_s2", 0.0))
    obstacle["heading_rad"] = float(track.get("heading_rad", np.nan))
    obstacle["heading_deg"] = float(track.get("heading_deg", np.nan))
    obstacle["heading_mean_rad"] = float(track.get("heading_mean_rad", np.nan))
    obstacle["heading_var_rad2"] = float(track.get("heading_var_rad2", np.nan))
    obstacle["pc1_m"] = float(track.get("pc1_mean_m", track.get("pc1_m", self.
obstacle_min_pc1_m)))
    obstacle["pc2_m"] = float(track.get("pc2_mean_m", track.get("pc2_m", self.
obstacle_min_pc2_m)))
    obstacle["pc1_var_m2"] = float(track.get("pc1_var_m2", 0.0))
    obstacle["pc2_var_m2"] = float(track.get("pc2_var_m2", 0.0))
    obstacle["length_axis_ne"] = np.asarray(
        track.get("heading_axis_ne", track.get("length_axis_ne", [1.0, 0.0])),
        dtype=float,
    ).reshape(2).tolist()
    obstacle["equivalent_radius_m"] = float(track.get("equivalent_radius_m",
self.obstacle_min_equivalent_radius_m))
    obstacle["stats_sample_count"] = int(track.get("stats_sample_count", 0))
    obstacle["motion_stable"] = bool(track.get("motion_stable", False))

```

```

        obstacle["prediction_model"] = track.get("prediction_model", "
ekf_constant_velocity")
        obstacle["predicted_trajectory_ne"] = prediction_ne.tolist()
        lidar_points_ne = np.asarray(obstacle.get("points_ne", []), dtype=float)
        if lidar_points_ne.ndim == 2 and lidar_points_ne.shape[1] == 2:
            track["lidar_points_ne"] = lidar_points_ne.tolist()

def prune_obstacle_tracks(self, now):
    self.apf_obstacle_tracks = [
        track for track in self.apf_obstacle_tracks
        if now - float(track.get("last_seen_s", track.get("stamp_s", now))) <=
self.apf_track_timeout_s
        and int(track.get("miss_count", 0)) <= 5
    ]

def update_apf_obstacle_tracks(self, stamp_s):
    if not self.obstacle_ekf_prediction_enabled:
        self.apf_obstacle_tracks = []
        self.apf_virtual_obstacles = []
        tracked_fields = {
            "track_id",
            "velocity_ne",
            "velocity_mean_ne",
            "velocity_var_ne",
            "accel_ne",
            "speed_m_s",
            "speed_mean_m_s",
            "speed_var_m2_s2",
            "heading_rad",
            "heading_deg",
            "heading_mean_rad",
            "heading_var_rad2",
            "pc1_m",
            "pc2_m",
            "pc1_var_m2",
            "pc2_var_m2",
            "stats_sample_count",
            "motion_stable",
            "prediction_model",
            "predicted_trajectory_ne",
        }
        for obstacle in self.lidar_obstacles:
            for field in tracked_fields:
                obstacle.pop(field, None)
        return

    now = float(stamp_s if stamp_s is not None else time.time())
    confirmation_hits = max(int(getattr(self, "obstacle_track_confirmation_hits"
, 2)), 2)
    detections = []

    for obstacle_index, obstacle in enumerate(self.lidar_obstacles):
        centre_ne = np.asarray(obstacle.get("centre_ne", [np.nan, np.nan]),
dtype=float).reshape(2)
        if np.isfinite(centre_ne).all():
            pc1_m = float(obstacle.get("pc1_m", self.obstacle_min_pc1_m))
            pc2_m = float(obstacle.get("pc2_m", self.obstacle_min_pc2_m))
            length_axis_ne = np.asarray(

```



```

        obstacle.get("length_axis_ne", [1.0, 0.0]),
        dtype=float,
    ).reshape(2)
    measurement_covariance = np.asarray(
        obstacle.get(
            "measurement_covariance",
            self.obstacle_measurement_covariance(centre_ne),
        ),
        dtype=float,
    ).reshape(2, 2)
    detections.append(
        (
            obstacle_index,
            centre_ne,
            pc1_m,
            pc2_m,
            length_axis_ne,
            measurement_covariance,
        )
    )

    if not detections:
        self.apf_obstacle_track_candidates = [
            candidate
            for candidate in self.apf_obstacle_track_candidates
            if now - float(candidate.get("last_seen_s", candidate.get("stamp_s",
now))) <= self.apf_track_timeout_s
        ]
        for track in self.apf_obstacle_tracks:
            self.predict_obstacle_track_to_time(track, now)
            track["miss_count"] = int(track.get("miss_count", 0)) + 1

        self.prune_obstacle_tracks(now)
        return

    detection_positions = np.asarray(
        [detection for _, detection, _, _, _ in detections],
        dtype=float,
    )
    predicted_tracks = []
    candidates = []
    matched_candidates = set()
    promoted_candidates = set()

    for track_index, track in enumerate(self.apf_obstacle_tracks):
        dt = max(now - float(track.get("stamp_s", now)), 0.0)
        predicted_state, predicted_covariance = self.obstacle_ekf_predict(
            track.get("state", np.r_[track.get("pos_ne", [np.nan, np.nan]),
track.get("vel_ne", [0.0, 0.0])]),
            track.get("covariance", np.eye(4, dtype=float)),
            dt,
        )
        predicted_tracks.append((predicted_state, predicted_covariance))
        predicted_pos = predicted_state[0:2]

        for detection_index, detection in enumerate(detection_positions):
            distance = float(np.linalg.norm(detection - predicted_pos))
            candidates.append((distance, track_index, detection_index))

```

```

candidates.sort(key=lambda item: item[0])
assigned_tracks = set()
assigned_detections = set()
detection_track = {}

for distance, track_index, detection_index in candidates:
    if distance > self.apf_track_association_m:
        break
    if track_index in assigned_tracks or detection_index in
assigned_detections:
        continue

    track = self.apf_obstacle_tracks[track_index]
    detection = detection_positions[detection_index]
    predicted_state, predicted_covariance = predicted_tracks[track_index]
    corrected_state, corrected_covariance = self.obstacle_ekf_update(
        predicted_state,
        predicted_covariance,
        detection,
        detections[detection_index][5],
    )

    track["state"] = corrected_state
    track["covariance"] = corrected_covariance
    track["stamp_s"] = now
    track["last_seen_s"] = now
    track["hit_count"] = int(track.get("hit_count", 0)) + 1
    track["miss_count"] = 0
    self.sync_obstacle_track_fields(track)
    self.append_obstacle_track_history(
        track,
        pc1_m=detections[detection_index][2],
        pc2_m=detections[detection_index][3],
        length_axis_ne=detections[detection_index][4],
        detection_ne=detection,
        stamp_s=now,
    )
    assigned_tracks.add(track_index)
    assigned_detections.add(detection_index)
    detection_track[detection_index] = track

for track_index, track in enumerate(self.apf_obstacle_tracks):
    if track_index not in assigned_tracks:
        predicted_state, predicted_covariance = predicted_tracks[track_index]

        track["state"] = predicted_state
        track["covariance"] = predicted_covariance
        track["stamp_s"] = now
        track["miss_count"] = int(track.get("miss_count", 0)) + 1
        self.sync_obstacle_track_fields(track)

for detection_index, detection in enumerate(detections):
    if detection_index in assigned_detections:
        continue

    _, detection_ne, pc1_m, pc2_m, length_axis_ne, _ = detection
    best_candidate_index = None

```

```

        best_candidate_distance = np.inf
        for candidate_index, candidate in enumerate(self.
apf_obstacle_track_candidates):
            if candidate_index in matched_candidates:
                continue
            candidate_pos = np.asarray(candidate.get("centre_ne", [np.nan, np.
nan]), dtype=float).reshape(2)
            if not np.isfinite(candidate_pos).all():
                continue
            distance = float(np.linalg.norm(detection_ne - candidate_pos))
            if distance < best_candidate_distance:
                best_candidate_distance = distance
                best_candidate_index = candidate_index

        if best_candidate_index is not None and best_candidate_distance <= self.
apf_track_association_m:
            candidate = self.apf_obstacle_track_candidates[best_candidate_index]
            candidate["centre_ne"] = detection_ne.copy()
            candidate["last_seen_s"] = now
            candidate["hit_count"] = int(candidate.get("hit_count", 0)) + 1
            candidate["pc1_m"] = pc1_m
            candidate["pc2_m"] = pc2_m
            candidate["length_axis_ne"] = length_axis_ne
            matched_candidates.add(best_candidate_index)
            if int(candidate["hit_count"]) >= confirmation_hits:
                track = self.make_obstacle_track(
                    detection_ne,
                    now,
                    pc1_m=pc1_m,
                    pc2_m=pc2_m,
                    length_axis_ne=length_axis_ne,
                )
                self.apf_obstacle_tracks.append(track)
                assigned_detections.add(detection_index)
                detection_track[detection_index] = track
                promoted_candidates.add(best_candidate_index)
            continue

        self.apf_obstacle_track_candidates.append(
            self.make_obstacle_track_candidate(
                detection_ne,
                now,
                pc1_m=pc1_m,
                pc2_m=pc2_m,
                length_axis_ne=length_axis_ne,
            )
        )

    for detection_index, track in detection_track.items():
        obstacle_index, _, _, _, _ = detections[detection_index]
        self.annotate_lidar_obstacle_with_track(self.lidar_obstacles[
obstacle_index], track)

    self.apf_obstacle_track_candidates = [
        candidate
        for candidate_index, candidate in enumerate(self.
apf_obstacle_track_candidates)
        if candidate_index not in promoted_candidates

```

```

        if now - float(candidate.get("last_seen_s", candidate.get("stamp_s", now
))) <= self.apf_track_timeout_s
    ]
    self.prune_obstacle_tracks(now)

def obstacle_track_visuals(self):
    if not self.obstacle_ekf_prediction_enabled:
        return []

    now = float(self.latest_lidar_received_s if self.latest_lidar_received_s is
not None else time.time())
    visuals = []

    for track in self.apf_obstacle_tracks:
        if now - float(track.get("last_seen_s", track.get("stamp_s", now))) >
self.apf_track_timeout_s:
            continue

        state, _ = self.obstacle_ekf_predict(
            track.get("state", np.r_[track.get("pos_ne", [np.nan, np.nan]),
track.get("vel_ne", [0.0, 0.0])]),
            track.get("covariance", np.eye(4, dtype=float)),
            max(now - float(track.get("stamp_s", now)), 0.0),
        )
        velocity_ne = state[2:4].copy()
        speed_m_s = float(np.linalg.norm(velocity_ne))
        heading_rad = float(track.get("heading_rad", np.nan))
        if speed_m_s >= self.obstacle_heading_hold_speed_m_s:
            heading_rad = float(np.arctan2(velocity_ne[1], velocity_ne[0]))

        # Preserve the complete statistical state so the visualised
        # trajectory uses the same acceleration/stability gates as the
        # collision predictor and APF controller.
        prediction_track = dict(track)
        prediction_track["state"] = state
        prediction_track["velocity_mean_ne"] = velocity_ne
        prediction_ne = self.obstacle_track_prediction_ne(prediction_track)
        history_ne = np.asarray(track.get("history_ne", []), dtype=float)
        if history_ne.ndim != 2 or history_ne.shape[1] != 2:
            history_ne = np.empty((0, 2), dtype=float)
        lidar_history_ne = np.asarray(track.get("lidar_history_ne", []), dtype=
float)
        if lidar_history_ne.ndim != 2 or lidar_history_ne.shape[1] != 2:
            lidar_history_ne = np.empty((0, 2), dtype=float)

        visuals.append({
            "id": int(track["id"]),
            "position_ne": state[0:2].copy(),
            "velocity_ne": velocity_ne.copy(),
            "speed_m_s": speed_m_s,
            "heading_rad": heading_rad,
            "heading_deg": float(np.rad2deg(heading_rad)) if np.isfinite(
heading_rad) else np.nan,
            "prediction_ne": prediction_ne.copy(),
            "history_ne": history_ne.copy(),
            "lidar_history_ne": lidar_history_ne.copy(),
            "accel_ne": np.asarray(track.get("accel_ne", [0.0, 0.0]), dtype=
float).reshape(2).copy(),

```

```

        "speed_var_m2_s2": float(track.get("speed_var_m2_s2", 0.0)),
        "heading_var_rad2": float(track.get("heading_var_rad2", np.nan)),
        "pc1_m": float(track.get("pc1_mean_m", track.get("pc1_m", self.
obstacle_min_pc1_m))),
        "pc2_m": float(track.get("pc2_mean_m", track.get("pc2_m", self.
obstacle_min_pc2_m))),
        "virtual_position_ne": np.asarray(
            track.get("virtual_position_ne", [np.nan, np.nan]),
            dtype=float,
        ).reshape(2).copy(),
        "collision_time_s": float(track.get("collision_time_s", np.nan)),
        "prediction_model": track.get("prediction_model", "
ekf_constant_velocity"),
        "stats_sample_count": int(track.get("stats_sample_count", 0)),
        "motion_stable": bool(track.get("motion_stable", False)),
        "hit_count": int(track.get("hit_count", 0)),
        "miss_count": int(track.get("miss_count", 0)),
    })

    return visuals

def apf_track_for_obstacle(self, obstacle):
    if not self.obstacle_ekf_prediction_enabled:
        return None

    centre_ne = np.asarray(obstacle.get("centre_ne", [np.nan, np.nan]), dtype=
float).reshape(2)
    if not np.isfinite(centre_ne).all():
        return None

    now = float(self.latest_lidar_received_s if self.latest_lidar_received_s is
not None else time.time())
    best_track = None
    best_distance = np.inf

    for track in self.apf_obstacle_tracks:
        if now - float(track.get("last_seen_s", track.get("stamp_s", now))) >
self.apf_track_timeout_s:
            continue

        predicted_state, _ = self.obstacle_ekf_predict(
            track.get("state", np.r_[track.get("pos_ne", [np.nan, np.nan]),
track.get("vel_ne", [0.0, 0.0])]),
            track.get("covariance", np.eye(4, dtype=float)),
            max(now - float(track.get("stamp_s", now)), 0.0),
        )
        predicted_pos = predicted_state[0:2]
        distance = float(np.linalg.norm(centre_ne - predicted_pos))

        if distance < best_distance:
            best_distance = distance
            best_track = track

    if best_distance <= max(self.apf_track_association_m * 1.5, self.
lidar_dbscan_eps_m * 2.0):
        return best_track

    return None

```

```

def obstacle_pc_dimensions(self, obstacle):
    pc1_m = float(obstacle.get("pc1_m", self.obstacle_min_pc1_m))
    pc2_m = float(obstacle.get("pc2_m", self.obstacle_min_pc2_m))
    if not np.isfinite(pc1_m) or pc1_m <= 0.0:
        pc1_m = self.obstacle_min_pc1_m
    if not np.isfinite(pc2_m) or pc2_m <= 0.0:
        pc2_m = self.obstacle_min_pc2_m
    pc1_m = max(pc1_m, self.obstacle_min_pc1_m)
    pc2_m = max(min(pc2_m, pc1_m), self.obstacle_min_pc2_m)
    return pc1_m, pc2_m

def obstacle_length_axis_ne(self, obstacle):
    axis_ne = np.asarray(
        obstacle.get("length_axis_ne", [1.0, 0.0]),
        dtype=float,
    ).reshape(2)
    axis_norm = float(np.linalg.norm(axis_ne))
    if not np.isfinite(axis_ne).all() or axis_norm < 1e-6:
        return np.array([1.0, 0.0], dtype=float)
    return axis_ne / axis_norm

def apf_build_encounter_params(self, **overrides):
    params = {
        "cluster_range_enabled": bool(self.apf_cluster_range_enabled),
        "risk_pc_scale": float(self.apf_risk_pc_scale),
        "avoidance_pc_scale": float(self.apf_avoidance_pc_scale),
        "direction_pc_scale": float(self.apf_direction_pc_scale),
        "virtual_pc_scale": float(self.apf_virtual_pc_scale),
        "repulsive_gain": float(self.apf_repulsive_gain),
        "route_lookahead_m": float(self.apf_route_lookahead_m),
        "collision_horizon_s": float(self.apf_collision_horizon_s),
        "constant_descent_speed_m_s": float(self.apf_constant_descent_speed_m_s)
    },
    "dynamic_speed_threshold_m_s": float(self.
apf_dynamic_speed_threshold_m_s),
    }
    params.update(overrides)
    return params

def apf_profile_name_for_encounter(self, encounter):
    if encounter == "overtaking":
        return "overtaking"
    if encounter == "head_on":
        return "head_on"
    return "crossing"

def apf_params_for_encounter(self, encounter=None):
    profile_name = self.apf_profile_name_for_encounter(
        self.apf_encounter_mode if encounter is None else encounter
    )
    if profile_name == "overtaking":
        return self.apf_overtaking_params
    if profile_name == "head_on":
        return self.apf_head_on_params
    return self.apf_crossing_params

def apf_cpa_metrics(self, obs_pos_body, obs_vel_body, own_vel_body):

```

```

        if not self.obstacle_ekf_prediction_enabled:
            return np.nan, np.nan
        tcpa_s, dcpa_m, _, _ = straight_line_cpa(
            [0.0, 0.0], own_vel_body, obs_pos_body, obs_vel_body
        )
        return tcpa_s, dcpa_m

def apf_pass_astern_side_from_velocity(self, obs_vel_body):
    obs_vel_body = np.asarray(obs_vel_body, dtype=float).reshape(2)
    if not np.isfinite(obs_vel_body).all():
        return 0.0

    if float(np.linalg.norm(obs_vel_body)) < self.
apf_dynamic_speed_threshold_m_s:
        return 0.0

    lateral_speed = float(obs_vel_body[1])
    if abs(lateral_speed) < self.apf_dynamic_speed_threshold_m_s:
        return 0.0

    # Body y is positive to starboard/right. The planner side sign is
    # positive to port/left, so the astern side has the same sign here.
    return float(np.sign(lateral_speed))

def own_prediction_velocity_ne(self):
    current_ne = np.array([float(self.North), float(self.East)], dtype=float)
    to_goal_ne = self.goal_ne - current_ne
    to_goal_distance_m = float(np.linalg.norm(to_goal_ne))
    route_direction_ne = (
        to_goal_ne / to_goal_distance_m
        if self.final_approach_active(to_goal_distance_m) and to_goal_distance_m
>= 1e-6
        else self.route_path_unit_ne
    )
    return route_direction_ne * max(
        float(self.route_tracking_speed_m_s),
        self.apf_min_forward_speed,
    )

def obstacle_track_state_at(self, track, dt_s):
    dt_s = max(float(dt_s), 0.0)
    state = np.asarray(track.get("state", [np.nan, np.nan, 0.0, 0.0]), dtype=
float).reshape(4)
    if not np.isfinite(state).all():
        return None, None

    velocity_ne = state[2:4].copy()

    accel_ne = self.obstacle_track_prediction_accel_ne(track)

    pos_ne = state[0:2] + velocity_ne * dt_s + 0.5 * accel_ne * dt_s * dt_s
    vel_ne = velocity_ne + accel_ne * dt_s
    return pos_ne, vel_ne

def virtual_collision_visuals(self):
    visuals = []
    for obstacle in self.apf_virtual_obstacles:
        if not bool(obstacle.get("predicted_risk_active", False)):

```

```

        continue
    collision_position_ne = np.asarray(
        obstacle.get("collision_position_ne", [np.nan, np.nan]),
        dtype=float,
    ).reshape(2)
    if not np.isfinite(collision_position_ne).all():
        continue
    visuals.append({
        "track_id": int(obstacle.get("track_id", 0)),
        "collision_position_ne": collision_position_ne.copy(),
        "own_prediction_ne": np.asarray(
            obstacle.get("own_prediction_ne", [np.nan, np.nan]),
            dtype=float,
        ).reshape(2),
        "tcpa_s": float(obstacle.get("tcpa_s", np.nan)),
        "predicted_separation_m": float(obstacle.get("dcpa_m", np.nan)),
        "collision_level": float(obstacle.get("collision_level", np.nan)),
    })
    return visuals

def apf_classify_encounter(self, obs_pos_body, obs_vel_body, own_vel_body):
    obs_pos_body = np.asarray(obs_pos_body, dtype=float).reshape(2)
    obs_vel_body = np.asarray(obs_vel_body, dtype=float).reshape(2)
    own_vel_body = np.asarray(own_vel_body, dtype=float).reshape(2)
    body_angle_rad = float(np.arctan2(obs_pos_body[1], obs_pos_body[0]))
    bearing_starboard_deg = float(np.rad2deg(wrap_angle(-body_angle_rad)))
    obs_speed = float(np.linalg.norm(obs_vel_body))
    own_speed = float(np.linalg.norm(own_vel_body))
    dynamic_obstacle = obs_speed >= self.apf_dynamic_speed_threshold_m_s
    relative_heading_deg = np.nan

    if "overtaking" in str(getattr(self, "webots_environment", "")).lower():
        return "overtaking", -1.0, "COLREG Rule 13: overtake on starboard side"

    if dynamic_obstacle:
        relative_heading_deg = abs(float(np.rad2deg(wrap_angle(np.arctan2(
            obs_vel_body[1], obs_vel_body[0])))))

    if np.isfinite(relative_heading_deg) and abs(bearing_starboard_deg) <= 22.5
    and relative_heading_deg >= 157.5:
        return "head_on", -1.0, "COLREG Rule 14: alter to starboard"

    if (
        np.isfinite(relative_heading_deg)
        and abs(bearing_starboard_deg) <= 67.5
        and relative_heading_deg <= 67.5
        and own_speed > obs_speed + self.apf_dynamic_speed_threshold_m_s
    ):
        return "overtaking", -1.0, "COLREG Rule 13: overtake on starboard side"

    pass_astern_side = self.apf_pass_astern_side_from_velocity(obs_vel_body)

    if dynamic_obstacle and 0.0 < bearing_starboard_deg <= 112.5:
        requested_side = pass_astern_side if pass_astern_side != 0.0 else -1.0
        return "crossing_from_starboard", requested_side, "COLREG Rule 15: give
way, pass astern"

    if dynamic_obstacle and -112.5 <= bearing_starboard_deg < 0.0:

```



```

        return "crossing_from_port", 0.0, "COLREG Rule 17: stand on"

    return "static_obstacle", 0.0, "none"

def apf_default_side_from_obstacle(self, obs_pos_body):
    body_angle_rad = float(np.arctan2(obs_pos_body[1], obs_pos_body[0]))
    if abs(wrap_angle(body_angle_rad)) > self.
apf_activation_front_half_angle_rad:
    return 0.0

    if abs(body_angle_rad) <= np.deg2rad(5.0):
        if self.left_clearance_m > self.right_clearance_m + 0.05:
            return 1.0
        if self.right_clearance_m > self.left_clearance_m + 0.05:
            return -1.0
        return -1.0

    return -1.0 if body_angle_rad > 0.0 else 1.0

def apf_obstacle_in_priority_front_sector(self, obstacle):
    obs_pos_body = np.asarray(obstacle.get("centre_body", [np.nan, np.nan]),
dtype=float).reshape(2)
    if not np.isfinite(obs_pos_body).all():
        return False

    body_angle_rad = float(np.arctan2(obs_pos_body[1], obs_pos_body[0]))
    return abs(wrap_angle(body_angle_rad)) <= self.
apf_priority_front_half_angle_rad

def apf_lock_side(self, requested_side, obstacle_level):
    now_s = float(self.timefromstart) if self.timefromstart is not None else 0.0
    obstacle_close = (
        np.isfinite(obstacle_level)
        and obstacle_level <= self.apf_side_lock_exit_level
    )

    if requested_side == 0.0:
        if self.apf_side_lock_sign != 0.0 and (obstacle_close or now_s < self.
apf_side_lock_until_s):
            self.apf_side_lock_active = True
            return self.apf_side_lock_sign

        self.apf_side_lock_sign = 0.0
        self.apf_side_lock_active = False
        return 0.0

    requested_side = float(np.sign(requested_side))
    if (
        self.apf_side_lock_sign != 0.0
        and (obstacle_close or now_s < self.apf_side_lock_until_s)
    ):
        self.apf_side_lock_active = True
        return self.apf_side_lock_sign

    self.apf_side_lock_sign = requested_side
    self.apf_side_lock_until_s = now_s + self.apf_side_lock_s
    self.apf_side_lock_active = True
    return self.apf_side_lock_sign

```

```

def refresh_apf_side_lock(self, nearest_forward_level=np.inf):
    if self.apf_side_lock_sign == 0.0:
        self.apf_side_lock_active = False
        return False

    now_s = float(self.timefromstart) if self.timefromstart is not None else 0.0
    obstacle_close = (
        np.isfinite(nearest_forward_level)
        and nearest_forward_level <= self.apf_side_lock_exit_level
    )

    if obstacle_close or now_s < self.apf_side_lock_until_s:
        self.apf_side_lock_active = True
        return True

    self.apf_side_lock_sign = 0.0
    self.apf_side_lock_active = False
    return False

def route_progress_and_point(self, lookahead_m=0.0):
    current_ne = np.array([float(self.North), float(self.East)], dtype=float)

    if self.route_path_length_m < 1e-9:
        return 0.0, self.goal_ne.copy()

    along_m = float(np.dot(current_ne - self.start_ne, self.route_path_unit_ne))
    closest_along_m = float(np.clip(along_m, 0.0, self.route_path_length_m))
    target_along_m = float(np.clip(along_m + lookahead_m, 0.0, self.
route_path_length_m))
    target_ne = self.start_ne + target_along_m * self.route_path_unit_ne
    return closest_along_m, target_ne

def goal_distance_m(self):
    current_ne = np.array([float(self.North), float(self.East)], dtype=float)
    return float(np.linalg.norm(self.goal_ne - current_ne))

def final_approach_active(self, final_distance_m=None):
    if final_distance_m is None:
        final_distance_m = self.goal_distance_m()

    if not np.isfinite(final_distance_m):
        return False

    if final_distance_m <= self.final_approach_distance_m:
        return True

    if self.route_path_length_m < 1e-9:
        return True

    current_ne = np.array([float(self.North), float(self.East)], dtype=float)
    along_m = float(np.dot(current_ne - self.start_ne, self.route_path_unit_ne))
    return along_m >= self.route_path_length_m - self.final_approach_distance_m

def apf_path_attraction_body(self):
    path_vec = self.goal_ne - self.start_ne
    path_len_sq = float(np.dot(path_vec, path_vec))
    if path_len_sq < 1e-9:

```

```

        return np.zeros(2, dtype=float)

        current_ne = np.array([float(self.North), float(self.East)], dtype=float)
        ratio = float(np.clip(np.dot(current_ne - self.start_ne, path_vec) /
path_len_sq, 0.0, 1.0))
        closest_ne = self.start_ne + ratio * path_vec
        to_path_body = self.earth_point_to_body(closest_ne)
        distance_m = float(np.linalg.norm(to_path_body))

        if distance_m < self.apf_path_threshold_m or distance_m < 1e-6:
            return np.zeros(2, dtype=float)

        return self.apf_path_gain * to_path_body

def apf_goal_attraction_body(self, target_body):
    target_body = np.asarray(target_body, dtype=float).reshape(2)
    distance_m = float(np.linalg.norm(target_body))
    if distance_m < 1e-6:
        return np.zeros(2, dtype=float)

    magnitude = self.apf_goal_gain * min(distance_m, self.
apf_attraction_saturation_m)
    return magnitude * target_body / distance_m

def apf_primary_encounter_mode(self):
    own_vel_body = self.current_velocity_body()
    obstacles = self.lidar_obstacles + self.update_apf_virtual_obstacles()
    saw_crossing = False

    for obstacle in obstacles:
        obs_pos_body = np.asarray(obstacle.get("centre_body", [np.nan, np.nan]),
dtype=float).reshape(2)
        if not np.isfinite(obs_pos_body).all():
            continue

        angle_rad = abs(wrap_angle(float(np.arctan2(obs_pos_body[1],
obs_pos_body[0]))))
        if (
            not bool(obstacle.get("virtual", False))
            and angle_rad > self.apf_activation_front_half_angle_rad
        ):
            continue

        if bool(obstacle.get("virtual", False)):
            encounter = str(obstacle.get("encounter_mode", "crossing"))
        else:
            track = self.apf_track_for_obstacle(obstacle)
            if track is None or not self.obstacle_track_motion_is_stable(track):
                continue
            track_velocity_ne = np.asarray(track.get("vel_ne", [np.nan, np.nan])
, dtype=float).reshape(2)
            if not np.isfinite(track_velocity_ne).all():
                continue
            obs_vel_body = self.earth_vector_to_body(track_velocity_ne)
            encounter, _, _ = self.apf_classify_encounter(obs_pos_body,
obs_vel_body, own_vel_body)

        if encounter == "head_on":

```

```

        return "head_on"
    if encounter == "overtaking":
        return "overtaking"
    if encounter in {"crossing_from_starboard", "crossing_from_port"}:
        saw_crossing = True

    return "crossing" if saw_crossing else "crossing"

def apf_avoidance_needed(self):
    virtual_obstacles = self.update_apf_virtual_obstacles()
    for obstacle in self.lidar_obstacles + virtual_obstacles:
        obs_pos_body = np.asarray(obstacle.get("centre_body", [np.nan, np.nan]),
dtype=float).reshape(2)
        if not np.isfinite(obs_pos_body).all():
            continue
        angle_rad = abs(wrap_angle(float(np.arctan2(obs_pos_body[1],
obs_pos_body[0]))))
        if not bool(obstacle.get("virtual", False)) and angle_rad > self.
apf_activation_front_half_angle_rad:
            continue
        if self.apf_repulsion_for_obstacle(obstacle, None, None)[1]:
            return True
    return False

def apf_overtaking_close_turn_command(
    self,
    encounter,
    nearest_obstacle_distance_m,
    force_angle,
    surge_speed,
):
    overtaking_world = "overtaking" in str(
        getattr(self, "webots_environment", "")
    ).lower()
    if (
        encounter not in {"overtaking", "static_obstacle", "none"}
        and not overtaking_world
    ) or nearest_obstacle_distance_m > self.apf_overtaking_close_distance_m:
        return force_angle, surge_speed

    turn_sign = float(np.sign(force_angle))
    if turn_sign == 0.0:
        turn_sign = float(np.sign(getattr(self, "apf_side_lock_sign", -1.0))) or
-1.0
    force_angle = turn_sign * max(
        abs(float(force_angle)),
        self.apf_overtaking_close_turn_angle_rad,
    )
    heading_offset = abs(wrap_angle(float(self.Yaw) - self.route_heading_rad))
    if heading_offset < self.apf_overtaking_turn_release_angle_rad:
        surge_speed = min(surge_speed, self.apf_overtaking_close_turn_speed_m_s)
    return force_angle, surge_speed

def compute_apf_control(self, t, u_track):
    primary_encounter = "overtaking"
    active_params = self.apf_overtaking_params
    final_approach = self.final_approach_active()
    if final_approach:

```

```

        target_ne = self.goal_ne.copy()
    else:
        _, target_ne = self.route_progress_and_point(
            float(active_params.get("route_lookahead_m", self.
apf_route_lookahead_m))
        )

    target_body = self.earth_point_to_body(target_ne)
    goal_body = self.earth_point_to_body(self.goal_ne)
    path_force = np.zeros(2, dtype=float) if final_approach else self.
apf_path_attraction_body()
    attractive_force = self.apf_goal_attraction_body(goal_body) + path_force
    force_body = attractive_force.copy()
    repulsive_force = np.zeros(2, dtype=float)
    own_vel_body = self.current_velocity_body()
    any_repulsion = False
    clearance_offset_m = 0.0
    self.reset_apf_diagnostics()
    self.apf_target_ne = target_ne.copy()
    self.apf_attractive_force_body = attractive_force

    virtual_obstacles = self.update_apf_virtual_obstacles()
    obstacles = self.lidar_obstacles + virtual_obstacles
    nearest_obstacle_distance_m = np.inf
    priority_obstacles = []
    secondary_obstacles = []
    for obstacle in obstacles:
        obs_pos_body = np.asarray(
            obstacle.get("centre_body", [np.nan, np.nan]),
            dtype=float,
        ).reshape(2)
        if (
            not bool(obstacle.get("virtual", False))
            and np.isfinite(obs_pos_body).all()
            and obs_pos_body[0] > 0.0
            and abs(float(np.arctan2(obs_pos_body[1], obs_pos_body[0])))
            <= np.deg2rad(22.5)
        ):
            nearest_obstacle_distance_m = min(
                nearest_obstacle_distance_m,
                float(np.linalg.norm(obs_pos_body)),
            )
        if self.apf_obstacle_in_priority_front_sector(obstacle):
            priority_obstacles.append(obstacle)
        else:
            secondary_obstacles.append(obstacle)

    for obstacle in priority_obstacles:
        repulsion, active, obstacle_params = self.apf_repulsion_for_obstacle(
obstacle, target_body, own_vel_body)
        repulsive_force += repulsion
        force_body += repulsion
        any_repulsion = any_repulsion or active
        if active:
            active_params = obstacle_params
            clearance_offset_m = max(
                clearance_offset_m,

```

```

        self.apf_direction_clearance_m(obstacle, params=obstacle_params)
    ,
    )

    if not any_repulsion:
        for obstacle in secondary_obstacles:
            repulsion, active, obstacle_params = self.apf_repulsion_for_obstacle
(obstacle, target_body, own_vel_body)
            repulsive_force += repulsion
            force_body += repulsion
            any_repulsion = any_repulsion or active
            if active:
                active_params = obstacle_params
                clearance_offset_m = max(
                    clearance_offset_m,
                    self.apf_direction_clearance_m(obstacle, params=
obstacle_params),
                )

    if any_repulsion and self.apf_side_lock_sign != 0.0:
        # side_sign uses the COLREG convention used throughout this
        # controller: +1 is port/left and -1 is starboard/right.
        route_normal_left_ne = np.array(
            [self.route_path_unit_ne[1], -self.route_path_unit_ne[0]],
            dtype=float,
        )
        offset_target_ne = (
            target_ne
            + self.apf_side_lock_sign
            * max(clearance_offset_m, self.obstacle_min_pc2_m)
            * route_normal_left_ne
        )
        offset_target_body = self.earth_point_to_body(offset_target_ne)
        offset_distance = float(np.linalg.norm(offset_target_body))
        if offset_distance > 1e-6:
            offset_force = (
                self.apf_clearance_gain
                * min(offset_distance, self.apf_attraction_saturation_m)
                * offset_target_body
                / offset_distance
            )
            force_body += offset_force
            attractive_force += offset_force
            self.apf_target_ne = offset_target_ne.copy()

    self.apf_attractive_force_body = attractive_force
    force_norm = float(np.linalg.norm(force_body))
    if not np.isfinite(force_norm) or force_norm < 1e-6:
        force_body = np.array([1e-3, 0.0], dtype=float)
        force_norm = float(np.linalg.norm(force_body))

    self.apf_force_body = force_body
    self.apf_repulsive_force_body = repulsive_force
    steering_force = force_body.copy()
    if any_repulsion and steering_force[0] <= 0.0:
        side_sign = float(np.sign(steering_force[1]))

    if side_sign == 0.0:

```

```

        side_sign = self.apf_side_lock_sign

        if side_sign == 0.0:
            if self.left_clearance_m > self.right_clearance_m + 0.05:
                side_sign = 1.0
            else:
                side_sign = -1.0

        lateral_mag = max(abs(float(steering_force[1])), 0.5 * abs(float(
force_body[0])), 0.20)
        steering_force[1] = side_sign * lateral_mag
        steering_force[0] = max(0.25 * lateral_mag, 0.05)

        self.apf_steering_force_body = steering_force
        now_s = float(self.timefromstart) if self.timefromstart is not None else 0.0
        self.apf_visual_hold_until_s = now_s + self.apf_visual_hold_s
        force_angle = wrap_angle(float(np.arctan2(steering_force[1], steering_force
[0])))
        force_angle = float(np.clip(force_angle, -self.apf_heading_step_limit_rad,
self.apf_heading_step_limit_rad))

        surge_speed = float(
            active_params.get("constant_descent_speed_m_s", self.
apf_constant_descent_speed_m_s)
        )
        force_angle, surge_speed = self.apf_overtaking_close_turn_command(
            primary_encounter,
            nearest_obstacle_distance_m,
            force_angle,
            surge_speed,
        )

        u_cmd = Vector(2)
        u_cmd[1, 0] = np.clip(-self.apf_heading_gain * force_angle / max(self.lastdt
, 1e-3), -self.w_max, self.w_max)
        u_cmd[0, 0] = float(np.clip(surge_speed, 0.0, self.v_max))

        if any_repulsion or self.apf_colreg_active or self.apf_side_lock_active:
            if self.apf_colreg_active:
                self.navigation_mode = "apf_colreg"
            else:
                self.navigation_mode = "apf_avoid"
        else:
            self.navigation_mode = "apf_track"

        return u_cmd

def compute_route_tracking_control(self, t):
    current_ne = np.array([self.North, self.East], dtype=float)
    final_distance = float(np.linalg.norm(self.goal_ne - current_ne))

    if self.final_approach_active(final_distance):
        ref_ne = self.goal_ne.copy()
        to_goal_ne = ref_ne - current_ne
        if final_distance > 1e-6:
            desired_heading = float(np.arctan2(to_goal_ne[1], to_goal_ne[0]))
        else:
            desired_heading = self.route_heading_rad

```

```

        heading_error = wrap_angle(float(self.Yaw) - desired_heading)
        speed_fraction = float(np.clip(final_distance / max(self.
final_slowdown_distance_m, 1e-3), 0.0, 1.0))
        if abs(heading_error) >= self.final_heading_slow_angle_rad:
            heading_speed_scale = 0.0
        else:
            heading_speed_scale = max(float(np.cos(heading_error)), 0.15)

        p_ref = Vector(3)
        p_ref[0, 0] = ref_ne[0]
        p_ref[1, 0] = ref_ne[1]
        p_ref[2, 0] = desired_heading

        u_ref = Vector(2)
        u_ref[0, 0] = self.route_tracking_speed_m_s * speed_fraction

        u_track = Vector(2)
        u_track[0, 0] = self.route_tracking_speed_m_s * speed_fraction *
heading_speed_scale
        u_track[1, 0] = 1.4 * heading_error
        return p_ref, u_ref, u_track

# Track the straight start-goal line by projecting the current position onto
# the route and aiming at a short look-ahead point on that same line.
_, ref_ne = self.route_progress_and_point(self.route_tracking_lookahead_m)

# Convert route error into the robot body frame:
# lateral error and heading error adjust yaw rate while surge stays steady.
error_body = self.earth_vector_to_body(ref_ne - current_ne)
heading_error = wrap_angle(float(self.Yaw) - self.route_heading_rad)

p_ref = Vector(3)
p_ref[0, 0] = ref_ne[0]
p_ref[1, 0] = ref_ne[1]
p_ref[2, 0] = self.route_heading_rad

u_ref = Vector(2)
u_ref[0, 0] = self.route_tracking_speed_m_s

u_track = Vector(2)
u_track[0, 0] = self.route_tracking_speed_m_s
u_track[1, 0] = -0.8 * error_body[1] + 1.2 * heading_error
return p_ref, u_ref, u_track

def groundtruth_callback(self, msg):
    self.webots_environment = msg.header.frame_id or getattr(
        self, "webots_environment", "WEBOTS_UNKNOWN"
    )
    # generate fake aruco data at a set interval
    self.pseudo_aruco_counter += 1

    t = time.time()
    pose = msg.pose
    n = pose.position.x
    e = pose.position.y
    d = pose.position.z
    ox = pose.orientation.x

```



```

    oy = pose.orientation.y
    oz = pose.orientation.z
    ow = pose.orientation.w
    q = [ox,oy,oz,ow]
    r = R.from_quat(q) # note: [x, y, z, w] order
    roll, pitch, yaw = r.as_euler('xyz', degrees=True) # radians
    yaw = np.mod(yaw, 360.0)
    self.webots_robot_truth_ne = np.array([n, e], dtype=float)
    self.webots_robot_truth_yaw_rad = np.deg2rad(yaw)

    if self.pseudo_aruco_counter== 80:
        self.pseudo_aruco_counter = 0
        self.sensed_pos_stamp_s = t
        self.sensed_pos_northings_m = n
        self.sensed_pos_eastings_m = e
        self.sensed_pos_yaw_rad = np.deg2rad(yaw)
        broadcast = True
    else:
        broadcast = False

    # log groundtruth if running webots simulation
    with self.groundtruth_log.open('a') as f:
        f.write(f"{t},{t-self.starttime},{n},{e},{d},{roll},{pitch},{yaw},{
broadcast}\n")

def feedback_control(self, ds, ks = None, kn = None, kg = None):

    if ks == None: ks = 0.1
    if kn == None: kn = 0.1
    if kg == None: kg = 0.1

    dv = ks*ds[0]
    dw = kn*ds[1]+kg*ds[2]

    du = Vector(2)

    du[0] = dv
    du[1] = dw

    return du
def motion_model(self, state, control_input, dt):
    """
    EKF motion model:
    state x = [N, E, G, Ndot, Edot, Gdot]^T
    control_input = T = [T_R, T_L]^T (thruster forces in N)

    Returns:
        predicted_state (6x1 Vector)
        F                (6x6 Jacobian)
    """

    # Thruster forces in body frame from allocation matrix
    Fb = self.G @ control_input # [Fx, Fy, tau_z]^T in body frame

    # Convert thrust from body to earth frame
    H_eb = HomogeneousTransformation(state[N:E+1], state[G])
    Fe = H_eb.H_R @ Fb # [Fx_e, Fy_e, tau_z_e]^T

```

```

# Dynamics in earth frame
ve = self.robot.model(
    dynamics_translation_e,
    dynamics_rotation_e,
    Fe,
    state[DOTN:DOTG+1],
    dt,
) # ve = [Ndot, Edot, Gdot]^T

# Body-frame velocity
vb = Inverse(H_eb.H_R) @ ve

# Twist used for kinematics
u = Vector(2)
u[0, 0] = vb[0, 0] # surge speed v
u[1, 0] = vb[2, 0] # yaw rate w

# Pose update
p = rigid_body_kinematics(state[N:G+1], u, dt)
p[2, 0] = p[2, 0] % (2 * np.pi)

# Build predicted state vector
predicted_state = Vector(6)
predicted_state[N] = p[0]
predicted_state[E] = p[1]
predicted_state[G] = p[2]
predicted_state[DOTN] = ve[0]
predicted_state[DOTE] = ve[1]
predicted_state[DOTG] = ve[2]

# Simple Jacobian: integrate velocity (good enough for EKF here)
F = Identity(6)
F[N, DOTN] = dt
F[E, DOTE] = dt
F[G, DOTG] = dt

return predicted_state, F

def thruster_force_limits(self):
    if self.unbounded_overtaking_speed_enabled():
        return -np.inf, np.inf

    max_rpm = self.prop_rate_limit_rad_s * 60.0 / (2.0 * np.pi)
    forward_force = float(rpm2N(max_rpm))
    reverse_force = float(rpm2N(-max_rpm))

    if not np.isfinite(forward_force) or forward_force <= 0.0:
        forward_force = 1.0

    if not np.isfinite(reverse_force) or reverse_force >= 0.0:
        reverse_force = -0.5 * forward_force

    return reverse_force, forward_force

def unbounded_overtaking_speed_enabled(self):
    return False

def allocate_propulsion_rates(self, v_cmd, w_cmd):

```

```

v_cmd = float(v_cmd) if np.isfinite(v_cmd) else 0.0
w_cmd = float(w_cmd) if np.isfinite(w_cmd) else 0.0
v_cmd = max(v_cmd, 0.0)

desired_force_x = self.robot.k_drag * v_cmd * abs(v_cmd)
desired_tau_z = self.robot.B_66 * w_cmd
reverse_force, forward_force = self.thruster_force_limits()

yaw_arm = 0.5 * (float(self.G[2, 0]) - float(self.G[2, 1]))
if abs(yaw_arm) < 1e-6:
    thrust = np.linalg.pinv(self.G) @ 12m([desired_force_x, 0.0,
desired_tau_z])
    right_force = float(np.clip(thrust[0, 0], reverse_force, forward_force))
    left_force = float(np.clip(thrust[1, 0], reverse_force, forward_force))
else:
    # Preserve yaw authority first. If the requested surge and yaw cannot
    # both fit within the prop limits, reduce surge instead of losing turn.
    desired_delta = desired_tau_z / yaw_arm
    max_delta = max(forward_force - reverse_force, 1e-6)
    delta = float(np.clip(desired_delta, -max_delta, max_delta))

    force_lower = max(
        0.0,
        2.0 * reverse_force - delta,
        2.0 * reverse_force + delta,
    )
    force_upper = min(
        2.0 * forward_force - delta,
        2.0 * forward_force + delta,
    )

    if force_upper < force_lower:
        force_x = max(0.0, min(desired_force_x, 2.0 * forward_force))
    else:
        force_x = float(np.clip(desired_force_x, force_lower, force_upper))

    right_force = 0.5 * (force_x + delta)
    left_force = 0.5 * (force_x - delta)
    right_force = float(np.clip(right_force, reverse_force, forward_force))
    left_force = float(np.clip(left_force, reverse_force, forward_force))

if self.unbounded_overtaking_speed_enabled():
    rpm_R = -force_to_rpm_unbounded(right_force)
    rpm_L = force_to_rpm_unbounded(left_force)
    right_rate = float(rpm_R * (2.0 * np.pi / 60.0))
    left_rate = float(rpm_L * (2.0 * np.pi / 60.0))
else:
    rpm_R = -N2rpm(right_force)
    rpm_L = N2rpm(left_force)
    right_rate = float(np.clip(rpm_R * (2.0 * np.pi / 60.0), -self.
prop_rate_limit_rad_s, self.prop_rate_limit_rad_s))
    left_rate = float(np.clip(rpm_L * (2.0 * np.pi / 60.0), -self.
prop_rate_limit_rad_s, self.prop_rate_limit_rad_s))
    return right_rate, left_rate

def empty_measurement(x):
    H = Matrix(5)
    return x, H

```

```

##### MAIN ROBOT LOOP #####
def loop(self):
    """This main loop is completed every 0.2 seconds.
    Once initialised, it repeats until stopped.
    It runs sequentially so consider how to structure your code.
    You won't receive data from the IMU or ARUCO in every loop.
    Don't make the loop rely on new data.
    """
    current_epoch_s = time.time()
    self.timefromstart = current_epoch_s - self.starttime

    ### RECEIVE SENSOR DATA #####
    self.sensed_pos_stamp_s = None
    self.sensed_pos_northings_m = None
    self.sensed_pos_eastings_m = None
    self.sensed_pos_yaw_rad = None

    sensed_pos = self.aruco_driver.read()
    if sensed_pos is not None:
        self.sensed_pos_stamp_s = sensed_pos[0]
        self.sensed_pos_northings_m = sensed_pos[1]
        self.sensed_pos_eastings_m = sensed_pos[2]
        self.sensed_pos_yaw_rad = sensed_pos[6]
        print(
            "Received position update from",
            current_epoch_s - self.sensed_pos_stamp_s,
            "seconds ago",
        )

    if self.initialise_pose and self.sensed_pos_northings_m is not None:
        self.mu[N] = self.sensed_pos_northings_m
        self.mu[E] = self.sensed_pos_eastings_m
        self.mu[G] = self.sensed_pos_yaw_rad
        self.mu[DOTN] = 0
        self.mu[DOTE] = 0
        self.mu[DOTG] = 0

        self.p_robot[0] = self.mu[N]
        self.p_robot[1] = self.mu[E]
        self.p_robot[2] = self.mu[G]
        self.v_robot[0] = self.mu[DOTN]
        self.v_robot[1] = self.mu[DOTE]
        self.v_robot[2] = self.mu[DOTG]
        self.integrated_yaw = self.sensed_pos_yaw_rad
        self.initialise_pose = False
        print("Initialised pose")

    imu_fresh = (
        self.sensed_imu_stamp_s is not None
        and current_epoch_s - self.sensed_imu_stamp_s < self.lastdt
    )

    if imu_fresh:
        if self.sensed_imu_prev_stamp_s is not None:
            dt_imu = self.sensed_imu_stamp_s - self.sensed_imu_prev_stamp_s
            if dt_imu <= 0 or dt_imu > 1.0:
                dt_imu = self.lastdt

```

```

else:
    dt_imu = self.lastdt

print(
    "Received IMU update from",
    current_epoch_s - self.sensed_imu_stamp_s,
    "seconds ago",
)

self.sensed_imu_prev_stamp_s = self.sensed_imu_stamp_s
self.sensed_yaw_rate = self.sensed_imu_yaw_rate_rad_s
self.integrated_yaw += self.sensed_yaw_rate * dt_imu
self.integrated_yaw %= 2 * np.pi

if (
    self.sensed_bottom_depth_stamp_s is not None
    and current_epoch_s - self.sensed_bottom_depth_stamp_s < self.lastdt
):
    print(
        "Received Echosounder update from",
        current_epoch_s - self.sensed_bottom_depth_stamp_s,
        "seconds ago",
    )

if self.sensed_imu_stamp_s is not None or self.OPERATING_MODE == 2:
    ### EKF PREDICT/UPDATE #####
    rpm_limit = None if self.unbounded_overtaking_speed_enabled() else 2000
    reverse_rpm_limit = None if rpm_limit is None else -rpm_limit
    right_N = rpm2N(
        -self.right_rate * 60 / (2 * np.pi),
        rpm_limit,
        reverse_rpm_limit,
    )
    left_N = rpm2N(
        self.left_rate * 60 / (2 * np.pi),
        rpm_limit,
        reverse_rpm_limit,
    )
    u_thrusters = l2m([right_N, left_N])
    self.mu, self.Sigma = extended_kalman_filter_predict(
        self.mu,
        self.Sigma,
        u_thrusters,
        self.motion_model,
        self.Q,
        self.lastdt,
    )

    if self.sensed_pos_stamp_s is not None:
        z_pose = Vector(6)
        z_pose[N] = self.sensed_pos_northings_m
        z_pose[E] = self.sensed_pos_eastings_m
        z_pose[G] = self.sensed_pos_yaw_rad

        self.mu, self.Sigma = extended_kalman_filter_update(
            self.mu,
            self.Sigma,
            z_pose,

```

```

        h_pose_update,
        self.R_pose,
        wrap_index=G,
    )

    if imu_fresh and self.sensed_yaw_rate is not None:
        z_rate = Vector(6)
        z_rate[DOTG] = self.sensed_yaw_rate

        self.mu, self.Sigma = extended_kalman_filter_update(
            self.mu,
            self.Sigma,
            z_rate,
            h_grate_update,
            self.R_grate,
        )

    self.p_robot[0] = self.mu[N]
    self.p_robot[1] = self.mu[E]
    self.p_robot[2] = self.mu[G]
    self.v_robot[0] = self.mu[DOTN]
    self.v_robot[1] = self.mu[DOT E]
    self.v_robot[2] = self.mu[DOTG]
    self.last_nav_t = current_epoch_s
    self.Yaw = self.p_robot[2][0]
    self.North = self.p_robot[0][0]
    self.East = self.p_robot[1][0]

    ### ROUTE TRACKING + MODIFIED APF CONTROL #####
    t = self.timefromstart
    _, _, u_track = self.compute_route_tracking_control(t)
    final_distance = self.goal_distance_m()
    final_approach = self.final_approach_active(final_distance)

    if final_distance <= self.goal_tolerance_m:
        self.goal_reached = True
        if np.isnan(self.s.t_complete):
            self.s.t_complete = self.timefromstart

    if self.goal_reached:
        self.u = Vector(2)
        self.navigation_mode = "arrived"
        self.reset_apf_diagnostics()
    elif self.apf_avoidance_needed():
        self.u = self.compute_apf_control(t, u_track)
    else:
        self.u = u_track
        self.navigation_mode = "track"
        self.reset_apf_diagnostics(clear_visual=not self.
apf_visual_hold_active())

    if self.goal_reached:
        self.u[0, 0] = 0.0
        self.u[1, 0] = 0.0
    else:
        if not final_approach:
            self.u[1, 0] = self.limit_heading_deviation_command(self.u[1,
0])

```

```

        self.u[1, 0] = np.clip(self.u[1, 0], -self.w_max, self.w_max)
        self.u[0, 0] = np.clip(self.u[0, 0], 0.0, self.v_max)
        self.prev_sensed = t
        self.U = self.u.T

        v = float(self.U[0][0])
        w = float(self.U[0][1])
        if self.goal_reached:
            self.right_rate = 0.0
            self.left_rate = 0.0
        else:
            self.right_rate, self.left_rate = self.allocate_propulsion_rates(v,
w)

        control_msg = Vector3()
        control_msg.x = int(self.right_rate)
        control_msg.y = int(self.left_rate)
        control_msg.z = float(self.route_tracking_speed_m_s)

        self.control_pub.publish(control_msg)
        print(
            'Navigation mode:', self.navigation_mode,
            'encounter:', self.apf_encounter_mode,
            'side=', int(self.apf_avoidance_side_sign),
            'DCPA=', round(float(self.apf_colreg_dcpa_m), 2) if np.isfinite(self.
apf_colreg_dcpa_m) else 'nan',
            'TCPA=', round(float(self.apf_colreg_tcpa_s), 2) if np.isfinite(self.
apf_colreg_tcpa_s) else 'nan',
            'v=', round(float(v), 3),
            'w=', round(float(w), 3),
            'F_body=', np.round(self.apf_force_body, 3),
        )
        print('Prop rates: R=',self.right_rate,', L=',self.left_rate,'rad/s')

        nearest_obstacle_north = np.nan
        nearest_obstacle_east = np.nan
        nearest_obstacle_distance = np.nan
        if self.nearest_lidar_obstacle is not None:
            centre_ne = np.asarray(
                self.nearest_lidar_obstacle.get("centre_ne", [np.nan, np.nan]),
                dtype=float,
            ).reshape(2)
            nearest_obstacle_north = centre_ne[0]
            nearest_obstacle_east = centre_ne[1]
            nearest_obstacle_distance = float(
                self.nearest_lidar_obstacle.get(
                    "min_distance_m",
                    self.nearest_lidar_obstacle.get("distance_m", np.nan),
                )
            )

        ### LOG DATA #####
        with self.filename.open("a") as f:
            f.write(f"{current_epoch_s},{self.timefromstart},{self.right_rate},{self.
left_rate},{self.lastdt},{self.Yaw},{self.North},{self.East},{self.
sensed_yaw_rate},{self.integrated_yaw},{self.sensed_imu_stamp_s},{self.
sensed_pos_northings_m},{self.sensed_pos_eastings_m},{self.sensed_pos_yaw_rad},
{self.sensed_pos_stamp_s}, {self.sensed_bottom_depth_stamp_s},{self.

```

```

sensed_bottom_depth_m},{self.navigation_mode},{self.apf_encounter_mode},{self.
apf_avoidance_side_sign},{self.apf_colreg_dcpa_m},{self.apf_colreg_tcpa_s},{self.
apf_force_body[0]},{self.apf_force_body[1]},{nearest_obstacle_north},{
nearest_obstacle_east},{nearest_obstacle_distance}\n")
    self.write_obstacle_snapshot()

    ### VISUALISE DATA #####
    if self.OPERATING_MODE != 0:
        reference_path = getattr(self.s, "P_arc", getattr(self.s, "P", None))
        mission_complete = not np.isnan(getattr(self.s, "t_complete", np.nan))
        return(self.right_rate, self.left_rate, self.lastdt, self.Yaw, self.
North, self.East, self.sensed_yaw_rate,self.integrated_yaw, self.
sensed_imu_stamp_s, self.sensed_pos_northings_m, self.sensed_pos_eastings_m,
self.sensed_pos_yaw_rad, self.sensed_pos_stamp_s, self.waypoints, reference_path,
self.sensed_bottom_depth_m, self.sensed_bottom_depth_stamp_s, mission_complete,
self.lidar_data)

##### END MAIN LOOP #####

def main():
    LaptopController(OPERATING_MODE=0)

"""Unified COLREG controller with embedded strategy implementations."""

from contextlib import contextmanager
import importlib.util
import json
import os
from pathlib import Path
import re
import subprocess
import sys
from types import MethodType

import numpy as np
from behavior_tree import BTAction, BTCondition, BTSelector, BTSequence, BTStatus
from colreg_apf import classify_colreg_zone, obstacle_stern_waypoint,
    smooth_ellipse_repulsion, straight_line_cpa

_HERE = Path(__file__).resolve().parent

def _load_source_module(name, filename):
    path = _HERE / filename
    spec = importlib.util.spec_from_file_location(name, path)
    if spec is None or spec.loader is None:
        raise ImportError(f"Cannot load {path}")
    module = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(module)
    return module

_overtaking = sys.modules[__name__]
_head_on = _load_source_module("_laptop_head_on_source", "laptop-headon.py")

```



```

_crossing = _load_source_module("_laptop_crossing_source", "laptop-crossing.py")

_OvertakingController = _ControllerCore
_HeadOnController = _head_on.LaptopController
_CrossingController = _crossing.LaptopController

def segment_intersects_ellipse(start_ne, end_ne, centre_ne, axis_ne, half_long_m,
    half_lateral_m, margin_m=0.0):
    """Whether a route segment enters an APF ellipse (including its safety margin).
    """
    start = np.asarray(start_ne, dtype=float).reshape(2)
    end = np.asarray(end_ne, dtype=float).reshape(2)
    centre = np.asarray(centre_ne, dtype=float).reshape(2)
    axis = np.asarray(axis_ne, dtype=float).reshape(2)
    axis_norm = float(np.linalg.norm(axis))
    half_long_m = float(half_long_m)
    half_lateral_m = float(half_lateral_m)
    if (
        not np.isfinite(np.r_[start, end, centre, axis]).all()
        or axis_norm < 1e-9
        or half_long_m <= 0.0
        or half_lateral_m <= 0.0
    ):
        return False
    axis /= axis_norm
    lateral = np.array([-axis[1], axis[0]])
    scale = np.array([half_long_m + margin_m, half_lateral_m + margin_m])
    offset = np.array([np.dot(start - centre, axis), np.dot(start - centre, lateral)
    ]) / scale
    direction = np.array([np.dot(end - start, axis), np.dot(end - start, lateral)])
    / scale
    denominator = float(np.dot(direction, direction))
    alpha = 0.0 if denominator < 1e-12 else float(np.clip(-np.dot(offset, direction)
    / denominator, 0.0, 1.0))
    return float(np.linalg.norm(offset + alpha * direction)) <= 1.0

def _switches_from_combination(value):
    value = str(value).strip().lower()
    match = re.fullmatch(r"ekf_(on|off)_cluster_(on|off)", value)
    if not match:
        raise ValueError(
            "SWITCH_COMBINATION must be ekf_<on|off>_cluster_<on|off>"
        )
    return value, match.group(1) == "on", match.group(2) == "on"

SWITCH_COMBINATION, ENABLE_OBSTACLE_EKF_PREDICTION, ENABLE_CLUSTER_BASED_APF_RANGE =
    (
        _switches_from_combination(os.environ.get("SWITCH_COMBINATION", "
        ekf_on_cluster_on"))
    )

def _mode_value(robot_value, simulation_value):
    return lambda controller: simulation_value if controller.OPERATING_MODE == 2
    else robot_value

```

```

# APF parameters whose values are identical in all three strategy files.
# Parameters with different original values stay
# strategy-specific and are not listed here.
UNIFIED_APF_PARAMS = {
    "apf_risk_pc_scale": 2.5,
    "apf_avoidance_pc_scale": 2.5,
    "apf_direction_pc_scale": 2.5,
    "apf_virtual_pc_scale": 4.0,
    "apf_activation_front_half_angle_rad": np.deg2rad(150.0),
    "apf_priority_front_half_angle_rad": np.deg2rad(90.0),
    "apf_goal_gain": 5.5,
    "apf_path_gain": 7.5,
    # Single calibrated gain for both dynamic ellipse fields. Their geometry
    # remains 2.5x (current) and 4x (predicted).
    "apf_repulsive_gain": 3.0,
    "apf_attraction_saturation_m": 3.5,
    "apf_path_threshold_m": 0.10,
    "apf_route_lookahead_m": _mode_value(1.8, 2.1),
    "apf_collision_horizon_s": _mode_value(6.0, 10.0),
    "apf_prediction_dt_s": 0.5,
    "apf_heading_gain": 0.9,
    "apf_heading_step_limit_rad": np.deg2rad(60.0),
    "apf_constant_descent_speed_m_s": lambda controller: controller.
route_tracking_speed_m_s,
    "apf_dynamic_speed_threshold_m_s": _mode_value(0.05, 0.06),
    "apf_dynamic_exit_speed_threshold_m_s": _mode_value(0.03, 0.04),
    "apf_crossing_pass_ahead_surge_m_s": lambda controller: (
        controller.route_tracking_speed_m_s + (0.18 if controller.OPERATING_MODE ==
2 else 0.08)
    ),
    "apf_crossing_pass_ahead_safe_dcpa_m": _mode_value(0.45, 0.60),
    "apf_track_association_m": _mode_value(0.60, 0.80),
    "apf_track_timeout_s": _mode_value(1.0, 1.5),
    "obstacle_axis_smoothing_alpha": 0.20,
    "obstacle_axis_min_aspect_ratio": 1.4,
    "obstacle_axis_max_velocity_gap_rad": np.deg2rad(30.0),
    "obstacle_min_pc1_m": _mode_value(0.30, 0.36),
    "obstacle_min_pc2_m": _mode_value(0.16, 0.20),
    "apf_own_equivalent_radius_m": _mode_value(0.25, 0.30),
    "apf_side_lock_s": _mode_value(5.0, 8.0),
    "apf_side_lock_exit_level": 1.15,
    "apf_visual_hold_s": 2.0,
}

def _sync_strategy_switches():
    for module in (_overtaking, _head_on, _crossing):
        module.ENABLE_OBSTACLE_EKF_PREDICTION = bool(ENABLE_OBSTACLE_EKF_PREDICTION)
        module.ENABLE_CLUSTER_BASED_APF_RANGE = bool(ENABLE_CLUSTER_BASED_APF_RANGE)

_sync_strategy_switches()

_CSV_CONTEXT_COLUMNS = (
    "WebotsEnvironment",
    "SwitchCombination",

```

```

        "EKFPredictionEnabled",
        "ClusterSizeAPFEnabled",
    )

def _apply_unified_apf_params(controller):
    for name, value in UNIFIED_APF_PARAMS.items():
        setattr(controller, name, value(controller) if callable(value) else value)

    controller.obstacle_min_equivalent_radius_m = 0.5 * controller.
    obstacle_min_pc1_m

    if hasattr(controller, "apf_build_encounter_params"):
        controller.apf_crossing_params = controller.apf_build_encounter_params()
        controller.apf_overtaking_params = controller.apf_build_encounter_params()
        controller.apf_head_on_params = controller.apf_build_encounter_params()

def _world_name_from_text(text):
    match = re.search(r'["\']([^\']+\.\wbt)[ "\']|(\S+\.\wbt)', str(text), flags=re.
    IGNORECASE)
    if not match:
        return None
    return Path(match.group(1) or match.group(2)).name

def _detect_webots_environment(operating_mode):
    if operating_mode != 2:
        return "not_webots"

    for name in (
        "WEBOTS_WORLD",
        "WEBOTS_WORLD_FILE",
        "WEBOTS_CURRENT_WORLD",
        "WEBOTS_SCENARIO",
        "WORLD_FILE",
    ):
        value = os.environ.get(name)
        if not value:
            continue
        return _world_name_from_text(value) or Path(value).name or value

    try:
        if os.name == "nt":
            commands = (
                ["wmic", "process", "where", "name like '%webots%', "get", "
                CommandLine", "/value"],
                ["powershell", "-NoProfile", "-Command", "Get-CimInstance -Query \"
                SELECT CommandLine FROM Win32_Process WHERE Name LIKE '%webots%'\" | Select-
                Object -ExpandProperty CommandLine"],
            )
        else:
            commands = (["ps", "-eo", "args"],)
        for command in commands:
            result = subprocess.run(
                command,
                capture_output=True,
                text=True,

```

```

        timeout=0.8,
    )
    detected = _world_name_from_text(result.stdout)
    if detected:
        return detected
except Exception:
    pass

return "WEBOTS_UNKNOWN"

_MISSING = object()

_CROSSING_METHODS = (
    "apf_obstacle_endpoint_direction_body",
    "apf_stern_direction_body",
    "apf_bow_direction_body",
    "apf_crossing_strategy_from_velocity",
    "apf_encounter_speed_m_s",
    "apf_obstacle_in_forward_half_plane",
)

_HEAD_ON_METHODS = (
    "apf_track_for_obstacle",
    "apf_build_encounter_params",
    "apf_profile_name_for_encounter",
    "apf_params_for_encounter",
    "apf_cpa_metrics",
    "own_prediction_velocity_ne",
    "obstacle_track_state_at",
    "virtual_collision_visuals",
    "apf_default_side_from_obstacle",
    "apf_obstacle_in_priority_front_sector",
    "apf_lock_side",
    "refresh_apf_side_lock",
    "route_progress_and_point",
    "goal_distance_m",
    "final_approach_active",
    "apf_path_attraction_body",
    "apf_goal_attraction_body",
)

class LaptopController(_OvertakingController):
    """One live controller instance with COLREG strategy dispatch."""

    _HEAD_ON_RULES = {"head_on"}
    _OVERTAKING_RULES = {"overtaking", "being_overtaken"}
    _CROSSING_RULES = {"crossing", "crossing_from_starboard", "crossing_from_port"}

    def __init__(self, OPERATING_MODE):
        _sync_strategy_switches()
        super().__init__(OPERATING_MODE)
        # Keep the live controller state tied to laptop.py's unified switches.
        self.obstacle_ekf_prediction_enabled = bool(ENABLE_OBSTACLE_EKF_PREDICTION)
        self.apf_cluster_range_enabled = bool(ENABLE_CLUSTER_BASED_APF_RANGE)

```

```

_apply_unified_apf_params(self)
self.apf_selected_controller = "default_apf"
self._last_colreg_decision = None
self.webots_environment = _detect_webots_environment(self.OPERATING_MODE)
self.apf_current_field_size_scale = 4.0
self.apf_predicted_field_size_scale = 7.0
# ponytail: constant route acceleration; replace with the propulsion
# model only if measured acceleration materially improves CPA timing.
self.apf_own_acceleration_m_s2 = 0.25 if self.OPERATING_MODE == 2 else 0.15
self.apf_stand_on_emergency_tcpa_s = 3.0
self.apf_last_dynamic_field_s = -np.inf
self._csv_context_last_patched_line_end = None
self._ensure_csv_context_header()
# Defaults used directly by laptop-crossing.py helpers when they run on
# this single overtaking-initialised controller instance.
self.apf_boundary_activation_level = getattr(self, "
apf_boundary_activation_level", 1.35)
self.apf_boundary_inside_boost = getattr(self, "apf_boundary_inside_boost",
3.0)
self.apf_min_detour_offset_m = getattr(
    self,
    "apf_min_detour_offset_m",
    1.0 if self.OPERATING_MODE == 2 else 0.8,
)
self.apf_pass_ahead_gain = getattr(self, "apf_pass_ahead_gain", 2.4)
self.apf_crossing_min_forward_speed = getattr(
    self,
    "apf_crossing_min_forward_speed",
    0.14 if self.OPERATING_MODE == 2 else 0.16,
)
self.apf_crossing_close_quarters_surge_m_s = getattr(
    self,
    "apf_crossing_close_quarters_surge_m_s",
    0.18 if self.OPERATING_MODE == 2 else 0.16,
)
self.obstacle_track_confirmation_hits = getattr(
    self,
    "obstacle_track_confirmation_hits",
    2,
)
self.obstacle_prediction_min_samples = getattr(
    self,
    "obstacle_prediction_min_samples",
    2,
)
self.obstacle_prediction_min_hits = getattr(
    self,
    "obstacle_prediction_min_hits",
    2,
)
# A full two-second EKF window rejects the repeating LiDAR visible-face
# jump before a target course is trusted for future-field placement.
self.obstacle_prediction_min_time_span_s = 2.0
self.obstacle_prediction_min_displacement_m = getattr(
    self,
    "obstacle_prediction_min_displacement_m",
    0.03,
)

```

```

self.apf_crossing_longitudinal_scale = getattr(
    self,
    "apf_crossing_longitudinal_scale",
    1.3,
)
self.apf_crossing_lateral_scale = getattr(
    self,
    "apf_crossing_lateral_scale",
    5.2,
)
self._mission_waypoints = list(getattr(self, "waypoints", []))
# Match the strategy controllers so position noise near the goal does
# not keep the vessel circling a point it has effectively reached.
self.goal_tolerance_m = 0.20
self.apf_waypoint_detour_enabled = True
self.apf_waypoint_acceptance_m = 0.45 if self.OPERATING_MODE == 2 else 0.30
self.apf_waypoint_replan_interval_s = 0.6 if self.OPERATING_MODE == 2 else
0.8
self.apf_waypoint_entry_margin_m = max(
    self.route_tracking_lookahead_m,
    1.2 if self.OPERATING_MODE == 2 else 0.8,
)
self.apf_waypoint_merge_margin_m = 1.8 if self.OPERATING_MODE == 2 else 1.1
self.apf_waypoint_lateral_margin_m = 0.45 if self.OPERATING_MODE == 2 else
0.28
self.apf_waypoint_path_ne = []
self.apf_waypoint_index = 0
self.apf_waypoint_side_sign = 0.0
self.apf_waypoint_merge_along_m = 0.0
self.apf_waypoint_planned_at_s = -np.inf
self.apf_waypoint_step_m = 0.90 if self.OPERATING_MODE == 2 else 0.55
self.apf_waypoint_preview_points = 6
self.apf_waypoint_force_blend = 0.65
self.apf_waypoint_speed_m_s = float(self.route_tracking_speed_m_s)
self._plan_initial_route()
self._colreg_recognition_bt = self._build_colreg_recognition_behavior_tree()
self._colreg_avoidance_bt = self._build_colreg_avoidance_behavior_tree()
self._colreg_bt = self._build_colreg_behavior_tree()

def _current_ne(self):
    return np.array([float(self.North), float(self.East)], dtype=float)

def _plan_initial_route(self):
    """Own the only persistent route; APF paths are temporary overlays."""
    self.main_route_ne = [self.start_ne.copy(), self.goal_ne.copy()]
    waypoint_type = type(self.waypoints[0]) if self.waypoints else None
    if waypoint_type is not None:
        self.waypoints = []
        for point_ne in self.main_route_ne:
            waypoint = waypoint_type()
            waypoint.y, waypoint.x = map(float, point_ne)
            self.waypoints.append(waypoint)
    self._mission_waypoints = list(self.waypoints)

def apf_obstacle_level_and_away(self, obstacle, *_args, **_kwargs):
    centre_body = self._obstacle_body_position(obstacle)
    axis_body = self.earth_vector_to_body(self.obstacle_length_axis_ne(obstacle)
)

```

```

    pc1_m, pc2_m = self.obstacle_pc_dimensions(obstacle)
    return ellipse_level_and_away(
        -centre_body,
        axis_body,
        0.5 * pc1_m + self.apf_own_equivalent_radius_m,
        0.5 * pc2_m + self.apf_own_equivalent_radius_m,
    )

def obstacle_pc_dimensions(self, obstacle):
    if not self.apf_cluster_range_enabled:
        return float(self.obstacle_min_pc1_m), float(self.obstacle_min_pc2_m)
    return _CrossingController.obstacle_pc_dimensions(self, obstacle)

def obstacle_length_axis_ne(self, obstacle):
    return _CrossingController.obstacle_length_axis_ne(self, obstacle)

def apf_direction_clearance_m(self, obstacle, *_args, **kwargs):
    _, pc2_m = self.obstacle_pc_dimensions(obstacle)
    outer_scale = (
        self.apf_predicted_field_size_scale
        if bool(obstacle.get("virtual", False))
        else self.apf_current_field_size_scale
    )
    return outer_scale * (0.5 * pc2_m + self.apf_own_equivalent_radius_m)

def apf_pass_astern_side_from_velocity(self, obs_vel_body):
    # Body y and planner side are both positive to port/left. The
    # obstacle's trailing side uses the same sign in this planner.
    return _OvertakingController.apf_pass_astern_side_from_velocity(self,
obs_vel_body)

# The APF has exactly two repulsive fields: the measured LiDAR ellipse and
# the EKF/CPA-predicted ellipse. COLREG still selects the controller, but
# never injects an additional force into either field.
def _dynamic_ellipse_repulsion(self, obstacle, outer_scale):
    centre_body = self._obstacle_body_position(obstacle)
    if not np.isfinite(centre_body).all():
        return np.zeros(2, dtype=float), False
    axis_body = self.earth_vector_to_body(
        self.obstacle_length_axis_ne(obstacle)
    )
    pc1_m, pc2_m = self.obstacle_pc_dimensions(obstacle)
    # Measured, bridge, and final virtual fields are all evaluated at
    # their actual map positions. DCPA is telemetry only, not a second
    # force location.
    evaluation_offset_body = -centre_body
    level, away = ellipse_level_and_away(
        evaluation_offset_body,
        axis_body,
        0.5 * pc1_m + self.apf_own_equivalent_radius_m,
        0.5 * pc2_m + self.apf_own_equivalent_radius_m,
    )
    if level >= float(outer_scale):
        return np.zeros(2, dtype=float), False
    # Smoothly fades from the hull boundary (level=1) to zero at the
    # requested size multiple; inside the measured ellipse remains unsafe.
    size_gain = np.sqrt(max(pc1_m * pc2_m, 1e-6))
    gain = self.apf_repulsive_gain * np.clip(size_gain, 0.5, 2.0)

```

```

        return smooth_ellipse_repulsion(
            level,
            away,
            gain,
            outer_scale,
        )

def apf_repulsion_for_obstacle(self, obstacle, target_body, own_vel_body):
    del target_body, own_vel_body
    force, active = self._dynamic_ellipse_repulsion(
        obstacle,
        float(getattr(
            self,
            "apf_predicted_field_size_scale"
            if bool(obstacle.get("virtual", False))
            else "apf_current_field_size_scale",
            4.0 if bool(obstacle.get("virtual", False)) else 2.5,
        )),
    )
    if active:
        self.apf_last_dynamic_field_s = (
            float(self.timefromstart) if self.timefromstart is not None else 0.0
        )
        obs_pos_body = self._obstacle_body_position(obstacle)
        obs_vel_body = self._obstacle_body_velocity(obstacle)
        own_vel_body = self.current_velocity_body()
        encounter, requested_side, action = self.apf_classify_encounter(
            obs_pos_body,
            obs_vel_body,
            own_vel_body,
        )
        if bool(getattr(self, "apf_head_on_latched", False)):
            encounter = "head_on"
            requested_side = -1.0
            action = "COLREG Rule 14: alter to starboard (latched)"
        elif bool(getattr(self, "apf_overtaking_latched", False)):
            encounter = "overtaking"
            requested_side = -1.0
            action = "COLREG Rule 13: overtake on starboard side (EKF-latched)"
        elif encounter in self._CROSSING_RULES:
            stern_side = self.apf_pass_astern_side_from_velocity(obs_vel_body)
            if stern_side != 0.0:
                requested_side = stern_side
                action = "Crossing: pass astern of obstacle ship"
        tcpa_s, dcpa_m = self.apf_cpa_metrics(
            obs_pos_body,
            obs_vel_body,
            own_vel_body,
        )
        if bool(getattr(self, "apf_overtaking_latched", False)):
            # Rule 13 must not inherit a port-side lock established by a
            # preceding crossing classification.
            self.apf_side_lock_sign = -1.0
            self.apf_side_lock_until_s = (
                float(self.timefromstart) if self.timefromstart is not None else
0.0
            ) + self.apf_side_lock_s
            self.apf_side_lock_active = True

```



```

        locked_side = -1.0
    else:
        locked_side = self.apf_lock_side(requested_side, 1.0)
    self.apf_encounter_mode = encounter
    self.apf_colreg_rule = action
    self.apf_avoidance_side_sign = locked_side
    self.apf_colreg_dcpa_m = dcpa_m
    self.apf_colreg_tcpa_s = tcpa_s
    self.apf_colreg_active = locked_side != 0.0

    # Overtaking/head-on legacy loops also consume their profile mapping.
    # It is retained solely for that call signature; it adds no force.
    return force, active, getattr(self, "apf_overtaking_params", {})

def _predict_unavoided_route(self, times_s):
    """Predict the configured start-to-goal motion before APF avoidance."""
    times_s = np.asarray(times_s, dtype=float)
    route = np.asarray(self.route_path_unit_ne, dtype=float).reshape(2)
    current_ne = self._current_ne()
    along_now = float(np.dot(current_ne - self.start_ne, route))
    velocity_ne = np.asarray(getattr(self, "v_robot", np.zeros((3, 1))), dtype=
float).reshape(-1)[:2]
    speed = max(float(np.dot(velocity_ne, route)), 0.0)
    target_speed = float(self.route_tracking_speed_m_s)
    acceleration = max(float(self.apf_own_acceleration_m_s2), 0.0)
    if speed >= target_speed or acceleration <= 1e-9:
        distance = target_speed * times_s
    else:
        ramp_s = (target_speed - speed) / acceleration
        ramp_distance = speed * ramp_s + 0.5 * acceleration * ramp_s * ramp_s
        distance = np.where(
            times_s <= ramp_s,
            speed * times_s + 0.5 * acceleration * times_s * times_s,
            ramp_distance + target_speed * (times_s - ramp_s),
        )
    along = np.clip(along_now + distance, 0.0, self.route_path_length_m)
    return self.start_ne + along[:, None] * route

def obstacle_track_state_at(self, track, dt_s):
    """Predict the 4-D [north, east, v_north, v_east] EKF state."""
    state = np.asarray(track.get("state", [np.nan] * 4), dtype=float).reshape(4)
    position_ne = state[:2]
    velocity_ne = state[2:4]
    if not np.isfinite(position_ne).all() or not np.isfinite(velocity_ne).all():
        return None, None
    return position_ne + velocity_ne * max(float(dt_s), 0.0), velocity_ne.copy()

def update_apf_virtual_obstacles(self):
    """Build the 2×TCPA field and overlapping bridge from each live EKF track.
    """
    self.apf_virtual_obstacles = []
    if not self.obstacle_ekf_prediction_enabled:
        return self.apf_virtual_obstacles

    own_start_ne = self._current_ne()
    measured_velocity_ne = np.asarray(self.v_robot[0:2], dtype=float).reshape(2)
    own_speed_m_s = float(np.linalg.norm(measured_velocity_ne))
    if not np.isfinite(own_speed_m_s) or own_speed_m_s < 1e-6:

```

```

        own_speed_m_s = float(self.route_tracking_speed_m_s)
        own_velocity_ne = self.body_vector_to_earth(np.array([own_speed_m_s, 0.0]))
        horizon_s = float(self.apf_collision_horizon_s)

    for track in self.apf_obstacle_tracks:
        if not self.obstacle_track_motion_is_stable(track):
            continue
        start_ne, velocity_ne = self.obstacle_track_state_at(track, 0.0)
        if start_ne is None or velocity_ne is None:
            continue
        start_ne = np.asarray(start_ne, dtype=float).reshape(2)
        velocity_ne = np.asarray(velocity_ne, dtype=float).reshape(2)
        speed_m_s = float(np.linalg.norm(velocity_ne))
        if not np.isfinite(start_ne).all() or not np.isfinite(velocity_ne).all()
    or speed_m_s < self.apf_dynamic_speed_threshold_m_s:
        continue
        tcpa_s, dcpa_m, own_cpa_ne, obstacle_cpa_ne = straight_line_cpa(
            own_start_ne, own_velocity_ne, start_ne, velocity_ne
        )
        if not 0.0 < tcpa_s <= horizon_s:
            continue

        pc1_m, pc2_m = self.obstacle_pc_dimensions(track)
        axis_ne = velocity_ne / speed_m_s
        final_ne = start_ne + velocity_ne * (2.0 * tcpa_s)
        field_half_along_m = self.apf_predicted_field_size_scale * (
            0.5 * pc1_m + self.apf_own_equivalent_radius_m
        )
        field_half_lateral_m = self.apf_predicted_field_size_scale * (
            0.5 * pc2_m + self.apf_own_equivalent_radius_m
        )
        base_radius_m = min(pc1_m, pc2_m) * 0.5 + self.
apf_own_equivalent_radius_m
        max_gap_m = max(
            (self.apf_current_field_size_scale + self.
apf_predicted_field_size_scale) * base_radius_m,
            1e-6,
        )
        bridge_count = max(int(np.ceil(np.linalg.norm(final_ne - start_ne) /
max_gap_m)) - 1, 0)

    def field(position_ne, bridge=False):
        return {
            "label": -1000 - int(track.get("id", 0)), "virtual": True,
            "bridge": bridge, "track_id": int(track.get("id", 0)),
            "centre_ne": np.asarray(position_ne, dtype=float).tolist(),
            "centre_body": self.earth_point_to_body(position_ne).tolist(),
            "pc1_m": pc1_m, "pc2_m": pc2_m,
            "field_half_along_m": field_half_along_m,
            "field_half_lateral_m": field_half_lateral_m,
            "length_axis_ne": axis_ne.tolist(), "apf_ellipse_axis_ne":
axis_ne.tolist(),
            "apf_ellipse_pc1_m": pc1_m, "apf_ellipse_pc2_m": pc2_m,
            "heading_rad": float(np.arctan2(velocity_ne[1], velocity_ne[0]))
        },
        "velocity_ne": velocity_ne.tolist(), "tcpa_s": tcpa_s, "dcpa_m":
dcpa_m,
        "virtual_time_s": 2.0 * tcpa_s,

```

```

        "collision_position_ne": own_cpa_ne.tolist(),
        "obstacle_dcpa_position_ne": obstacle_cpa_ne.tolist(),
        "obstacle_min_separation_point_ne": obstacle_cpa_ne.tolist(),
        "own_prediction_velocity_ne": own_velocity_ne.tolist(),
        "own_prediction_heading_rad": float(self.Yaw),
        "predicted_risk_active": True,
    }

    # Spacing is at most the combined transverse support radii, so
    # waypoint projection sees one continuous exclusion corridor.
    for index in range(1, bridge_count + 1):
        self.apf_virtual_obstacles.append(field(
            start_ne + (final_ne - start_ne) * index / (bridge_count + 1),
bridge=True
        ))
        self.apf_virtual_obstacles.append(field(final_ne))
    return self.apf_virtual_obstacles

def _route_normal_left_ne(self):
    unit = np.asarray(self.route_path_unit_ne, dtype=float).reshape(2)
    norm = float(np.linalg.norm(unit))
    if not np.isfinite(unit).all() or norm < 1e-9:
        return np.array([0.0, 1.0], dtype=float)
    unit = unit / norm
    # Coordinates are [north, east]; positive planner side is port/left.
    return np.array([unit[1], -unit[0]], dtype=float)

def _point_on_main_route(self, along_m):
    along_m = float(np.clip(along_m, 0.0, self.route_path_length_m))
    return self.start_ne + along_m * self.route_path_unit_ne

def _project_to_main_route(self, point_ne):
    point_ne = np.asarray(point_ne, dtype=float).reshape(2)
    if self.route_path_length_m < 1e-9:
        return 0.0, 0.0, self.goal_ne.copy()
    delta_ne = point_ne - self.start_ne
    along_m = float(np.dot(delta_ne, self.route_path_unit_ne))
    along_m = float(np.clip(along_m, 0.0, self.route_path_length_m))
    closest_ne = self._point_on_main_route(along_m)
    lateral_m = float(np.dot(point_ne - closest_ne, self._route_normal_left_ne()))
    return along_m, lateral_m, closest_ne

def _apf_waypoint_path_active(self):
    return self.apf_waypoint_index < len(self.apf_waypoint_path_ne)

def _rejoined_predicted_trajectory(self):
    merge_along_m = getattr(self, "apf_waypoint_merge_along_m", None)
    if not self._apf_waypoint_path_active() or merge_along_m is None:
        return False
    along_m, lateral_m, _ = self._project_to_main_route(self._current_ne())
    return (
        along_m >= merge_along_m - self.apf_waypoint_acceptance_m
        and abs(lateral_m) <= self.apf_waypoint_acceptance_m
    )

def _restore_display_waypoints(self):
    if getattr(self, "_mission_waypoints", None) is not None:

```

```

        self.waypoints = list(self._mission_waypoints)

def _update_display_waypoints(self):
    if not self._apf_waypoint_path_active():
        self._restore_display_waypoints()
        return
    if not getattr(self, "_mission_waypoints", None):
        return

    waypoint_type = type(self._mission_waypoints[0])
    detour_waypoints = []
    for point_ne in self.apf_waypoint_path_ne[self.apf_waypoint_index:]:
        waypoint = waypoint_type()
        waypoint.y = float(point_ne[0])
        waypoint.x = float(point_ne[1])
        detour_waypoints.append(waypoint)
    self.waypoints = detour_waypoints + list(self._mission_waypoints)

def _clear_apf_waypoint_path(self):
    self.apf_waypoint_path_ne = []
    self.apf_waypoint_index = 0
    self.apf_waypoint_side_sign = 0.0
    self.apf_waypoint_merge_along_m = 0.0
    self.apf_waypoint_planned_at_s = -np.inf
    self.apf_waypoint_speed_m_s = float(self.route_tracking_speed_m_s)
    self._restore_display_waypoints()

def _advance_apf_waypoint_progress(self, current_ne=None):
    if current_ne is None:
        current_ne = self._current_ne()
    current_ne = np.asarray(current_ne, dtype=float).reshape(2)

    while self._apf_waypoint_path_active():
        waypoint_ne = np.asarray(
            self.apf_waypoint_path_ne[self.apf_waypoint_index],
            dtype=float,
        ).reshape(2)
        acceptance_m = (
            self.goal_tolerance_m
            if self.apf_waypoint_index == len(self.apf_waypoint_path_ne) - 1
            and np.allclose(waypoint_ne, self.goal_ne)
            else self.apf_waypoint_acceptance_m
        )
        if float(np.linalg.norm(waypoint_ne - current_ne)) > acceptance_m:
            break
        self.apf_waypoint_index += 1

    if not self._apf_waypoint_path_active():
        self._clear_apf_waypoint_path()
    else:
        self._update_display_waypoints()

def _detour_side_sign(self):
    if self.apf_side_lock_sign != 0.0:
        return float(np.sign(self.apf_side_lock_sign))

    repulsive_force = np.asarray(self.apf_repulsive_force_body, dtype=float).
    reshape(2)

```

```

        if np.isfinite(repulsive_force).all() and abs(float(repulsive_force[1])) > 1
e-6:
            return float(np.sign(repulsive_force[1]))

        if self.left_clearance_m > self.right_clearance_m + 0.05:
            return 1.0
        if self.right_clearance_m > self.left_clearance_m + 0.05:
            return -1.0
        return -1.0

    def _waypoint_detour_side(self):
        """Keep an active detour on its original side of the planned route."""
        if self._apf_waypoint_path_active() and self.apf_waypoint_side_sign != 0.0:
            return float(np.sign(self.apf_waypoint_side_sign))
        return self._detour_side_sign()

    def _detour_candidate_obstacles(self):
        current_ne = self._current_ne()
        current_along_m, _, _ = self._project_to_main_route(current_ne)
        route_normal_left_ne = self._route_normal_left_ne()
        candidates = []

        try:
            virtual_obstacles = list(self.update_apf_virtual_obstacles() or [])
        except Exception:
            virtual_obstacles = []

        for obstacle in list(getattr(self, "lidar_obstacles", []) or []) +
virtual_obstacles:
            centre_ne = np.asarray(
                obstacle.get("centre_ne", [np.nan, np.nan]),
                dtype=float,
            ).reshape(2)
            if not np.isfinite(centre_ne).all():
                continue

            centre_body = np.asarray(
                obstacle.get("centre_body", [np.nan, np.nan]),
                dtype=float,
            ).reshape(2)
            if np.isfinite(centre_body).all() and centre_body[0] < -0.25:
                continue

            along_m, _, closest_ne = self._project_to_main_route(centre_ne)
            if along_m < current_along_m - self.route_tracking_lookahead_m:
                continue

            obstacle_to_route_ne = centre_ne - closest_ne
            lateral_m = float(np.dot(obstacle_to_route_ne, route_normal_left_ne))
            pc1_m = max(
                float(obstacle.get("pc1_m", self.obstacle_min_pc1_m)),
                float(self.obstacle_min_pc1_m),
            )
            pc2_m = max(
                float(obstacle.get("pc2_m", self.obstacle_min_pc2_m)),
                float(self.obstacle_min_pc2_m),
            )
            candidates.append(

```

```

        {
            "along_m": along_m,
            "lateral_m": lateral_m,
            "pc1_m": pc1_m,
            "pc2_m": pc2_m,
            "virtual": bool(obstacle.get("virtual", False)),
            "centre_ne": centre_ne,
            "axis_ne": self.obstacle_length_axis_ne(obstacle),
            "velocity_ne": np.asarray(
                obstacle.get("velocity_mean_ne", obstacle.get("velocity_ne",
                    [np.nan, np.nan])),
                dtype=float,
            ).reshape(2),
        }
    )

    for candidate in candidates:
        field_scale = float(getattr(
            self,
            "apf_predicted_field_size_scale" if candidate["virtual"] else "
apf_current_field_size_scale",
            8.0 if candidate["virtual"] else 3.0,
        ))
        own_radius_m = float(getattr(self, "apf_own_equivalent_radius_m", 0.0))
        candidate["field_long_m"] = candidate["field_half_along_m"] =
field_scale * (
            0.5 * candidate["pc1_m"] + own_radius_m
        )
        candidate["field_lateral_m"] = candidate["field_half_lateral_m"] =
field_scale * (
            0.5 * candidate["pc2_m"] + own_radius_m
        )

    return candidates

def _main_route_has_collision_risk(self):
    """Risk exists only while an APF field intersects the remaining main route.
    """
    if self.route_path_length_m < 1e-9:
        return False
    current_along_m, _, _ = self._project_to_main_route(self._current_ne())
    route_start_ne = self._point_on_main_route(current_along_m)
    for obstacle in self._detour_candidate_obstacles():
        if segment_intersects_ellipse(
            route_start_ne,
            self.goal_ne,
            obstacle["centre_ne"],
            obstacle["axis_ne"],
            obstacle["field_long_m"],
            obstacle["field_lateral_m"],
            self.apf_waypoint_lateral_margin_m,
        ):
            return True
    return False

def _keep_waypoints_outside_apf_fields(self, points_ne, candidates, side_sign):
    """Project waypoint samples outside every measured and predicted APF ellipse.
    """

```

```

margin_m = float(self.apf_waypoint_lateral_margin_m)
route_normal_ne = self._route_normal_left_ne()
safe_points = []
for point_ne in points_ne:
    point_ne = np.asarray(point_ne, dtype=float).reshape(2)
    for obstacle in candidates:
        axis_ne = np.asarray(obstacle["axis_ne"], dtype=float).reshape(2)
        axis_norm = float(np.linalg.norm(axis_ne))
        if not np.isfinite(axis_ne).all() or axis_norm < 1e-9:
            axis_ne = self.route_path_unit_ne
        else:
            axis_ne = axis_ne / axis_norm
            lateral_ne = np.array([-axis_ne[1], axis_ne[0]])
            long_m = float(obstacle.get("field_long_m", obstacle.get("
field_half_along_m", 0.0)))
            lateral_m = float(obstacle.get("field_lateral_m", obstacle.get("
field_half_lateral_m", 0.0)))
            if long_m <= 0.0 or lateral_m <= 0.0:
                continue
            offset = point_ne - np.asarray(obstacle["centre_ne"], dtype=float).
reshape(2)
            scaled = np.array([np.dot(offset, axis_ne) / long_m, np.dot(offset,
lateral_ne) / lateral_m])
            level = float(np.linalg.norm(scaled))
            if level >= 1.0 + margin_m / min(long_m, lateral_m):
                continue
            direction = offset if level > 1e-9 else float(side_sign) *
route_normal_ne
            direction_norm = float(np.linalg.norm(direction))
            if direction_norm < 1e-9:
                direction = lateral_ne
                direction_norm = 1.0
            # ponytail: radial projection is conservative for rotated ellipses;
use a full local planner only for dense obstacle fields.
            direction = direction / direction_norm
            boundary_distance_m = 1.0 / np.hypot(
                np.dot(direction, axis_ne) / long_m,
                np.dot(direction, lateral_ne) / lateral_m,
            )
            point_ne = np.asarray(obstacle["centre_ne"], dtype=float).reshape(2)
            + direction * (
                boundary_distance_m + margin_m
            )
            safe_points.append(point_ne)
    return safe_points

def _keep_waypoints_on_detour_side(self, points_ne, side_sign):
    kept_points = []
    for point_ne in points_ne:
        along_m, lateral_m, route_point_ne = self._project_to_main_route(
point_ne)
        del along_m
        kept_points.append(
            np.asarray(point_ne, dtype=float).reshape(2)
            if side_sign * lateral_m >= 0.0 else route_point_ne
        )
    return kept_points

```

```

def _activate_apf_waypoint_path(self, path_ne, side_sign, merge_along_m,
current_ne=None):
    if current_ne is None:
        current_ne = self._current_ne()

    filtered_path = []
    previous_point = np.asarray(current_ne, dtype=float).reshape(2)
    previous_along_m, _, _ = self._project_to_main_route(previous_point)
    for point_ne in path_ne:
        point_ne = np.asarray(point_ne, dtype=float).reshape(2)
        if not np.isfinite(point_ne).all():
            continue
        point_along_m, _, _ = self._project_to_main_route(point_ne)
        if point_along_m <= previous_along_m + 1e-6:
            continue
        if float(np.linalg.norm(point_ne - previous_point)) < 0.25:
            continue
        filtered_path.append(point_ne)
        previous_point = point_ne
        previous_along_m = point_along_m

    if not filtered_path:
        return False

    self.apf_waypoint_path_ne = filtered_path
    self.apf_waypoint_index = 0
    self.apf_waypoint_side_sign = float(side_sign)
    self.apf_waypoint_merge_along_m = float(merge_along_m)
    self.apf_waypoint_planned_at_s = (
        float(self.timefromstart) if self.timefromstart is not None else 0.0
    )
    self._advance_apf_waypoint_progress(current_ne)
    self._update_display_waypoints()
    return self._apf_waypoint_path_active()

def _force_guidance_direction_ne(self):
    guidance_body = np.asarray(self.apf_force_body, dtype=float).reshape(2)
    if not np.isfinite(guidance_body).all() or float(np.linalg.norm(
guidance_body)) < 1e-6:
        guidance_body = np.asarray(self.apf_repulsive_force_body, dtype=float).
reshape(2)
    if not np.isfinite(guidance_body).all() or float(np.linalg.norm(
guidance_body)) < 1e-6:
        return None

    guidance_ne = np.asarray(self.body_vector_to_earth(guidance_body), dtype=
float).reshape(2)
    guidance_norm = float(np.linalg.norm(guidance_ne))
    if not np.isfinite(guidance_ne).all() or guidance_norm < 1e-6:
        return None
    return guidance_ne / guidance_norm

def _plan_apf_waypoint_path(self):
    if not self.apf_waypoint_detour_enabled or self.route_path_length_m < 1e-9:
        return False

    if not self._main_route_has_collision_risk():
        self._clear_apf_waypoint_path()

```



```

        return False

    current_ne = self._current_ne()
    current_along_m, _, _ = self._project_to_main_route(current_ne)
    side_sign = self._waypoint_detour_side()
    if side_sign == 0.0:
        return False

    candidates = self._detour_candidate_obstacles()
    repulsive_force = np.asarray(self.apf_repulsive_force_body, dtype=float).
    reshape(2)
    repulsive_norm = (
        float(np.linalg.norm(repulsive_force))
        if np.isfinite(repulsive_force).all()
        else 0.0
    )
    if not candidates and repulsive_norm < 1e-6:
        return False

    lateral_offset_m = max(
        float(getattr(self, "apf_min_detour_offset_m", 0.0)),
        float(self.apf_waypoint_lateral_margin_m + self.
    apf_own_equivalent_radius_m),
    )
    lateral_offset_m = max(
        lateral_offset_m,
        float(getattr(self, "obstacle_min_pc2_m", 0.0)) + self.
    apf_waypoint_lateral_margin_m,
        lateral_offset_m + 0.35 * min(repulsive_norm, 2.0),
    )
    furthest_along_m = current_along_m + self.apf_waypoint_entry_margin_m

    for obstacle in candidates:
        half_length_m = 0.5 * obstacle["pc1_m"] + self.
    apf_waypoint_merge_margin_m
        half_width_m = (
            0.5 * obstacle["pc2_m"]
            + self.apf_own_equivalent_radius_m
            + self.apf_waypoint_lateral_margin_m
        )
        furthest_along_m = max(furthest_along_m, obstacle["along_m"] +
    half_length_m)
        lateral_offset_m = max(
            lateral_offset_m,
            side_sign * obstacle["lateral_m"] + half_width_m,
        )

    lateral_offset_m = max(lateral_offset_m, self.apf_waypoint_acceptance_m +
    0.05)
    entry_along_m = float(
        np.clip(
            min(
                current_along_m + self.apf_waypoint_entry_margin_m,
                furthest_along_m,
            ),
            0.0,
            self.route_path_length_m,
        )
    )

```

```

    )
    offset_along_m = float(
        np.clip(
            max(entry_along_m + self.route_tracking_lookahead_m,
furthest_along_m),
            0.0,
            self.route_path_length_m,
        )
    )
    merge_along_m = float(
        np.clip(
            offset_along_m + self.apf_waypoint_merge_margin_m,
            0.0,
            self.route_path_length_m,
        )
    )

    route_normal_left_ne = self._route_normal_left_ne()
    force_dir_ne = self._force_guidance_direction_ne()
    if force_dir_ne is None:
        force_dir_ne = self.route_path_unit_ne.copy()

    blend_dir_ne = self.route_path_unit_ne + self.apf_waypoint_force_blend *
force_dir_ne
    blend_norm = float(np.linalg.norm(blend_dir_ne))
    if not np.isfinite(blend_dir_ne).all() or blend_norm < 1e-6:
        blend_dir_ne = self.route_path_unit_ne.copy()
    else:
        blend_dir_ne = blend_dir_ne / blend_norm

    preview_points = max(int(getattr(self, "apf_waypoint_preview_points", 3)),
2)
    waypoint_step_m = max(
        float(getattr(self, "apf_waypoint_step_m", self.
route_tracking_lookahead_m)),
        self.apf_waypoint_acceptance_m + 0.05,
    )
    first_guided_point = (
        current_ne
        + waypoint_step_m * blend_dir_ne
        + side_sign * min(lateral_offset_m, waypoint_step_m) * 0.55 *
route_normal_left_ne
    )
    candidate_points = [first_guided_point]
    for index in range(preview_points):
        alpha = float(index + 1) / float(preview_points)
        along_m = entry_along_m + alpha * max(offset_along_m - entry_along_m,
0.0)

        route_point_ne = self._point_on_main_route(along_m)
        force_bias_m = waypoint_step_m * (0.35 + 0.45 * alpha)
        candidate_points.append(
            route_point_ne
            + side_sign * lateral_offset_m * route_normal_left_ne
            + force_bias_m * blend_dir_ne
        )
    if self.apf_encounter_mode in self._CROSSING_RULES and candidates:
        stern_candidates = [obstacle for obstacle in candidates if obstacle["
virtual"]] or candidates

```

```

        obstacle = min(stern_candidates, key=lambda item: item["along_m"])
        stern_ne = obstacle_stern_waypoint(
            obstacle["centre_ne"],
            obstacle["velocity_ne"],
            obstacle["field_long_m"] + self.apf_waypoint_lateral_margin_m,
        )
        if stern_ne is not None:
            candidate_points.append(stern_ne)
            candidate_points.sort(
                key=lambda point: float(np.dot(point - self.start_ne, self.
route_path_unit_ne))
            )
            candidate_points.append(self._point_on_main_route(merge_along_m))
            resume_along_m = min(
                merge_along_m + waypoint_step_m,
                self.route_path_length_m - self.apf_waypoint_acceptance_m,
            )
            if resume_along_m > merge_along_m + 1e-6:
                candidate_points.append(self._point_on_main_route(resume_along_m))
                candidate_points = self._keep_waypoints_on_detour_side(candidate_points,
side_sign)
                candidate_points = self._keep_waypoints_outside_apf_fields(
                    candidate_points, candidates, side_sign,
                )
                candidate_points = [
                    point for point in candidate_points
                    if self._project_to_main_route(point)[0] < self.route_path_length_m - 1e
-6
                ]
                candidate_points.append(self.goal_ne.copy())

        return self._activate_apf_waypoint_path(
            candidate_points,
            side_sign,
            merge_along_m,
            current_ne=current_ne,
        )

def _ensure_apf_waypoint_path(self):
    if not self._main_route_has_collision_risk():
        self._clear_apf_waypoint_path()
        return False
    now_s = float(self.timefromstart) if self.timefromstart is not None else 0.0
    if self._apf_waypoint_path_active():
        self._advance_apf_waypoint_progress()
        if self._apf_waypoint_path_active():
            target_ne = np.asarray(
                self.apf_waypoint_path_ne[self.apf_waypoint_index],
                dtype=float,
            ).reshape(2)
            target_blocked = any(
                ellipse_level_and_away(
                    target_ne - obstacle["centre_ne"],
                    obstacle["axis_ne"],
                    obstacle["field_long_m"],
                    obstacle["field_lateral_m"],
                )[0] < 1.0
            )
            for obstacle in self._detour_candidate_obstacles()

```

```

        )
        if target_blocked:
            self._clear_apf_waypoint_path()
        elif now_s - self.apf_waypoint_planned_at_s < self.
apf_waypoint_replan_interval_s:
            return True
        return self._plan_apf_waypoint_path() or self._apf_waypoint_path_active()

def _compute_waypoint_tracking_control(self, navigation_mode):
    current_ne = self._current_ne()
    self._advance_apf_waypoint_progress(current_ne)
    if not self._apf_waypoint_path_active():
        return None

    target_ne = np.asarray(
        self.apf_waypoint_path_ne[self.apf_waypoint_index],
        dtype=float,
    ).reshape(2)
    to_target_ne = target_ne - current_ne
    target_distance_m = float(np.linalg.norm(to_target_ne))
    if target_distance_m < 1e-9:
        return None

    desired_heading = float(np.arctan2(to_target_ne[1], to_target_ne[0]))
    heading_error = _overtaking.wrap_angle(float(self.Yaw) - desired_heading)
    speed_fraction = float(
        np.clip(
            target_distance_m / max(self.final_slowdown_distance_m, 1e-3),
            0.35,
            1.0,
        )
    )

    if abs(heading_error) >= self.final_heading_slow_angle_rad:
        heading_speed_scale = 0.0
    else:
        heading_speed_scale = max(float(np.cos(heading_error)), 0.15)

    target_speed_m_s = float(getattr(self, "apf_waypoint_speed_m_s", self.
route_tracking_speed_m_s))
    if not np.isfinite(target_speed_m_s) or target_speed_m_s <= 0.0:
        target_speed_m_s = float(self.route_tracking_speed_m_s)

    p_ref = _overtaking.Vector(3)
    p_ref[0, 0] = target_ne[0]
    p_ref[1, 0] = target_ne[1]
    p_ref[2, 0] = desired_heading

    u_ref = _overtaking.Vector(2)
    u_ref[0, 0] = target_speed_m_s * speed_fraction

    u_track = _overtaking.Vector(2)
    if "overtaking" in str(getattr(self, "webots_environment", "")).lower():
        u_track[0, 0] = self.route_tracking_speed_m_s
    else:
        u_track[0, 0] = max(
            self.apf_min_forward_speed,
            target_speed_m_s * speed_fraction * heading_speed_scale,
        )

```

```

        u_track[1, 0] = 1.4 * heading_error
        self.apf_target_ne = target_ne.copy()
        self.navigation_mode = navigation_mode
        return p_ref, u_ref, u_track

    def limit_heading_deviation_command(self, yaw_rate_cmd):
        if self._apf_waypoint_path_active():
            return float(yaw_rate_cmd)
        return _OvertakingController.limit_heading_deviation_command(self,
                                                                        yaw_rate_cmd)

    def update_apf_obstacle_tracks(self, stamp_s):
        _CrossingController.update_apf_obstacle_tracks(self, stamp_s)

    def _run_context(self):
        return {
            "webots_environment": self.webots_environment,
            "switch_combination": SWITCH_COMBINATION,
            "ekf_prediction_enabled": bool(self.obstacle_ekf_prediction_enabled),
            "cluster_size_apf_enabled": bool(self.apf_cluster_range_enabled),
        }

    def _csv_context_values(self):
        context = self._run_context()
        return [
            context["webots_environment"],
            context["switch_combination"],
            int(context["ekf_prediction_enabled"]),
            int(context["cluster_size_apf_enabled"]),
        ]

    def _ensure_csv_context_header(self):
        try:
            lines = self.filename.read_text().splitlines()
            if not lines:
                return
            header = lines[0].split(",")
            if all(column in header for column in _CSV_CONTEXT_COLUMNS):
                return
            lines[0] = lines[0] + "," + ",".join(_CSV_CONTEXT_COLUMNS)
            self.filename.write_text("\n".join(lines) + "\n")
        except Exception:
            return

    def _patch_latest_csv_context_row(self):
        try:
            values = ",".join(str(value) for value in self._csv_context_values())
            with self.filename.open("rb+") as f:
                f.seek(0, os.SEEK_END)
                end = f.tell()
                if end <= 0:
                    return

                pos = end - 1
                while pos >= 0:
                    f.seek(pos)
                    if f.read(1) not in (b"\n", b"\r"):
                        break

```

```

        pos -= 1
        if pos < 0:
            return

        line_end = pos + 1
        if getattr(self, "_csv_context_last_patched_line_end", None) ==
line_end:
            return

        f.seek(line_end)
        f.truncate()
        f.write(("," + values + "\n").encode("utf-8"))
        self._csv_context_last_patched_line_end = f.tell() - 1
    except Exception:
        return

@contextmanager
def _crossing_strategy_methods(self):
    """Temporarily use crossing.py APF helpers on this same controller."""
    previous = {}
    for name in _CROSSING_METHODS:
        method = getattr(_CrossingController, name, None)
        if method is None:
            continue
        previous[name] = self.__dict__.get(name, _MISSING)
        setattr(self, name, MethodType(method, self))

    try:
        yield
    finally:
        for name, value in previous.items():
            if value is _MISSING:
                self.__dict__.pop(name, None)
            else:
                setattr(self, name, value)

@contextmanager
def _head_on_strategy_methods(self):
    """Temporarily use headon.py APF helpers on this same controller."""
    previous = {}
    for name in _HEAD_ON_METHODS:
        method = getattr(_HeadOnController, name, None)
        if method is None:
            continue
        previous[name] = self.__dict__.get(name, _MISSING)
        setattr(self, name, MethodType(method, self))

    try:
        yield
    finally:
        for name, value in previous.items():
            if value is _MISSING:
                self.__dict__.pop(name, None)
            else:
                setattr(self, name, value)

def _obstacle_body_position(self, obstacle):
    centre_body = np.asarray(

```

```

        obstacle.get("centre_body", [np.nan, np.nan]),
        dtype=float,
    ).reshape(2)
    if np.isfinite(centre_body).all():
        return centre_body

    centre_ne = np.asarray(obstacle.get("centre_ne", [np.nan, np.nan]), dtype=
float).reshape(2)
    if np.isfinite(centre_ne).all():
        return self.earth_point_to_body(centre_ne)

    return centre_body

def _obstacle_body_velocity(self, obstacle):
    velocity_ne = np.asarray(
        obstacle.get("velocity_ne", [np.nan, np.nan]),
        dtype=float,
    ).reshape(2)
    if np.isfinite(velocity_ne).all():
        return self.earth_vector_to_body(velocity_ne)

    track = None if bool(obstacle.get("virtual", False)) else self.
apf_track_for_obstacle(obstacle)
    if track is None or not self.obstacle_track_motion_is_stable(track):
        return np.zeros(2, dtype=float)

    velocity_ne = np.asarray(
        track.get("velocity_mean_ne", track.get("vel_ne", [np.nan, np.nan])),
        dtype=float,
    ).reshape(2)
    if not np.isfinite(velocity_ne).all():
        velocity_ne = np.asarray(track.get("vel_ne", [np.nan, np.nan]), dtype=
float).reshape(2)
    if np.isfinite(velocity_ne).all():
        return self.earth_vector_to_body(velocity_ne)

    return np.zeros(2, dtype=float)

def apf_track_for_obstacle(self, obstacle):
    centre_ne = np.asarray(obstacle.get("centre_ne", [np.nan, np.nan]), dtype=
float).reshape(2)
    if not np.isfinite(centre_ne).all():
        return None

    now = float(self.latest_lidar_received_s if self.latest_lidar_received_s is
not None else 0.0)
    best_track = None
    best_distance = np.inf
    for track in self.apf_obstacle_tracks:
        if now - float(track.get("last_seen_s", track.get("stamp_s", now))) >
self.apf_track_timeout_s:
            continue
        pos_ne = np.asarray(track.get("pos_ne", [np.nan, np.nan]), dtype=float).
reshape(2)
        if not np.isfinite(pos_ne).all():
            continue
        distance = float(np.linalg.norm(centre_ne - pos_ne))
        if distance < best_distance:

```

```

        best_distance = distance
        best_track = track

    if best_distance <= max(self.apf_track_association_m * 1.5, self.
lidar_dbscan_eps_m * 2.0):
        return best_track
    return None

def apf_cpa_metrics(self, obs_pos_body, obs_vel_body, own_vel_body):
    tcpa_s, dcpa_m, _, _ = straight_line_cpa(
        [0.0, 0.0], own_vel_body, obs_pos_body, obs_vel_body
    )
    return tcpa_s, dcpa_m

def _rule_priority(self, rule):
    if rule == "head_on":
        return 0
    if rule in self._OVERTAKING_RULES:
        return 1
    if rule in self._CROSSING_RULES:
        return 2
    if rule == "static_obstacle":
        return 3
    return 4

def apf_ekf_colreg_prediction(self, obs_pos_body, obs_vel_body):
    """Return the short-horizon EKF motion estimate for COLREG, if trusted."""
    obs_pos_body = np.asarray(obs_pos_body, dtype=float).reshape(2)
    obs_vel_body = np.asarray(obs_vel_body, dtype=float).reshape(2)
    if (
        not bool(getattr(self, "obstacle_ekf_prediction_enabled", False))
        or not np.isfinite(obs_pos_body).all()
    ):
        return obs_pos_body, obs_vel_body, False

    try:
        obstacle_ne = self._current_ne() + self.body_vector_to_earth(
obs_pos_body)
        track = self.apf_track_for_obstacle({"centre_ne": obstacle_ne})
        if track is None or not self.obstacle_track_motion_is_stable(track):
            return obs_pos_body, obs_vel_body, False

        heading_var = float(track.get("heading_var_rad2", np.inf))
        max_heading_var = float(getattr(
            self, "apf_colreg_max_heading_std_rad", np.deg2rad(30.0)
        )) ** 2
        if not np.isfinite(heading_var) or heading_var > max_heading_var:
            return obs_pos_body, obs_vel_body, False

        state = np.asarray(track.get("state", [np.nan] * 4), dtype=float).
reshape(4)
        if not np.isfinite(state).all():
            return obs_pos_body, obs_vel_body, False
        horizon_s = max(float(getattr(self, "apf_colreg_prediction_horizon_s",
1.0)), 0.0)
        accel_ne = self.obstacle_track_prediction_accel_ne(track)
        predicted_ne = state[:2] + state[2:4] * horizon_s + 0.5 * accel_ne *
horizon_s ** 2

```



```

        predicted_velocity_ne = state[2:4] + accel_ne * horizon_s
    if (
        not np.isfinite(predicted_ne).all()
        or not np.isfinite(predicted_velocity_ne).all()
        or np.linalg.norm(predicted_velocity_ne) < self.
apf_dynamic_speed_threshold_m_s
    ):
        return obs_pos_body, obs_vel_body, False
    return (
        self.earth_point_to_body(predicted_ne),
        self.earth_vector_to_body(predicted_velocity_ne),
        True,
    )
except (AttributeError, TypeError, ValueError):
    return obs_pos_body, obs_vel_body, False

def apf_classify_encounter(self, obs_pos_body, obs_vel_body, own_vel_body):
    """Four-zone COLREG classifier from Li et al. Fig. 3."""
    obs_pos_body = np.asarray(obs_pos_body, dtype=float).reshape(2)
    obs_vel_body = np.asarray(obs_vel_body, dtype=float).reshape(2)
    own_vel_body = np.asarray(own_vel_body, dtype=float).reshape(2)
    obs_pos_body, obs_vel_body, has_ekf_direction = self.
apf_ekf_colreg_prediction(
    obs_pos_body, obs_vel_body
)
    obs_speed = float(np.linalg.norm(obs_vel_body))
    if obs_speed < self.apf_dynamic_speed_threshold_m_s:
        return "static_obstacle", 0.0, "none"

    # Body y is port/left, while maritime bearings/headings increase to
    # starboard, so both angles use the same clockwise conversion.
    bearing_deg = (-np.rad2deg(np.arctan2(obs_pos_body[1], obs_pos_body[0]))) %
360.0
    relative_heading_deg = (-np.rad2deg(np.arctan2(obs_vel_body[1], obs_vel_body
[0]))) % 360.0
    own_bearing_from_obstacle_deg = (
        np.rad2deg(np.arctan2(-obs_pos_body[1], -obs_pos_body[0]) - np.arctan2(
obs_vel_body[1], obs_vel_body[0]))
    ) % 360.0
    own_speed_m_s = float(np.linalg.norm(own_vel_body))
    tcpa_s, dcpa_m = self.apf_cpa_metrics(obs_pos_body, obs_vel_body,
own_vel_body)
    emergency = (
        np.isfinite(tcpa_s)
        and np.isfinite(dcpa_m)
        and 0.0 < tcpa_s <= self.apf_stand_on_emergency_tcpa_s
        and dcpa_m <= 2.0 * self.apf_own_equivalent_radius_m
    )
    encounter = classify_colreg_zone(
        bearing_deg,
        relative_heading_deg,
        emergency,
        own_speed_m_s=own_speed_m_s,
        obstacle_speed_m_s=obs_speed,
        own_bearing_from_obstacle_deg=own_bearing_from_obstacle_deg,
    )
    # Rule 13 remains EKF-confirmed because it is easily confused with a
    # crossing. Rule 14 is allowed to use the measured direction until

```

```

        # the EKF becomes stable, so approaching traffic is caught early.
        if encounter[0] == "overtaking" and not has_ekf_direction:
            return "static_obstacle", 0.0, "EKF direction not yet stable for
overtaking"
        return encounter

def _select_colreg_strategy_impl(self):
    """IP_test5 COLREG rule detection, kept intact under the behaviour tree
shell."""
    own_vel_body = self.current_velocity_body()
    try:
        virtual_obstacles = self.update_apf_virtual_obstacles()
    except Exception:
        virtual_obstacles = []

    best = {
        "rule": "none",
        "controller": "default_apf",
        "tcpa_s": np.nan,
        "dcpa_m": np.nan,
        "score": (self._rule_priority("none"), np.inf, np.inf),
    }
    forward_obstacle_present = False

    for obstacle in list(getattr(self, "lidar_obstacles", []) or []) + list(
virtual_obstacles or []):
        try:
            obs_pos_body = self._obstacle_body_position(obstacle)
            if not np.isfinite(obs_pos_body).all():
                continue
            forward_obstacle_present = forward_obstacle_present or obs_pos_body
[0] > -0.25

            angle_rad = abs(_overtaking.wrap_angle(float(np.arctan2(obs_pos_body
[1], obs_pos_body[0]))))
            if (
                not bool(obstacle.get("virtual", False))
                and angle_rad > self.apf_activation_front_half_angle_rad
            ):
                continue

            obs_vel_body = self._obstacle_body_velocity(obstacle)
            if bool(obstacle.get("virtual", False)):
                rule = str(obstacle.get("encounter_mode", "crossing"))
                if rule in {"dynamic_virtual_obstacle", "predicted_collision", "
none", ""}:
                    rule, _, _ = self.apf_classify_encounter(obs_pos_body,
obs_vel_body, own_vel_body)
                else:
                    rule, _, _ = self.apf_classify_encounter(obs_pos_body,
obs_vel_body, own_vel_body)

                if rule not in self._HEAD_ON_RULES | self._OVERTAKING_RULES | self.
_CROSSING_RULES:
                    rule = "static_obstacle"

                tcpa_s, dcpa_m = self.apf_cpa_metrics(obs_pos_body, obs_vel_body,
own_vel_body)

```

```

        distance_m = float(np.linalg.norm(obs_pos_body))
        tcpa_score = float(tcpa_s) if np.isfinite(tcpa_s) else np.inf
        distance_score = float(dcpa_m) if np.isfinite(dcpa_m) else
distance_m
        score = (self._rule_priority(rule), tcpa_score, distance_score)

        if score < best["score"]:
            best.update(
                {
                    "rule": rule,
                    "tcpa_s": tcpa_s,
                    "dcpa_m": dcpa_m,
                    "score": score,
                }
            )
        except Exception:
            continue

    rule = best["rule"]
    if rule in self._HEAD_ON_RULES:
        self.apf_head_on_latched = True
        self.apf_overtaking_latched = False
    elif rule in self._OVERTAKING_RULES:
        self.apf_head_on_latched = False
        self.apf_overtaking_latched = True
    elif bool(getattr(self, "apf_head_on_latched", False)) and
forward_obstacle_present:
        # A predicted TCPA field cannot relabel an active Rule 14
        # encounter as crossing while the vessel remains ahead.
        rule = "head_on"
        best["rule"] = rule
        best["score"] = (self._rule_priority(rule), best["score"][1], best["
score"] [2])
    elif bool(getattr(self, "apf_overtaking_latched", False)) and
forward_obstacle_present:
        # Rule 13 is based on the stable EKF direction at first detection;
        # retain it while virtual TCPA fields continue to be present.
        rule = "overtaking"
        best["rule"] = rule
        best["score"] = (self._rule_priority(rule), best["score"][1], best["
score"] [2])
    else:
        self.apf_head_on_latched = False
        self.apf_overtaking_latched = False
    if rule in self._HEAD_ON_RULES:
        best["controller"] = "head_on"
    elif rule in self._OVERTAKING_RULES:
        best["controller"] = "overtaking"
    elif rule in self._CROSSING_RULES:
        best["controller"] = "crossing"

    self._last_colreg_decision = best
    return rule

def _build_colreg_recognition_behavior_tree(self):
    def recognize_rule(bb):
        bb["rule"] = self._select_colreg_strategy_impl()
        return BTStatus.SUCCESS

```

```

        return BTSequence(BTAction(recognize_rule))

# COLREG rule detection location.
def select_colreg_strategy(self):
    """Pick the current primary COLREG rule without creating another controller.
    """
    blackboard = {}
    status = self._colreg_recognition_bt.tick(blackboard)
    if status != BTStatus.SUCCESS:
        return "none"
    return blackboard.get("rule", "none")

def apf_primary_encounter_mode(self):
    return self.select_colreg_strategy()

def _mark_selected_controller(self, rule):
    if rule in self._HEAD_ON_RULES:
        self.apf_selected_controller = "head_on"
        self.apf_active_profile_name = "head_on"
    elif rule in self._OVERTAKING_RULES:
        self.apf_selected_controller = "overtaking"
        self.apf_active_profile_name = "overtaking"
    elif rule in self._CROSSING_RULES:
        self.apf_selected_controller = "crossing"
        self.apf_active_profile_name = "crossing"
    else:
        self.apf_selected_controller = "default_apf"

def _build_colreg_behavior_tree(self):
    def keep_selected_rule(bb):
        bb["rule"] = bb.get("rule", "none")
        return BTStatus.SUCCESS

    def dispatch_head_on(bb):
        with self._head_on_strategy_methods():
            bb["u_cmd"] = _HeadOnController.compute_apf_control(self, bb["t"],
bb["u_track"])
        return BTStatus.SUCCESS

    def dispatch_overtaking(bb):
        bb["u_cmd"] = _OvertakingController.compute_apf_control(self, bb["t"],
bb["u_track"])
        return BTStatus.SUCCESS

    def dispatch_crossing(bb):
        with self._crossing_strategy_methods():
            bb["u_cmd"] = _CrossingController.compute_apf_control(self, bb["t"],
bb["u_track"])
        return BTStatus.SUCCESS

    def dispatch_default(bb):
        bb["u_cmd"] = _OvertakingController.compute_apf_control(self, bb["t"],
bb["u_track"])
        return BTStatus.SUCCESS

    def is_head_on(bb):
        return bb.get("rule") in self._HEAD_ON_RULES

```

```

def is_overtaking(bb):
    return bb.get("rule") in self._OVERTAKING_RULES

def is_crossing(bb):
    return bb.get("rule") in self._CROSSING_RULES

return BTSequence(
    BTAction(keep_selected_rule),
    BTSelector(
        BTSequence(BTCondition(is_head_on), BTAction(dispatch_head_on)),
        BTSequence(BTCondition(is_overtaking), BTAction(dispatch_overtaking))
    ),
    BTSequence(BTCondition(is_crossing), BTAction(dispatch_crossing)),
    BTAction(dispatch_default),
),
)

def _tick_colreg_bt(self, rule, t, u_track):
    blackboard = {"rule": rule, "t": t, "u_track": u_track}
    self._mark_selected_controller(rule)
    status = self._colreg_bt.tick(blackboard)
    if status != BTStatus.SUCCESS:
        return None
    return blackboard.get("u_cmd")

def _apf_avoidance_needed_impl(self, selected_rule):
    self._mark_selected_controller(selected_rule)
    if selected_rule in self._HEAD_ON_RULES:
        with self._head_on_strategy_methods():
            return _HeadOnController.apf_avoidance_needed(self)
    if selected_rule in self._CROSSING_RULES:
        with self._crossing_strategy_methods():
            return _CrossingController.apf_avoidance_needed(self)
    return _OvertakingController.apf_avoidance_needed(self)

def _build_colreg_avoidance_behavior_tree(self):
    def evaluate_avoidance(bb):
        bb["result"] = self._apf_avoidance_needed_impl(bb.get("rule", "none"))
        return BTStatus.SUCCESS

    return BTSequence(BTAction(evaluate_avoidance))

def _waypoint_speed_from_command(self, u_cmd):
    if "overtaking" in str(getattr(self, "webots_environment", "")).lower():
        return float(self.route_tracking_speed_m_s)
    target_speed_m_s = float(self.route_tracking_speed_m_s)
    if u_cmd is None:
        return target_speed_m_s

    try:
        values = np.asarray(u_cmd, dtype=float).reshape(-1)
    except Exception:
        return target_speed_m_s

    if values.size <= 0 or not np.isfinite(values[0]):
        return target_speed_m_s

```

```

        return float(np.clip(values[0], self.apf_min_forward_speed, self.v_max))

# crossing/overtaking/head_on dispatch location.
def compute_apf_control(self, t, u_track):
    if not self.obstacle_ekf_prediction_enabled:
        # EKF-off mode is local: no predicted-trajectory state may survive.
        self._clear_apf_waypoint_path()
        self.apf_side_lock_sign = 0.0
        self.apf_side_lock_active = False
        self.apf_colreg_active = False
        self.apf_colreg_rule = "none"
        self.apf_colreg_dcpa_m = np.nan
        self.apf_colreg_tcpa_s = np.nan

        # Use the local ellipse-field composition, which suppresses route
        # attraction in close quarters; the legacy default controller lets
        # attraction overwhelm the measured obstacle field.
        u_cmd = _CrossingController.compute_apf_control(self, t, u_track)
        self.apf_waypoint_speed_m_s = self._waypoint_speed_from_command(u_cmd)
        return u_cmd

    selected_rule = self.select_colreg_strategy()
    u_cmd = self._tick_colreg_bt(selected_rule, t, u_track)
    if u_cmd is None:
        u_cmd = _OvertakingController.compute_apf_control(self, t, u_track)
    if "overtaking" in str(getattr(self, "webots_environment", "")).lower():
        u_cmd[0, 0] = self.route_tracking_speed_m_s
        self.apf_waypoint_speed_m_s = self._waypoint_speed_from_command(u_cmd)

    self._mark_selected_controller(selected_rule)
    if self.apf_encounter_mode in ("none", "static_obstacle") and selected_rule
    != "none":
        self.apf_encounter_mode = selected_rule
    if self._last_colreg_decision is not None:
        self.apf_colreg_tcpa_s = self._last_colreg_decision.get("tcpa_s", self.
        apf_colreg_tcpa_s)
        self.apf_colreg_dcpa_m = self._last_colreg_decision.get("dcpa_m", self.
        apf_colreg_dcpa_m)
    if self._apf_waypoint_path_active() and not (self.apf_colreg_active or self.
    apf_side_lock_active):
        if "overtaking" in str(getattr(self, "webots_environment", "")).lower():
            self._ensure_apf_waypoint_path()
            tracking = self._compute_waypoint_tracking_control("apf_return")
            if tracking is not None:
                return tracking[2]
    if self._ensure_apf_waypoint_path():
        tracking = self._compute_waypoint_tracking_control(
            "apf_colreg" if self.apf_colreg_active else "apf_waypoint",
        )
        if tracking is not None:
            return tracking[2]
    self._clear_apf_waypoint_path()
    return u_cmd

def apf_avoidance_needed(self):
    self.select_colreg_strategy()
    risk_active = self._main_route_has_collision_risk()
    if not risk_active:

```

```

        self._clear_apf_waypoint_path()
        return risk_active

def compute_route_tracking_control(self, t):
    if self._apf_waypoint_path_active() and not self._main_route_has_collision_risk():
        self._clear_apf_waypoint_path()
        tracking = self._compute_waypoint_tracking_control("apf_return")
        if tracking is not None:
            return tracking
        self._clear_apf_waypoint_path()
        return _OvertakingController.compute_route_tracking_control(self, t)

def write_obstacle_snapshot(self):
    stamp_s = self.latest_lidar_received_s
    virtual_obstacles = self.update_apf_virtual_obstacles()
    if virtual_obstacles:
        closest = min(virtual_obstacles, key=lambda item: float(item["dcpa_m"]))
        self.apf_colreg_dcpa_m = float(closest["dcpa_m"])
        self.apf_colreg_tcpa_s = float(closest["tcpa_s"])
    else:
        self.apf_colreg_dcpa_m = np.nan
        self.apf_colreg_tcpa_s = np.nan
    _OvertakingController.write_obstacle_snapshot(self)
    self._patch_latest_csv_context_row()
    if stamp_s is None:
        return

    path = self.obstacle_log_dir / f"obstacle_{int(round(float(stamp_s) * 1000.0))}.json"
    if not path.exists():
        return

    try:
        with path.open("r") as f:
            payload = json.load(f)
            payload["run_context"] = self._run_context()
            measured_velocity_ne = np.asarray(self.v_robot[0:2], dtype=float).reshape(2)
            speed_m_s = float(np.linalg.norm(measured_velocity_ne))
            if not np.isfinite(speed_m_s) or speed_m_s < 1e-6:
                speed_m_s = float(self.route_tracking_speed_m_s)
            forward_velocity_ne = self.body_vector_to_earth(np.array([speed_m_s, 0.0]))
            payload["robot_ekf"] = {
                "position_ne": [float(self.North), float(self.East)],
                "velocity_ne": forward_velocity_ne,
                "speed_m_s": speed_m_s,
                "heading_rad": float(self.Yaw),
            }
            apf = payload.setdefault("apf", {})
            apf["goal_ne"] = self.goal_ne
            apf["path_end_ne"] = self.goal_ne
            apf["selected_controller"] = self.apf_selected_controller
            apf["colreg_active"] = self.apf_colreg_active
            apf["active_profile"] = self.apf_active_profile_name
            if virtual_obstacles:
                apf["tcpa_s"] = float(closest["tcpa_s"])

```

```

        apf["dcpa_m"] = float(closest["dcpa_m"])
        apf["dcpa_position_ne"] = closest["centre_ne"]
        apf["obstacle_dcpa_position_ne"] = closest["
obstacle_dcpa_position_ne"]
        apf["obstacle_min_separation_point_ne"] = closest["
obstacle_min_separation_point_ne"]
        apf["obstacle_ekf_prediction_velocity_ne"] = closest["velocity_ne"]
        apf["obstacle_ekf_prediction_heading_rad"] = closest["heading_rad"]
        apf["obstacle_apf_ellipse_axis_ne"] = closest["apf_ellipse_axis_ne"]
        apf["obstacle_apf_ellipse_pc1_m"] = closest["apf_ellipse_pc1_m"]
        apf["obstacle_apf_ellipse_pc2_m"] = closest["apf_ellipse_pc2_m"]
        apf["robot_ekf_prediction_ne"] = closest["collision_position_ne"]
        apf["robot_ekf_prediction_velocity_ne"] = closest["
own_prediction_velocity_ne"]
        apf["robot_ekf_prediction_heading_rad"] = closest["
own_prediction_heading_rad"]
    else:
        apf.pop("tcpa_s", None)
        apf.pop("dcpa_m", None)
        with path.open("w") as f:
            json.dump(self.json_safe(payload), f, indent=2)
    except Exception:
        return

for _crossing_method_name in _CROSSING_METHODS:
    if not hasattr(LaptopController, _crossing_method_name):
        setattr(
            LaptopController,
            _crossing_method_name,
            getattr(_CrossingController, _crossing_method_name),
        )

```

A.4 Crossing encounter controller

```

"""
Copyright (c) 2025 The uos_sess6072_build Authors.
Authors: Blair Thornton, Alec O'Loughlin, Miquel Massot
All rights reserved.
Licensed under the BSD 3-Clause License.
See LICENSE.md file in the project root for full license information.
"""

import numpy as np
import time
from colreg_apf import constrain_velocity_to_axis, smooth_undirected_axis,
    straight_line_cpa
from math_sess6072 import Vector
from math_sess6072 import Vector

def Vector(dim):
    return np.zeros((dim, 1), dtype=float)

def wrap_angle(a):
    return (a + np.pi) % (2 * np.pi) - np.pi

def ellipse_level_and_away(offset, length_axis, semi_length_m, semi_width_m):

```



```

offset = np.asarray(offset, dtype=float).reshape(2)
length_axis = np.asarray(length_axis, dtype=float).reshape(2)
axis_norm = float(np.linalg.norm(length_axis))
if axis_norm < 1e-09:
    length_axis = np.array([1.0, 0.0], dtype=float)
else:
    length_axis = length_axis / axis_norm
width_axis = np.array([-length_axis[1], length_axis[0]], dtype=float)
semi_length_m = max(float(semi_length_m), 1e-06)
semi_width_m = max(float(semi_width_m), 1e-06)
along = float(np.dot(offset, length_axis))
across = float(np.dot(offset, width_axis))
level = float(np.hypot(along / semi_length_m, across / semi_width_m))
gradient = along / (semi_length_m * semi_length_m) * length_axis + across / (
    semi_width_m * semi_width_m) * width_axis
gradient_norm = float(np.linalg.norm(gradient))
if gradient_norm < 1e-09:
    offset_norm = float(np.linalg.norm(offset))
    away = offset / offset_norm if offset_norm >= 1e-09 else -length_axis
else:
    away = gradient / gradient_norm
return (level, away)

```

class LaptopController:

```

def earth_vector_to_body(self, vector_ne):
    pose = np.asarray(self.p_robot, dtype=float).reshape(-1)
    yaw = pose[2]
    c = np.cos(yaw)
    s = np.sin(yaw)
    vector_ne = np.asarray(vector_ne, dtype=float).reshape(2)
    return np.array([c * vector_ne[0] + s * vector_ne[1], -s * vector_ne[0] + c
        * vector_ne[1]])

def earth_point_to_body(self, point_ne):
    pose = np.asarray(self.p_robot, dtype=float).reshape(-1)
    return self.earth_vector_to_body(np.asarray(point_ne, dtype=float).reshape
        (2) - pose[0:2])

def body_vector_to_earth(self, vector_body):
    pose = np.asarray(self.p_robot, dtype=float).reshape(-1)
    yaw = pose[2]
    c = np.cos(yaw)
    s = np.sin(yaw)
    vector_body = np.asarray(vector_body, dtype=float).reshape(2)
    return np.array([c * vector_body[0] - s * vector_body[1], s * vector_body[0]
        + c * vector_body[1]])

def reset_apf_diagnostics(self, clear_visual=True):
    if clear_visual:
        self.apf_force_body = np.zeros(2, dtype=float)
        self.apf_repulsive_force_body = np.zeros(2, dtype=float)
        self.apf_attractive_force_body = np.zeros(2, dtype=float)
        self.apf_steering_force_body = np.zeros(2, dtype=float)
        self.apf_target_ne = np.array([np.nan, np.nan], dtype=float)
        self.apf_virtual_obstacles = []
        self.apf_visual_hold_until_s = 0.0
        self.apf_encounter_mode = 'none'

```

```

self.apf_colreg_rule = 'none'
self.apf_avoidance_side_sign = 0.0
self.apf_colreg_dcpa_m = np.nan
self.apf_colreg_tcpa_s = np.nan
self.apf_colreg_active = False

def current_velocity_body(self):
    vel_ne = np.asarray(self.v_robot[0:2], dtype=float).reshape(2)
    vel_body = self.earth_vector_to_body(vel_ne)
    if not np.isfinite(vel_body).all():
        return np.zeros(2, dtype=float)
    return vel_body

def obstacle_ekf_process_noise(self, dt):
    dt = max(float(dt), 0.001)
    q = float(self.obstacle_ekf_accel_std_m_s2) ** 2
    dt2 = dt * dt
    dt3 = dt2 * dt
    dt4 = dt2 * dt2
    return q * np.array([[0.25 * dt4, 0.0, 0.5 * dt3, 0.0], [0.0, 0.25 * dt4,
0.0, 0.5 * dt3], [0.5 * dt3, 0.0, dt2, 0.0], [0.0, 0.5 * dt3, 0.0, dt2]], dtype=
float)

def obstacle_measurement_covariance(self, centre_body):
    centre_body = np.asarray(centre_body, dtype=float).reshape(2)
    range_m = float(np.linalg.norm(centre_body)) if np.isfinite(centre_body).all
() else 0.0
    yaw_rate_rad_s = float(abs(getattr(self, 'sensed_imu_yaw_rate_rad_s', 0.0)
or 0.0))
    measurement_std_m = float(self.obstacle_ekf_measurement_std_m)
    lateral_std_m = measurement_std_m * (1.0 + 0.08 * range_m + 0.25 * range_m *
yaw_rate_rad_s)
    return np.diag([measurement_std_m ** 2, lateral_std_m ** 2])

def obstacle_ekf_predict(self, state, covariance, dt):
    dt = max(float(dt), 0.0)
    state = np.asarray(state, dtype=float).reshape(4)
    covariance = np.asarray(covariance, dtype=float).reshape(4, 4)
    transition = np.array([[1.0, 0.0, dt, 0.0], [0.0, 1.0, 0.0, dt], [0.0, 0.0,
1.0, 0.0], [0.0, 0.0, 0.0, 1.0]], dtype=float)
    return (transition @ state, transition @ covariance @ transition.T + self.
obstacle_ekf_process_noise(dt))

def obstacle_ekf_update(self, state, covariance, measurement_ne,
measurement_covariance=None):
    state = np.asarray(state, dtype=float).reshape(4)
    covariance = np.asarray(covariance, dtype=float).reshape(4, 4)
    measurement_ne = np.asarray(measurement_ne, dtype=float).reshape(2)
    observation = np.array([[1.0, 0.0, 0.0, 0.0], [0.0, 1.0, 0.0, 0.0]], dtype=
float)
    noise = self.obstacle_ekf_measurement_std_m ** 2 * np.eye(2, dtype=float) if
measurement_covariance is None else np.asarray(measurement_covariance, dtype=
float).reshape(2, 2)
    innovation = measurement_ne - observation @ state
    innovation_covariance = observation @ covariance @ observation.T + noise
    gain = covariance @ observation.T @ np.linalg.pinv(innovation_covariance)
    corrected_state = state + gain @ innovation
    correction = np.eye(4, dtype=float) - gain @ observation

```

```

        corrected_covariance = correction @ covariance @ correction.T + gain @ noise
        @ gain.T
        return (corrected_state, corrected_covariance)

def obstacle_track_motion_is_stable(self, track):
    sample_count = int(track.get('stats_sample_count', 0))
    hit_count = int(track.get('hit_count', 0))
    if sample_count < self.obstacle_prediction_min_samples or hit_count < self.
obstacle_prediction_min_hits or int(track.get('miss_count', 0)) > 0:
        return False
    samples = track.get('motion_window', [])
    if len(samples) < 2:
        return False
    first_stamp = float(samples[0].get('stamp_s', 0.0))
    last_stamp = float(samples[-1].get('stamp_s', first_stamp))
    if last_stamp - first_stamp < self.obstacle_prediction_min_time_span_s:
        return False
    speed_m_s = float(track.get('speed_mean_m_s', 0.0))
    was_stable = bool(track.get('motion_stable', False))
    speed_threshold = self.apf_dynamic_exit_speed_threshold_m_s if was_stable
else self.apf_dynamic_speed_threshold_m_s
    if not np.isfinite(speed_m_s) or speed_m_s < max(speed_threshold, getattr(
self, 'obstacle_prediction_min_speed_m_s', 0.0)):
        return False
    displacement_m = float(track.get('displacement_m', 0.0))
    if displacement_m < float(getattr(self, '
obstacle_prediction_min_displacement_m', 0.0)):
        return False
    return True

def obstacle_track_prediction_ne(self, track):
    if not self.obstacle_ekf_prediction_enabled:
        return np.empty((0, 2), dtype=float)
    state = np.asarray(track.get('state', [np.nan] * 4), dtype=float).reshape(4)
    if not np.isfinite(state).all():
        return np.empty((0, 2), dtype=float)
    horizon_s = float(track.get('collision_time_s', np.nan))
    if not np.isfinite(horizon_s) or horizon_s <= 0.0:
        horizon_s = float(getattr(self, 'obstacle_prediction_horizon_s', 0.0))
    if horizon_s <= 0.0:
        return np.empty((0, 2), dtype=float)
    step_s = max(float(self.obstacle_prediction_step_s), 0.001)
    times_s = np.arange(0.0, horizon_s, step_s, dtype=float)
    times_s = np.r_[times_s, horizon_s]
    return state[:2] + times_s[:, None] * state[2:4]

def obstacle_track_regularize_velocity(self, track):
    state = np.asarray(track.get('state', [np.nan] * 4), dtype=float).reshape(4)
    speed_m_s = float(np.linalg.norm(state[2:4]))
    if speed_m_s < float(self.obstacle_ekf_static_speed_reset_m_s):
        track['state'][2:4] = 0.0
    elif speed_m_s > float(self.obstacle_v_max_m_s):
        track['state'][2:4] *= float(self.obstacle_v_max_m_s) / speed_m_s

def sync_obstacle_track_fields(self, track):
    state = np.asarray(track.get('state', [np.nan] * 4), dtype=float).reshape(4)
    covariance = np.asarray(track.get('covariance', np.eye(4)), dtype=float).
reshape(4, 4)

```

```

track['state'] = state
track['covariance'] = covariance
track['pos_ne'] = state[0:2].copy()
track['vel_ne'] = state[2:4].copy()
track['position_uncertainty_m2'] = float(np.trace(covariance[0:2, 0:2]))
pc1_m = float(track.get('pc1_m', self.obstacle_min_pc1_m))
pc2_m = float(track.get('pc2_m', self.obstacle_min_pc2_m))
if not np.isfinite(pc1_m) or pc1_m <= 0.0:
    pc1_m = self.obstacle_min_pc1_m
if not np.isfinite(pc2_m) or pc2_m <= 0.0:
    pc2_m = self.obstacle_min_pc2_m
track['pc1_m'] = max(pc1_m, self.obstacle_min_pc1_m)
track['pc2_m'] = max(min(pc2_m, track['pc1_m']), self.obstacle_min_pc2_m)
track['radius_m'] = 0.5 * track['pc1_m']
track['equivalent_radius_m'] = track['radius_m']
length_axis_ne = np.asarray(track.get('length_axis_ne', [1.0, 0.0]), dtype=
float).reshape(2)
axis_norm = float(np.linalg.norm(length_axis_ne))
track['length_axis_ne'] = length_axis_ne / axis_norm if np.isfinite(
length_axis_ne).all() and axis_norm >= 1e-06 else np.array([1.0, 0.0], dtype=
float)
if bool(track.get('motion_stable', False)) and track['pc1_m'] >= self.
obstacle_axis_min_aspect_ratio * track['pc2_m']:
    state[2:4] = constrain_velocity_to_axis(state[2:4], track['
length_axis_ne'], self.obstacle_axis_max_velocity_gap_rad)
    track['state'] = state
    track['vel_ne'] = state[2:4].copy()
    display_velocity_ne = np.asarray(track['vel_ne'], dtype=float).reshape(2)
    if not np.isfinite(display_velocity_ne).all():
        display_velocity_ne = track['vel_ne']
    speed_m_s = float(np.linalg.norm(display_velocity_ne))
    track['speed_m_s'] = speed_m_s
    previous_heading_rad = float(track.get('heading_rad', np.nan))
    if speed_m_s >= float(getattr(self, 'obstacle_heading_hold_speed_m_s', 0.03)
):
        heading_rad = float(np.arctan2(display_velocity_ne[1],
display_velocity_ne[0]))
        track['heading_rad'] = heading_rad
        track['heading_deg'] = float(np.rad2deg(heading_rad))
        track['heading_axis_ne'] = display_velocity_ne / speed_m_s
    else:
        if not np.isfinite(previous_heading_rad):
            previous_heading_rad = float(np.arctan2(track['length_axis_ne'][1],
track['length_axis_ne'][0]))
        track['heading_rad'] = wrap_angle(previous_heading_rad)
        track['heading_deg'] = float(np.rad2deg(track['heading_rad']))
        track['heading_axis_ne'] = track['length_axis_ne'].copy()
if not self.obstacle_ekf_prediction_enabled:
    track['prediction_model'] = 'disabled'
    track['prediction_ne'] = np.empty((0, 2), dtype=float)
    return
if bool(track.get('motion_stable', False)):
    track['prediction_model'] = 'cv_ekf'
else:
    track['prediction_model'] = 'cv_ekf_warmup'
track['prediction_ne'] = self.obstacle_track_prediction_ne(track)

```

```

def make_obstacle_track(self, detection_ne, stamp_s, pc1_m=np.nan, pc2_m=np.nan,
length_axis_ne=None):
    detection_ne = np.asarray(detection_ne, dtype=float).reshape(2)
    pc1_m = float(pc1_m) if np.isfinite(pc1_m) and pc1_m > 0.0 else self.
obstacle_min_pc1_m
    pc2_m = float(pc2_m) if np.isfinite(pc2_m) and pc2_m > 0.0 else self.
obstacle_min_pc2_m
    length_axis_ne = np.asarray([1.0, 0.0] if length_axis_ne is None else
length_axis_ne, dtype=float).reshape(2)
    covariance = np.diag([self.obstacle_ekf_initial_position_std_m ** 2, self.
obstacle_ekf_initial_position_std_m ** 2, self.
obstacle_ekf_initial_velocity_std_m_s ** 2, self.
obstacle_ekf_initial_velocity_std_m_s ** 2])
    track = {'id': self.apf_next_track_id, 'state': np.array([detection_ne[0],
detection_ne[1], 0.0, 0.0], dtype=float), 'covariance': covariance, 'stamp_s':
float(stamp_s), 'last_seen_s': float(stamp_s), 'hit_count': 1, 'miss_count': 0, '
history_ne': [], 'lidar_history_ne': [], 'motion_window': [], 'pc1_m': pc1_m, '
pc2_m': pc2_m, 'length_axis_ne': length_axis_ne, 'raw_detection_ne':
detection_ne.copy()}
    self.sync_obstacle_track_fields(track)
    self.append_obstacle_track_history(track, pc1_m=pc1_m, pc2_m=pc2_m,
length_axis_ne=length_axis_ne, detection_ne=detection_ne, stamp_s=stamp_s)
    self.apf_next_track_id += 1
    return track

def make_obstacle_track_candidate(self, detection_ne, stamp_s, pc1_m=np.nan,
pc2_m=np.nan, length_axis_ne=None):
    detection_ne = np.asarray(detection_ne, dtype=float).reshape(2)
    length_axis_ne = np.asarray([1.0, 0.0] if length_axis_ne is None else
length_axis_ne, dtype=float).reshape(2)
    pc1_m = float(pc1_m) if np.isfinite(pc1_m) and pc1_m > 0.0 else self.
obstacle_min_pc1_m
    pc2_m = float(pc2_m) if np.isfinite(pc2_m) and pc2_m > 0.0 else self.
obstacle_min_pc2_m
    return {'centre_ne': detection_ne.copy(), 'stamp_s': float(stamp_s), '
last_seen_s': float(stamp_s), 'hit_count': 1, 'pc1_m': pc1_m, 'pc2_m': pc2_m, '
length_axis_ne': length_axis_ne}

def predict_obstacle_track_to_time(self, track, stamp_s):
    now = float(stamp_s)
    dt = max(now - float(track.get('stamp_s', now)), 0.0)
    if not np.isfinite(dt) or dt < 0.001:
        return
    state, covariance = self.obstacle_ekf_predict(track.get('state', np.r_[track.
get('pos_ne', [np.nan, np.nan]), 0.0, 0.0]), track.get('covariance', np.eye(4,
dtype=float)), dt)
    track['state'] = state
    track['covariance'] = covariance
    track['stamp_s'] = now
    self.sync_obstacle_track_fields(track)

def append_obstacle_track_history(self, track, pc1_m=np.nan, pc2_m=np.nan,
length_axis_ne=None, detection_ne=None, stamp_s=None):
    stamp_s = float(stamp_s if stamp_s is not None else track.get('stamp_s',
time.time()))
    pos_ne = np.asarray(track['pos_ne'], dtype=float).reshape(2).copy()
    vel_ne = np.asarray(track.get('vel_ne', [0.0, 0.0]), dtype=float).reshape(2).
copy()

```

```

        if not np.isfinite(vel_ne).all():
            vel_ne = np.zeros(2, dtype=float)
            pc1_m = float(pc1_m) if np.isfinite(pc1_m) and pc1_m > 0.0 else float(track.
get('pc1_m', self.obstacle_min_pc1_m))
            pc2_m = float(pc2_m) if np.isfinite(pc2_m) and pc2_m > 0.0 else float(track.
get('pc2_m', self.obstacle_min_pc2_m))
            track['pc1_m'] = max(pc1_m, self.obstacle_min_pc1_m)
            track['pc2_m'] = max(min(pc2_m, track['pc1_m']), self.obstacle_min_pc2_m)
            if length_axis_ne is not None:
                length_axis_ne = np.asarray(length_axis_ne, dtype=float).reshape(2)
                axis_norm = float(np.linalg.norm(length_axis_ne))
                if np.isfinite(length_axis_ne).all() and axis_norm >= 1e-06:
                    length_axis_ne = length_axis_ne / axis_norm
                    previous_axis = np.asarray(track.get('length_axis_ne',
length_axis_ne), dtype=float).reshape(2)
                    if float(np.dot(length_axis_ne, previous_axis)) < 0.0:
                        length_axis_ne = -length_axis_ne
                    track['length_axis_ne'] = smooth_undirected_axis(previous_axis,
length_axis_ne, self.obstacle_axis_smoothing_alpha)
                history = track.setdefault('history_ne', [])
                history.append(pos_ne)
                if len(history) > self.obstacle_history_len:
                    del history[:-self.obstacle_history_len]
                if detection_ne is not None:
                    detection_ne = np.asarray(detection_ne, dtype=float).reshape(2)
                    if np.isfinite(detection_ne).all():
                        lidar_history = track.setdefault('lidar_history_ne', [])
                        lidar_times = track.setdefault('lidar_history_timestamps_s', [])
                        lidar_history.append(detection_ne.copy())
                        lidar_times.append(float(timestamp_s))
                        if len(lidar_history) > self.obstacle_history_len:
                            del lidar_history[:-self.obstacle_history_len]
                            del lidar_times[:-self.obstacle_history_len]
                    speed_m_s = float(np.linalg.norm(vel_ne))
                    heading_rad = float(np.arctan2(vel_ne[1], vel_ne[0])) if speed_m_s >= self.
apf_dynamic_speed_threshold_m_s else np.nan
                    motion_window = track.setdefault('motion_window', [])
                    motion_window.append({'stamp_s': timestamp_s, 'pos_ne': pos_ne, 'vel_ne': vel_ne
, 'speed_m_s': speed_m_s, 'heading_rad': heading_rad, 'pc1_m': track['pc1_m'], '
pc2_m': track['pc2_m']})
                    cutoff_s = timestamp_s - max(float(self.obstacle_stats_window_s), 0.0)
                    track['motion_window'] = [sample for sample in motion_window if float(sample.
get('stamp_s', timestamp_s)) >= cutoff_s][-self.obstacle_history_len:]
                    self.update_obstacle_track_statistics(track)

def update_obstacle_track_statistics(self, track):
    samples = track.get('motion_window', [])
    samples = [sample for sample in samples if np.isfinite(np.asarray(sample.get
('pos_ne', [np.nan, np.nan]), dtype=float)).all())
    track['stats_sample_count'] = int(len(samples))
    if not samples:
        track['velocity_mean_ne'] = np.asarray(track.get('vel_ne', [0.0, 0.0]),
dtype=float).reshape(2)
        track['velocity_var_ne'] = np.zeros(2, dtype=float)
        track['speed_mean_m_s'] = float(np.linalg.norm(track['velocity_mean_ne'
]))
        track['speed_var_m2_s2'] = 0.0
        track['heading_mean_rad'] = np.nan

```



```

        track['heading_var_rad2'] = np.nan
        track['heading_circular_variance'] = np.nan
        track['pc1_mean_m'] = track.get('pc1_m', self.obstacle_min_pc1_m)
        track['pc2_mean_m'] = track.get('pc2_m', self.obstacle_min_pc2_m)
        track['pc1_var_m2'] = 0.0
        track['pc2_var_m2'] = 0.0
        track['displacement_m'] = 0.0
        track['motion_stable'] = False
        self.sync_obstacle_track_fields(track)
        return
    velocities = np.asarray([sample['vel_ne'] for sample in samples], dtype=
float)
    finite_vel = np.isfinite(velocities).all(axis=1)
    velocities = velocities[finite_vel]
    sample_times = np.asarray([float(sample.get('stamp_s', 0.0)) for sample in
samples], dtype=float)
    sample_positions = np.asarray([sample['pos_ne'] for sample in samples],
dtype=float)
    time_span_s = float(np.ptp(sample_times)) if len(sample_times) > 1 else 0.0
    if len(samples) >= 3 and time_span_s >= 0.4:
        centred_times = sample_times - float(np.mean(sample_times))
        design = np.column_stack([centred_times, np.ones_like(centred_times)])
        fit, _, _, _ = np.linalg.lstsq(design, sample_positions, rcond=None)
        track['velocity_mean_ne'] = fit[0]
    elif len(velocities) > 0:
        track['velocity_mean_ne'] = np.mean(velocities, axis=0)
    else:
        track['velocity_mean_ne'] = np.asarray(track.get('vel_ne', [0.0, 0.0]),
dtype=float).reshape(2)
    if len(velocities) > 0:
        track['velocity_var_ne'] = np.var(velocities, axis=0)
    else:
        track['velocity_var_ne'] = np.zeros(2, dtype=float)
    speeds = np.asarray([sample['speed_m_s'] for sample in samples], dtype=float
)
    speeds = speeds[np.isfinite(speeds)]
    fitted_speed_m_s = float(np.linalg.norm(track['velocity_mean_ne']))
    if len(speeds) > 0:
        track['speed_mean_m_s'] = fitted_speed_m_s
        track['speed_var_m2_s2'] = float(np.var(speeds))
    else:
        track['speed_mean_m_s'] = fitted_speed_m_s
        track['speed_var_m2_s2'] = 0.0
    track['displacement_m'] = float(np.linalg.norm(sample_positions[-1] -
sample_positions[0])) if len(sample_positions) >= 2 else 0.0
    headings = np.asarray([sample['heading_rad'] for sample in samples], dtype=
float)
    headings = headings[np.isfinite(headings)]
    if len(headings) > 0:
        sin_mean = float(np.mean(np.sin(headings)))
        cos_mean = float(np.mean(np.cos(headings)))
        heading_mean = float(np.arctan2(sin_mean, cos_mean))
        heading_error = wrap_angle(headings - heading_mean)
        resultant_length = float(np.hypot(sin_mean, cos_mean))
        track['heading_mean_rad'] = heading_mean
        track['heading_var_rad2'] = float(np.var(heading_error))
        track['heading_circular_variance'] = float(1.0 - np.clip(
resultant_length, 0.0, 1.0))

```

```

else:
    track['heading_mean_rad'] = np.nan
    track['heading_var_rad2'] = np.nan
    track['heading_circular_variance'] = np.nan
    pc1_values = np.asarray([sample['pc1_m'] for sample in samples], dtype=float)
)
    pc2_values = np.asarray([sample['pc2_m'] for sample in samples], dtype=float)
)
    pc1_values = pc1_values[np.isfinite(pc1_values) & (pc1_values > 0.0)]
    pc2_values = pc2_values[np.isfinite(pc2_values) & (pc2_values > 0.0)]
    track['pc1_mean_m'] = max(float(np.mean(pc1_values)), self.
obstacle_min_pc1_m) if len(pc1_values) > 0 else float(track.get('pc1_m', self.
obstacle_min_pc1_m))
    track['pc2_mean_m'] = max(float(np.mean(pc2_values)), self.
obstacle_min_pc2_m) if len(pc2_values) > 0 else float(track.get('pc2_m', self.
obstacle_min_pc2_m))
    track['pc2_mean_m'] = min(track['pc2_mean_m'], track['pc1_mean_m'])
    track['pc1_var_m2'] = float(np.var(pc1_values)) if len(pc1_values) > 0 else
0.0
    track['pc2_var_m2'] = float(np.var(pc2_values)) if len(pc2_values) > 0 else
0.0
    track['pc1_m'] = track['pc1_mean_m']
    track['pc2_m'] = track['pc2_mean_m']
    track['motion_stable'] = self.obstacle_track_motion_is_stable(track)
    self.obstacle_track_regularize_velocity(track)
    self.sync_obstacle_track_fields(track)

def annotate_lidar_obstacle_with_track(self, obstacle, track):
    prediction_ne = np.asarray(track.get('prediction_ne', np.empty((0, 2))),
dtype=float)
    raw_centre_ne = np.asarray(obstacle.get('centre_ne', [np.nan, np.nan]),
dtype=float).reshape(2)
    obstacle['track_id'] = int(track['id'])
    obstacle['raw_centre_ne'] = raw_centre_ne.tolist()
    obstacle['raw_centre_body'] = np.asarray(obstacle.get('centre_body', [np.nan
, np.nan]), dtype=float).reshape(2).tolist()
    obstacle['raw_px'] = float(raw_centre_ne[0])
    obstacle['raw_py'] = float(raw_centre_ne[1])
    obstacle['filtered_px'] = float(track['pos_ne'][0])
    obstacle['filtered_py'] = float(track['pos_ne'][1])
    predicted_position_ne = prediction_ne[-1] if prediction_ne.ndim == 2 and len
(prediction_ne) > 0 else np.asarray(track.get('virtual_position_ne', [np.nan, np.
nan]), dtype=float).reshape(2)
    obstacle['predicted_px'] = float(predicted_position_ne[0])
    obstacle['predicted_py'] = float(predicted_position_ne[1])
    obstacle['centre_ne'] = track['pos_ne'].tolist()
    obstacle['centre_body'] = self.earth_point_to_body(track['pos_ne']).tolist()
    obstacle['distance_m'] = float(np.linalg.norm(obstacle['centre_body']))
    obstacle['min_distance_m'] = float(min(obstacle.get('min_distance_m',
obstacle['distance_m']), obstacle['distance_m']))
    obstacle['angle_rad'] = float(np.arctan2(obstacle['centre_body'][1],
obstacle['centre_body'][0]))
    obstacle['angle_deg'] = float(np.rad2deg(obstacle['angle_rad']))
    obstacle['velocity_ne'] = np.asarray(track['vel_ne'], dtype=float).reshape
(2).tolist()
    obstacle['velocity_mean_ne'] = np.asarray(track.get('velocity_mean_ne',
track['vel_ne']), dtype=float).reshape(2).tolist()

```



```

        obstacle['velocity_var_ne'] = np.asarray(track.get('velocity_var_ne', [0.0,
0.0]), dtype=float).reshape(2).tolist()
        obstacle['speed_m_s'] = float(track.get('speed_m_s', 0.0))
        obstacle['speed_mean_m_s'] = float(track.get('speed_mean_m_s', obstacle['
speed_m_s']))
        obstacle['speed_var_m2_s2'] = float(track.get('speed_var_m2_s2', 0.0))
        obstacle['heading_rad'] = float(track.get('heading_rad', np.nan))
        obstacle['heading_deg'] = float(track.get('heading_deg', np.nan))
        obstacle['heading_mean_rad'] = float(track.get('heading_mean_rad', np.nan))
        obstacle['heading_var_rad2'] = float(track.get('heading_var_rad2', np.nan))
        obstacle['pc1_m'] = float(track.get('pc1_mean_m', track.get('pc1_m', self.
obstacle_min_pc1_m)))
        obstacle['pc2_m'] = float(track.get('pc2_mean_m', track.get('pc2_m', self.
obstacle_min_pc2_m)))
        obstacle['pc1_var_m2'] = float(track.get('pc1_var_m2', 0.0))
        obstacle['pc2_var_m2'] = float(track.get('pc2_var_m2', 0.0))
        obstacle['length_axis_ne'] = np.asarray(track.get('heading_axis_ne', track.
get('length_axis_ne', [1.0, 0.0]), dtype=float).reshape(2).tolist()
        obstacle['equivalent_radius_m'] = float(track.get('equivalent_radius_m',
self.obstacle_min_equivalent_radius_m))
        obstacle['stats_sample_count'] = int(track.get('stats_sample_count', 0))
        obstacle['motion_stable'] = bool(track.get('motion_stable', False))
        obstacle['position_uncertainty_m2'] = float(track.get('
position_uncertainty_m2', np.nan))
        obstacle['covariance_trace'] = float(np.trace(np.asarray(track.get('
covariance', np.eye(4, dtype=float)), dtype=float)))
        obstacle['prediction_model'] = track.get('prediction_model', 'cv_ekf')
        obstacle['predicted_trajectory_ne'] = prediction_ne.tolist()
        lidar_points_ne = np.asarray(obstacle.get('points_ne', []), dtype=float)
        if lidar_points_ne.ndim == 2 and lidar_points_ne.shape[1] == 2:
            track['lidar_points_ne'] = lidar_points_ne.tolist()

def prune_obstacle_tracks(self, now):
    self.apf_obstacle_tracks = [track for track in self.apf_obstacle_tracks if
now - float(track.get('last_seen_s', track.get('stamp_s', now))) <= self.
apf_track_timeout_s and int(track.get('miss_count', 0)) <= 5]

def update_apf_obstacle_tracks(self, stamp_s):
    tracking_enabled = bool(getattr(self, 'obstacle_ekf_tracking_enabled', True)
)
    if not tracking_enabled:
        self.apf_obstacle_tracks = []
        self.apf_virtual_obstacles = []
        tracked_fields = {'track_id', 'raw_px', 'raw_py', 'filtered_px', '
filtered_py', 'predicted_px', 'predicted_py', 'velocity_ne', 'velocity_mean_ne',
'velocity_var_ne', 'speed_m_s', 'speed_mean_m_s', 'speed_var_m2_s2', '
heading_rad', 'heading_deg', 'heading_mean_rad', 'heading_var_rad2', 'pc1_m', '
pc2_m', 'pc1_var_m2', 'pc2_var_m2', 'stats_sample_count', 'motion_stable', '
position_uncertainty_m2', 'covariance_trace', 'prediction_model', '
predicted_trajectory_ne'}
        for obstacle in self.lidar_obstacles:
            for field in tracked_fields:
                obstacle.pop(field, None)
        return
    now = float(stamp_s if stamp_s is not None else time.time())
    confirmation_hits = max(int(getattr(self, 'obstacle_track_confirmation_hits'
, 2)), 2)
    detections = []

```

```

        for obstacle_index, obstacle in enumerate(self.lidar_obstacles):
            centre_ne = np.asarray(obstacle.get('centre_ne', [np.nan, np.nan]),
                                   dtype=float).reshape(2)
            if np.isfinite(centre_ne).all():
                pc1_m = float(obstacle.get('pc1_m', self.obstacle_min_pc1_m))
                pc2_m = float(obstacle.get('pc2_m', self.obstacle_min_pc2_m))
                length_axis_ne = np.asarray(obstacle.get('length_axis_ne', [1.0,
0.0]), dtype=float).reshape(2)
                measurement_covariance = np.asarray(obstacle.get('
measurement_covariance', self.obstacle_measurement_covariance(obstacle.get('
centre_body', [0.0, 0.0]))), dtype=float).reshape(2, 2)
                detections.append((obstacle_index, centre_ne, pc1_m, pc2_m,
length_axis_ne, measurement_covariance))
            if not detections:
                self.apf_obstacle_track_candidates = [candidate for candidate in self.
apf_obstacle_track_candidates if now - float(candidate.get('last_seen_s',
candidate.get('stamp_s', now))) <= self.apf_track_timeout_s]
                for track in self.apf_obstacle_tracks:
                    self.predict_obstacle_track_to_time(track, now)
                    track['miss_count'] = int(track.get('miss_count', 0)) + 1
                self.prune_obstacle_tracks(now)
            return
            detection_positions = np.asarray([detection for _, detection, _, _, _ in
detections], dtype=float)
            predicted_tracks = []
            candidates = []
            matched_candidates = set()
            promoted_candidates = set()
            for track_index, track in enumerate(self.apf_obstacle_tracks):
                dt = max(now - float(track.get('stamp_s', now)), 0.0)
                predicted_state, predicted_covariance = self.obstacle_ekf_predict(track.
get('state', np.r_[track.get('pos_ne', [np.nan, np.nan]), 0.0, 0.0]), track.get('
covariance', np.eye(4, dtype=float)), dt)
                predicted_tracks.append((predicted_state, predicted_covariance))
                predicted_pos = predicted_state[0:2]
                for detection_index, detection in enumerate(detection_positions):
                    distance = float(np.linalg.norm(detection - predicted_pos))
                    H = np.array([[1.0, 0.0, 0.0, 0.0], [0.0, 1.0, 0.0, 0.0]])
                    innovation = detection - predicted_pos
                    measurement_covariance = detections[detection_index][5]
                    innovation_covariance = H @ np.asarray(predicted_covariance, dtype=
float) @ H.T + measurement_covariance
                    try:
                        mahalanobis_sq = float(innovation.T @ np.linalg.inv(
innovation_covariance) @ innovation)
                    except np.linalg.LinAlgError:
                        mahalanobis_sq = float(innovation.T @ np.linalg.pinv(
innovation_covariance) @ innovation)
                    candidates.append((mahalanobis_sq, distance, track_index,
detection_index))
            candidates.sort(key=lambda item: (item[0], item[1]))
            assigned_tracks = set()
            assigned_detections = set()
            detection_track = {}
            for mahalanobis_sq, distance, track_index, detection_index in candidates:
                if distance > self.apf_track_association_m or mahalanobis_sq > 9.21:
                    continue

```

```

        if track_index in assigned_tracks or detection_index in
assigned_detections:
            continue
        track = self.apf_obstacle_tracks[track_index]
        detection = detection_positions[detection_index]
        predicted_state, predicted_covariance = predicted_tracks[track_index]
        corrected_state, corrected_covariance = self.obstacle_ekf_update(
predicted_state, predicted_covariance, detection, detections[detection_index
][5])
        track['state'] = corrected_state
        track['covariance'] = corrected_covariance
        track['stamp_s'] = now
        track['last_seen_s'] = now
        track['hit_count'] = int(track.get('hit_count', 0)) + 1
        track['miss_count'] = 0
        track['raw_detection_ne'] = detection.copy()
        self.sync_obstacle_track_fields(track)
        self.append_obstacle_track_history(track, pc1_m=detections[
detection_index][2], pc2_m=detections[detection_index][3], length_axis_ne=
detections[detection_index][4], detection_ne=detection, stamp_s=now)
        assigned_tracks.add(track_index)
        assigned_detections.add(detection_index)
        detection_track[detection_index] = track
    for track_index, track in enumerate(self.apf_obstacle_tracks):
        if track_index not in assigned_tracks:
            predicted_state, predicted_covariance = predicted_tracks[track_index
]

            track['state'] = predicted_state
            track['covariance'] = predicted_covariance
            track['stamp_s'] = now
            track['miss_count'] = int(track.get('miss_count', 0)) + 1
            self.sync_obstacle_track_fields(track)
    for detection_index, detection in enumerate(detections):
        if detection_index in assigned_detections:
            continue
        _, detection_ne, pc1_m, pc2_m, length_axis_ne, _ = detection
        best_candidate_index = None
        best_candidate_distance = np.inf
        for candidate_index, candidate in enumerate(self.
apf_obstacle_track_candidates):
            if candidate_index in matched_candidates:
                continue
            candidate_pos = np.asarray(candidate.get('centre_ne', [np.nan, np.
nan]), dtype=float).reshape(2)
            if not np.isfinite(candidate_pos).all():
                continue
            distance = float(np.linalg.norm(detection_ne - candidate_pos))
            if distance < best_candidate_distance:
                best_candidate_distance = distance
                best_candidate_index = candidate_index
        if best_candidate_index is not None and best_candidate_distance <= self.
apf_track_association_m:
            candidate = self.apf_obstacle_track_candidates[best_candidate_index]
            candidate['centre_ne'] = detection_ne.copy()
            candidate['last_seen_s'] = now
            candidate['hit_count'] = int(candidate.get('hit_count', 0)) + 1
            candidate['pc1_m'] = pc1_m
            candidate['pc2_m'] = pc2_m

```

```

        candidate['length_axis_ne'] = length_axis_ne
        matched_candidates.add(best_candidate_index)
        if int(candidate['hit_count']) >= confirmation_hits:
            track = self.make_obstacle_track(detection_ne, now, pc1_m=pc1_m,
            pc2_m=pc2_m, length_axis_ne=length_axis_ne)
            self.apf_obstacle_tracks.append(track)
            assigned_detections.add(detection_index)
            detection_track[detection_index] = track
            promoted_candidates.add(best_candidate_index)
        continue

    self.apf_obstacle_track_candidates.append(self.
make_obstacle_track_candidate(detection_ne, now, pc1_m=pc1_m, pc2_m=pc2_m,
length_axis_ne=length_axis_ne))
    for detection_index, track in detection_track.items():
        obstacle_index, _, _, _, _ = detections[detection_index]
        self.annotate_lidar_obstacle_with_track(self.lidar_obstacles[
obstacle_index], track)
        self.apf_obstacle_track_candidates = [candidate for candidate_index,
candidate in enumerate(self.apf_obstacle_track_candidates) if candidate_index
not in promoted_candidates if now - float(candidate.get('last_seen_s', candidate.
get('stamp_s', now))) <= self.apf_track_timeout_s]
        self.prune_obstacle_tracks(now)

def obstacle_pc_dimensions(self, obstacle):
    min_pc1_m = float(getattr(self, 'obstacle_min_pc1_m', 0.3))
    min_pc2_m = float(getattr(self, 'obstacle_min_pc2_m', 0.16))
    pc1_m = float(obstacle.get('pc1_m', min_pc1_m))
    pc2_m = float(obstacle.get('pc2_m', min_pc2_m))
    if not np.isfinite(pc1_m) or pc1_m <= 0.0:
        pc1_m = min_pc1_m
    if not np.isfinite(pc2_m) or pc2_m <= 0.0:
        pc2_m = min_pc2_m
    pc1_m = max(pc1_m, min_pc1_m)
    pc2_m = max(min(pc2_m, pc1_m), min_pc2_m)
    return (pc1_m, pc2_m)

def obstacle_length_axis_ne(self, obstacle):
    state = np.asarray(obstacle.get('state', []), dtype=float).reshape(-1)
    if state.size == 4 and np.isfinite(state).all():
        velocity_ne = state[2:4]
        velocity_norm = float(np.linalg.norm(velocity_ne))
        if velocity_norm >= self.apf_dynamic_speed_threshold_m_s:
            return velocity_ne / velocity_norm
    for key in ('velocity_ne', 'vel_ne', 'velocity_mean_ne'):
        velocity_ne = np.asarray(obstacle.get(key, [np.nan, np.nan]), dtype=
float).reshape(2)
        velocity_norm = float(np.linalg.norm(velocity_ne))
        if np.isfinite(velocity_ne).all() and velocity_norm >= self.
apf_dynamic_speed_threshold_m_s:
            return velocity_ne / velocity_norm
    axis_ne = np.asarray(obstacle.get('length_axis_ne', [1.0, 0.0]), dtype=float
).reshape(2)
    axis_norm = float(np.linalg.norm(axis_ne))
    if not np.isfinite(axis_ne).all() or axis_norm < 1e-06:
        return np.array([1.0, 0.0], dtype=float)
    return axis_ne / axis_norm

def update_apf_virtual_obstacles(self):

```

```

        """Log the same EKF/own-ship trajectories used for predicted CPA."""
        self.apf_virtual_obstacles = []
        if not self.obstacle_ekf_prediction_enabled:
            return self.apf_virtual_obstacles
        horizon_s = float(self.apf_collision_horizon_s)
        step_s = max(float(self.apf_prediction_dt_s), 0.05)
        times_s = np.arange(0.0, horizon_s + 0.5 * step_s, step_s)
        own_start_ne = np.array([float(self.North), float(self.East)], dtype=float)
        measured_velocity_ne = np.asarray(self.v_robot[0:2], dtype=float).reshape(2)
        speed_m_s = float(np.linalg.norm(measured_velocity_ne))
        if not np.isfinite(speed_m_s) or speed_m_s < 1e-06:
            speed_m_s = float(self.route_tracking_speed_m_s)
        own_velocity_ne = self.body_vector_to_earth(np.array([speed_m_s, 0.0]))
        own_trajectory_ne = own_start_ne + times_s[:, None] * own_velocity_ne
        for track in self.apf_obstacle_tracks:
            if not self.obstacle_track_motion_is_stable(track):
                continue
            start_ne = np.asarray(track.get('pos_ne', track.get('position_ne', [np.
nan, np.nan])), dtype=float).reshape(2)
            predicted_start_ne, velocity_ne = self.obstacle_track_state_at(track,
0.0)
            if predicted_start_ne is not None:
                start_ne = np.asarray(predicted_start_ne, dtype=float).reshape(2)
            if not np.isfinite(start_ne).all() or velocity_ne is None:
                continue
            velocity_ne = np.asarray(velocity_ne, dtype=float).reshape(2)
            speed_m_s = float(np.linalg.norm(velocity_ne))
            if not np.isfinite(velocity_ne).all() or speed_m_s < self.
apf_dynamic_speed_threshold_m_s:
                continue
            tcpa_s, dcpa_m, own_cpa_ne, obstacle_cpa_ne = straight_line_cpa(
own_start_ne, own_velocity_ne, start_ne, velocity_ne, horizon_s)
            if tcpa_s <= 0.0:
                continue
            pc1_m, pc2_m = self.obstacle_pc_dimensions(track)
            ellipse_axis_ne = velocity_ne / max(float(np.linalg.norm(velocity_ne)),
1e-06)
            collision_level, _ = ellipse_level_and_away(own_cpa_ne - obstacle_cpa_ne
, ellipse_axis_ne, 0.5 * pc1_m + self.apf_own_equivalent_radius_m, 0.5 * pc2_m +
self.apf_own_equivalent_radius_m)
            if collision_level >= float(self.apf_risk_pc_scale):
                continue
            centre_trajectory_ne = start_ne + times_s[:, None] * velocity_ne
            self.apf_virtual_obstacles.append({'label': -1000 - int(track.get('id',
0)), 'virtual': True, 'track_id': int(track.get('id', 0)), 'centre_ne':
obstacle_cpa_ne.tolist(), 'centre_body': self.earth_point_to_body(
obstacle_cpa_ne).tolist(), 'pc1_m': pc1_m, 'pc2_m': pc2_m, 'length_axis_ne':
ellipse_axis_ne.tolist(), 'apf_ellipse_axis_ne': ellipse_axis_ne.tolist(), '
lidar_motion_axis_ne': ellipse_axis_ne.tolist(), 'apf_ellipse_pc1_m': pc1_m, '
apf_ellipse_pc2_m': pc2_m, 'velocity_ne': velocity_ne.tolist(), 'tcpa_s': tcpa_s,
'heading_rad': float(np.arctan2(velocity_ne[1], velocity_ne[0])), 'dcpa_m':
dcpa_m, 'collision_position_ne': own_cpa_ne.tolist(), 'predicted_risk_active':
True, 'collision_level': float(collision_level), 'obstacle_dcpa_position_ne':
obstacle_cpa_ne.tolist(), 'obstacle_min_separation_point_ne': obstacle_cpa_ne.
tolist(), 'own_prediction_velocity_ne': own_velocity_ne.tolist(), '
own_prediction_heading_rad': float(self.Yaw), 'own_prediction_ne':
own_trajectory_ne.tolist(), 'obstacle_prediction_ne': centre_trajectory_ne.
tolist()})

```

```

        return self.apf_virtual_obstacles

def apf_pass_astern_side_from_velocity(self, obs_vel_body):
    obs_vel_body = np.asarray(obs_vel_body, dtype=float).reshape(2)
    if not np.isfinite(obs_vel_body).all():
        return 0.0
    if float(np.linalg.norm(obs_vel_body)) < self.
apf_dynamic_speed_threshold_m_s:
    return 0.0
    lateral_speed = float(obs_vel_body[1])
    if abs(lateral_speed) < self.apf_dynamic_speed_threshold_m_s:
        return 0.0
    return float(np.sign(lateral_speed))

def apf_obstacle_endpoint_direction_body(self, obs_pos_body, obs_vel_body,
obstacle, along_sign):
    obs_pos_body = np.asarray(obs_pos_body, dtype=float).reshape(2)
    obs_vel_body = np.asarray(obs_vel_body, dtype=float).reshape(2)
    if not np.isfinite(obs_pos_body).all() or not np.isfinite(obs_vel_body).all
():
        return np.zeros(2, dtype=float)
    obs_speed = float(np.linalg.norm(obs_vel_body))
    if obs_speed < self.apf_dynamic_speed_threshold_m_s:
        return np.zeros(2, dtype=float)
    axis_body = obs_vel_body / max(obs_speed, 1e-06)
    pc1_m, _ = self.obstacle_pc_dimensions(obstacle)
    endpoint_body = obs_pos_body + float(along_sign) * 0.5 * pc1_m * axis_body
    endpoint_distance = float(np.linalg.norm(endpoint_body))
    if endpoint_distance < 1e-06:
        return float(along_sign) * axis_body
    return endpoint_body / endpoint_distance

def apf_stern_direction_body(self, obs_pos_body, obs_vel_body, obstacle):
    return self.apf_obstacle_endpoint_direction_body(obs_pos_body, obs_vel_body,
obstacle, -1.0)

def apf_bow_direction_body(self, obs_pos_body, obs_vel_body, obstacle):
    return self.apf_obstacle_endpoint_direction_body(obs_pos_body, obs_vel_body,
obstacle, 1.0)

def apf_crossing_strategy_from_velocity(self, obs_vel_body):
    obs_vel_body = np.asarray(obs_vel_body, dtype=float).reshape(2)
    if not np.isfinite(obs_vel_body).all():
        return ('none', 0.0)
    if float(np.linalg.norm(obs_vel_body)) < self.
apf_dynamic_speed_threshold_m_s:
        return ('none', 0.0)
    lateral_speed = float(obs_vel_body[1])
    if abs(lateral_speed) < self.apf_dynamic_speed_threshold_m_s:
        return ('none', 0.0)
    return ('pass_astern', self.apf_pass_astern_side_from_velocity(obs_vel_body)
)

def apf_encounter_speed_m_s(self, encounter, nearest_active_level, force_angle):
    base_speed = float(getattr(self, 'apf_constant_descent_speed_m_s', self.
route_tracking_speed_m_s))
    if encounter == 'overtaking':

```



```

        return float(np.clip(max(base_speed, getattr(self, '
apf_overtaking_surge_m_s', base_speed)), 0.0, self.v_max))
        if isinstance(encounter, str) and encounter.startswith('crossing'):
            pass_ahead_active = 'ahead' in str(getattr(self, 'apf_colreg_rule', ''))
lower()
        if np.isfinite(nearest_active_level) and nearest_active_level <= 1.0:
            target_speed = max(getattr(self, 'apf_crossing_min_forward_speed',
base_speed), getattr(self, 'apf_crossing_close_quarters_surge_m_s', base_speed))
        else:
            target_speed = max(base_speed, getattr(self, '
apf_crossing_min_forward_speed', base_speed))
            if pass_ahead_active:
                pass_ahead_speed = float(getattr(self, '
apf_crossing_pass_ahead_surge_m_s', target_speed))
                target_speed = max(target_speed, pass_ahead_speed)
                tcpa_s = float(getattr(self, 'apf_colreg_tcpa_s', np.nan))
                dcpa_m = float(getattr(self, 'apf_colreg_dcpa_m', np.nan))
                horizon_s = max(float(getattr(self, 'apf_collision_horizon_s', 1.0))
, 1e-06)
                safe_dcpa_m = max(float(getattr(self, '
apf_crossing_pass_ahead_safe_dcpa_m', 0.0)), 1e-06)
                tcpa_urgency = 0.0 if not np.isfinite(tcpa_s) else float(np.clip((
horizon_s - max(tcpa_s, 0.0)) / horizon_s, 0.0, 1.0))
                dcpa_urgency = 0.0 if not np.isfinite(dcpa_m) else float(np.clip((
safe_dcpa_m - max(dcpa_m, 0.0)) / safe_dcpa_m, 0.0, 1.0))
                pass_ahead_urgency = max(tcpa_urgency, dcpa_urgency)
                target_speed = max(target_speed, base_speed + pass_ahead_urgency *
max(pass_ahead_speed - base_speed, 0.0))
            return float(np.clip(target_speed, 0.0, self.v_max))
        if encounter == 'head_on':
            turn_scale = float(np.clip(1.0 - abs(force_angle) / max(self.
apf_heading_step_limit_rad, 1e-06), 0.35, 1.0))
            target_speed = max(getattr(self, 'apf_min_forward_speed', 0.0), getattr(
self, 'apf_head_on_surge_m_s', base_speed) * turn_scale)
            return float(np.clip(target_speed, 0.0, self.v_max))
        if np.isfinite(nearest_active_level) and nearest_active_level <= 1.0:
            target_speed = max(getattr(self, 'apf_min_forward_speed', base_speed),
getattr(self, 'apf_close_quarters_surge_m_s', base_speed))
            return float(np.clip(target_speed, 0.0, self.v_max))
        return float(np.clip(base_speed, 0.0, self.v_max))

def obstacle_track_state_at(self, track, dt_s):
    dt_s = max(float(dt_s), 0.0)
    state = np.asarray(track.get('state', [np.nan] * 4), dtype=float).reshape(4)
    if not np.isfinite(state).all():
        return (None, None)
    return (state[:2] + state[2:4] * dt_s, state[2:4].copy())

def apf_obstacle_in_priority_front_sector(self, obstacle):
    obs_pos_body = np.asarray(obstacle.get('centre_body', [np.nan, np.nan]),
dtype=float).reshape(2)
    if not np.isfinite(obs_pos_body).all():
        return False
    body_angle_rad = float(np.arctan2(obs_pos_body[1], obs_pos_body[0]))
    return abs(wrap_angle(body_angle_rad)) <= self.
apf_priority_front_half_angle_rad

def apf_obstacle_in_forward_half_plane(self, obstacle):

```

```

        obs_pos_body = np.asarray(obstacle.get('centre_body', [np.nan, np.nan]),
dtype=float).reshape(2)
        if not np.isfinite(obs_pos_body).all():
            return False
        return float(obs_pos_body[0]) >= 0.0

def route_progress_and_point(self, lookahead_m=0.0):
    current_ne = np.array([float(self.North), float(self.East)], dtype=float)
    if self.route_path_length_m < 1e-09:
        return (0.0, self.goal_ne.copy())
    along_m = float(np.dot(current_ne - self.start_ne, self.route_path_unit_ne))
    closest_along_m = float(np.clip(along_m, 0.0, self.route_path_length_m))
    target_along_m = float(np.clip(along_m + lookahead_m, 0.0, self.
route_path_length_m))
    target_ne = self.start_ne + target_along_m * self.route_path_unit_ne
    return (closest_along_m, target_ne)

def goal_distance_m(self):
    current_ne = np.array([float(self.North), float(self.East)], dtype=float)
    return float(np.linalg.norm(self.goal_ne - current_ne))

def final_approach_active(self, final_distance_m=None):
    if final_distance_m is None:
        final_distance_m = self.goal_distance_m()
    if not np.isfinite(final_distance_m):
        return False
    if final_distance_m <= self.final_approach_distance_m:
        return True
    if self.route_path_length_m < 1e-09:
        return True
    current_ne = np.array([float(self.North), float(self.East)], dtype=float)
    along_m = float(np.dot(current_ne - self.start_ne, self.route_path_unit_ne))
    return along_m >= self.route_path_length_m - self.final_approach_distance_m

def apf_path_attraction_body(self):
    path_vec = self.goal_ne - self.start_ne
    path_len_sq = float(np.dot(path_vec, path_vec))
    if path_len_sq < 1e-09:
        return np.zeros(2, dtype=float)
    current_ne = np.array([float(self.North), float(self.East)], dtype=float)
    ratio = float(np.clip(np.dot(current_ne - self.start_ne, path_vec) /
path_len_sq, 0.0, 1.0))
    closest_ne = self.start_ne + ratio * path_vec
    to_path_body = self.earth_point_to_body(closest_ne)
    distance_m = float(np.linalg.norm(to_path_body))
    if distance_m < self.apf_path_threshold_m or distance_m < 1e-06:
        return np.zeros(2, dtype=float)
    return self.apf_path_gain * to_path_body

def apf_goal_attraction_body(self, target_body):
    target_body = np.asarray(target_body, dtype=float).reshape(2)
    distance_m = float(np.linalg.norm(target_body))
    if distance_m < 1e-06:
        return np.zeros(2, dtype=float)
    magnitude = self.apf_goal_gain * min(distance_m, self.
apf_attraction_saturation_m)
    return magnitude * target_body / distance_m

```



```

def apf_avoidance_needed(self):
    virtual_obstacles = self.update_apf_virtual_obstacles()
    for obstacle in self.lidar_obstacles + virtual_obstacles:
        obs_pos_body = np.asarray(obstacle.get('centre_body', [np.nan, np.nan]),
dtype=float).reshape(2)
        if not np.isfinite(obs_pos_body).all():
            continue
        if not bool(obstacle.get('virtual', False)) and (not self.
apf_obstacle_in_forward_half_plane(obstacle)):
            continue
        angle_rad = abs(wrap_angle(float(np.arctan2(obs_pos_body[1],
obs_pos_body[0]))))
        if not bool(obstacle.get('virtual', False)) and angle_rad > self.
apf_activation_front_half_angle_rad:
            continue
        if self.apf_repulsion_for_obstacle(obstacle, None, None)[1]:
            return True
    return False

def compute_apf_control(self, t, u_track):
    final_approach = self.final_approach_active()
    if final_approach:
        target_ne = self.goal_ne.copy()
    else:
        _, target_ne = self.route_progress_and_point(self.apf_route_lookahead_m)
    target_body = self.earth_point_to_body(target_ne)
    goal_body = self.earth_point_to_body(self.goal_ne)
    path_force = np.zeros(2, dtype=float) if final_approach else self.
apf_path_attraction_body()
    attractive_force = self.apf_goal_attraction_body(goal_body) + path_force
    force_body = attractive_force.copy()
    repulsive_force = np.zeros(2, dtype=float)
    own_vel_body = self.current_velocity_body()
    any_repulsion = False
    clearance_offset_m = 0.0
    nearest_active_level = np.inf
    avoidance_pc_scale = float(getattr(self, 'apf_avoidance_pc_scale', 2.0))
    self.reset_apf_diagnostics()
    self.apf_target_ne = target_ne.copy()
    self.apf_attractive_force_body = attractive_force
    virtual_obstacles = self.update_apf_virtual_obstacles()
    obstacles = self.lidar_obstacles + virtual_obstacles
    priority_obstacles = []
    secondary_obstacles = []
    for obstacle in obstacles:
        if self.apf_obstacle_in_priority_front_sector(obstacle):
            priority_obstacles.append(obstacle)
        else:
            secondary_obstacles.append(obstacle)
    for obstacle in priority_obstacles:
        repulsion, active = self.apf_repulsion_for_obstacle(obstacle,
target_body, own_vel_body)[:2]
        repulsive_force += repulsion
        force_body += repulsion
        any_repulsion = any_repulsion or active
        if active:
            obstacle_encounter = obstacle.get('encounter_mode', self.
apf_encounter_mode)

```

```

        clearance_offset_m = max(clearance_offset_m, self.
apf_direction_clearance_m(obstacle, obstacle_encounter))
        nearest_active_level = min(nearest_active_level, self.
apf_obstacle_level_and_away(obstacle, avoidance_pc_scale, encounter=
obstacle_encounter)[0])
        if not any_repulsion:
            for obstacle in secondary_obstacles:
                repulsion, active = self.apf_repulsion_for_obstacle(obstacle,
target_body, own_vel_body)[:2]
                repulsive_force += repulsion
                force_body += repulsion
                any_repulsion = any_repulsion or active
                if active:
                    obstacle_encounter = obstacle.get('encounter_mode', self.
apf_encounter_mode)
                    clearance_offset_m = max(clearance_offset_m, self.
apf_direction_clearance_m(obstacle, obstacle_encounter))
                    nearest_active_level = min(nearest_active_level, self.
apf_obstacle_level_and_away(obstacle, avoidance_pc_scale, encounter=
obstacle_encounter)[0])
                    if any_repulsion and self.apf_side_lock_sign != 0.0:
                        route_normal_left_ne = np.array([self.route_path_unit_ne[1], -self.
route_path_unit_ne[0]], dtype=float)
                        offset_target_ne = target_ne + self.apf_side_lock_sign * max(
clearance_offset_m, float(getattr(self, 'obstacle_min_pc2_m', 0.16))) *
route_normal_left_ne
                        offset_target_body = self.earth_point_to_body(offset_target_ne)
                        offset_distance = float(np.linalg.norm(offset_target_body))
                        if offset_distance > 1e-06:
                            offset_force = self.apf_clearance_gain * min(offset_distance, self.
apf_attraction_saturation_m) * offset_target_body / offset_distance
                            force_body += offset_force
                            attractive_force += offset_force
                            self.apf_target_ne = offset_target_ne.copy()
                        if any_repulsion and np.isfinite(nearest_active_level):
                            close_quarters_scale = float(np.clip(nearest_active_level, float(getattr
(self, 'apf_close_quarters_force_scale', 0.15)), 1.0))
                            force_body = repulsive_force + close_quarters_scale * attractive_force
                            self.apf_attractive_force_body = attractive_force
                            force_norm = float(np.linalg.norm(force_body))
                            if not np.isfinite(force_norm) or force_norm < 1e-06:
                                force_body = np.array([0.001, 0.0], dtype=float)
                                force_norm = float(np.linalg.norm(force_body))
                            self.apf_force_body = force_body
                            self.apf_repulsive_force_body = repulsive_force
                            steering_force = force_body.copy()
                            if any_repulsion and steering_force[0] <= 0.0:
                                side_sign = float(np.sign(steering_force[1]))
                                if side_sign == 0.0:
                                    side_sign = self.apf_side_lock_sign
                                if side_sign == 0.0:
                                    if self.left_clearance_m > self.right_clearance_m + 0.05:
                                        side_sign = 1.0
                                    else:
                                        side_sign = -1.0
                                lateral_mag = max(abs(float(steering_force[1])), 0.5 * abs(float(
force_body[0])), 0.2)
                                steering_force[1] = side_sign * lateral_mag

```

```

        steering_force[0] = max(0.25 * lateral_mag, 0.05)
        self.apf_steering_force_body = steering_force
        now_s = float(self.timefromstart) if self.timefromstart is not None else 0.0
        self.apf_visual_hold_until_s = now_s + self.apf_visual_hold_s
        force_angle = wrap_angle(float(np.arctan2(steering_force[1], steering_force
[0])))
        force_angle = float(np.clip(force_angle, -self.apf_heading_step_limit_rad,
self.apf_heading_step_limit_rad))
        if self.apf_encounter_mode == 'head_on':
            force_angle = min(force_angle, -abs(np.deg2rad(8.0)))
        u_cmd = Vector(2)
        u_cmd[1, 0] = np.clip(-self.apf_heading_gain * force_angle / max(self.lastdt
, 0.001), -self.w_max, self.w_max)
        surge_cmd = self.apf_encounter_speed_m_s(self.apf_encounter_mode,
nearest_active_level, force_angle)
        u_cmd[0, 0] = float(np.clip(surge_cmd, 0.0, self.v_max))
        if any_repulsion or self.apf_colreg_active or self.apf_side_lock_active:
            if self.apf_colreg_active:
                self.navigation_mode = 'apf_colreg'
            else:
                self.navigation_mode = 'apf_avoid'
        else:
            self.navigation_mode = 'apf_track'
        return u_cmd

```

A.5 Head-on encounter controller

```

"""
Copyright (c) 2025 The uos_sess6072_build Authors.
Authors: Blair Thornton, Alec O'Loughlin, Miquel Massot
All rights reserved.
Licensed under the BSD 3-Clause License.
See LICENSE.md file in the project root for full license information.
"""

import numpy as np
import time
from colreg_apf import straight_line_cpa
from math_sess6072 import Vector
from math_sess6072 import Vector

def Vector(dim):
    return np.zeros((dim, 1), dtype=float)

def wrap_angle(a):
    return (a + np.pi) % (2 * np.pi) - np.pi

class LaptopController:

    def earth_vector_to_body(self, vector_ne):
        pose = np.asarray(self.p_robot, dtype=float).reshape(-1)
        yaw = pose[2]
        c = np.cos(yaw)
        s = np.sin(yaw)
        vector_ne = np.asarray(vector_ne, dtype=float).reshape(2)
        return np.array([c * vector_ne[0] + s * vector_ne[1], -s * vector_ne[0] + c
* vector_ne[1]])

```

```

def earth_point_to_body(self, point_ne):
    pose = np.asarray(self.p_robot, dtype=float).reshape(-1)
    return self.earth_vector_to_body(np.asarray(point_ne, dtype=float).reshape
(2) - pose[0:2])

def reset_apf_diagnostics(self, clear_visual=True):
    if clear_visual:
        self.apf_force_body = np.zeros(2, dtype=float)
        self.apf_repulsive_force_body = np.zeros(2, dtype=float)
        self.apf_attractive_force_body = np.zeros(2, dtype=float)
        self.apf_steering_force_body = np.zeros(2, dtype=float)
        self.apf_target_ne = np.array([np.nan, np.nan], dtype=float)
        self.apf_virtual_obstacles = []
        self.apf_visual_hold_until_s = 0.0
    self.apf_active_profile_name = 'crossing'
    self.apf_encounter_mode = 'none'
    self.apf_colreg_rule = 'none'
    self.apf_avoidance_side_sign = 0.0
    self.apf_colreg_dcpa_m = np.nan
    self.apf_colreg_tcpa_s = np.nan
    self.apf_colreg_active = False

def current_velocity_body(self):
    vel_ne = np.asarray(self.v_robot[0:2], dtype=float).reshape(2)
    vel_body = self.earth_vector_to_body(vel_ne)
    if not np.isfinite(vel_body).all():
        return np.zeros(2, dtype=float)
    return vel_body

def obstacle_ekf_process_noise(self, dt):
    dt = max(float(dt), 0.001)
    q = self.obstacle_ekf_accel_std_m_s2 ** 2
    dt2 = dt * dt
    dt3 = dt2 * dt
    dt4 = dt2 * dt2
    return q * np.array([[0.25 * dt4, 0.0, 0.5 * dt3, 0.0], [0.0, 0.25 * dt4,
0.0, 0.5 * dt3], [0.5 * dt3, 0.0, dt2, 0.0], [0.0, 0.5 * dt3, 0.0, dt2]], dtype=
float)

def obstacle_ekf_predict(self, state, covariance, dt):
    dt = max(float(dt), 0.0)
    state = np.asarray(state, dtype=float).reshape(4)
    covariance = np.asarray(covariance, dtype=float).reshape(4, 4)
    F = np.array([[1.0, 0.0, dt, 0.0], [0.0, 1.0, 0.0, dt], [0.0, 0.0, 1.0,
0.0], [0.0, 0.0, 0.0, 1.0]], dtype=float)
    predicted_state = F @ state
    predicted_covariance = F @ covariance @ F.T + self.
obstacle_ekf_process_noise(dt)
    return (predicted_state, predicted_covariance)

def obstacle_track_prediction_accel_ne(self, track):
    if not bool(track.get('motion_stable', False)) or int(track.get('
stats_sample_count', 0)) < self.obstacle_prediction_accel_min_samples:
        return np.zeros(2, dtype=float)
    accel_ne = np.asarray(track.get('accel_ne', [0.0, 0.0]), dtype=float).
reshape(2)

```

```

        accel_var_ne = np.asarray(track.get('accel_var_ne', [np.inf, np.inf]), dtype=
=float).reshape(2)
        if not np.isfinite(accel_ne).all() or not np.isfinite(accel_var_ne).all():
            return np.zeros(2, dtype=float)
        accel_std_m_s2 = float(np.sqrt(max(float(np.max(accel_var_ne)), 0.0)))
        if accel_std_m_s2 > self.obstacle_prediction_max_accel_std_m_s2:
            return np.zeros(2, dtype=float)
        return accel_ne

def apf_track_for_obstacle(self, obstacle):
    if not self.obstacle_ekf_prediction_enabled:
        return None
    centre_ne = np.asarray(obstacle.get('centre_ne', [np.nan, np.nan]), dtype=
float).reshape(2)
    if not np.isfinite(centre_ne).all():
        return None
    now = float(self.latest_lidar_received_s if self.latest_lidar_received_s is
not None else time.time())
    best_track = None
    best_distance = np.inf
    for track in self.apf_obstacle_tracks:
        if now - float(track.get('last_seen_s', track.get('stamp_s', now))) >
self.apf_track_timeout_s:
            continue
        predicted_state, _ = self.obstacle_ekf_predict(track.get('state', np.r_[
track.get('pos_ne', [np.nan, np.nan]), track.get('vel_ne', [0.0, 0.0]])], track.
get('covariance', np.eye(4, dtype=float)), max(now - float(track.get('stamp_s',
now)), 0.0))
        predicted_pos = predicted_state[0:2]
        distance = float(np.linalg.norm(centre_ne - predicted_pos))
        if distance < best_distance:
            best_distance = distance
            best_track = track
    if best_distance <= max(self.apf_track_association_m * 1.5, self.
lidar_dbscan_eps_m * 2.0):
        return best_track
    return None

def apf_build_encounter_params(self, **overrides):
    params = {'cluster_range_enabled': bool(self.apf_cluster_range_enabled), '
risk_pc_scale': float(self.apf_risk_pc_scale), 'avoidance_pc_scale': float(self.
apf_avoidance_pc_scale), 'direction_pc_scale': float(self.apf_direction_pc_scale
), 'virtual_pc_scale': float(self.apf_virtual_pc_scale), 'repulsive_gain': float
(self.apf_repulsive_gain), 'route_lookahead_m': float(self.apf_route_lookahead_m
), 'collision_horizon_s': float(self.apf_collision_horizon_s), '
constant_descent_speed_m_s': float(self.apf_constant_descent_speed_m_s), '
dynamic_speed_threshold_m_s': float(self.apf_dynamic_speed_threshold_m_s)}
    params.update(overrides)
    return params

def apf_profile_name_for_encounter(self, encounter):
    if encounter == 'overtaking':
        return 'overtaking'
    if encounter == 'head_on':
        return 'head_on'
    return 'crossing'

def apf_params_for_encounter(self, encounter=None):

```

```

        profile_name = self.apf_profile_name_for_encounter(self.apf_encounter_mode
if encounter is None else encounter)
        if profile_name == 'overtaking':
            return self.apf_overtaking_params
        if profile_name == 'head_on':
            return self.apf_head_on_params
        return self.apf_crossing_params

def apf_cpa_metrics(self, obs_pos_body, obs_vel_body, own_vel_body):
    if not self.obstacle_ekf_prediction_enabled:
        return (np.nan, np.nan)
    tcpa_s, dcpa_m, _, _ = straight_line_cpa([0.0, 0.0], own_vel_body,
obs_pos_body, obs_vel_body)
    return (tcpa_s, dcpa_m)

def own_prediction_velocity_ne(self):
    current_ne = np.array([float(self.North), float(self.East)], dtype=float)
    to_goal_ne = self.goal_ne - current_ne
    to_goal_distance_m = float(np.linalg.norm(to_goal_ne))
    route_direction_ne = to_goal_ne / to_goal_distance_m if self.
final_approach_active(to_goal_distance_m) and to_goal_distance_m >= 1e-06 else
self.route_path_unit_ne
    return route_direction_ne * max(float(self.route_tracking_speed_m_s), self.
apf_min_forward_speed)

def obstacle_track_state_at(self, track, dt_s):
    dt_s = max(float(dt_s), 0.0)
    state = np.asarray(track.get('state', [np.nan, np.nan, 0.0, 0.0]), dtype=
float).reshape(4)
    if not np.isfinite(state).all():
        return (None, None)
    velocity_ne = state[2:4].copy()
    accel_ne = self.obstacle_track_prediction_accel_ne(track)
    pos_ne = state[0:2] + velocity_ne * dt_s + 0.5 * accel_ne * dt_s * dt_s
    vel_ne = velocity_ne + accel_ne * dt_s
    return (pos_ne, vel_ne)

def virtual_collision_visuals(self):
    visuals = []
    for obstacle in self.apf_virtual_obstacles:
        if not bool(obstacle.get('predicted_risk_active', False)):
            continue
        collision_position_ne = np.asarray(obstacle.get('collision_position_ne',
[np.nan, np.nan]), dtype=float).reshape(2)
        if not np.isfinite(collision_position_ne).all():
            continue
        visuals.append({'track_id': int(obstacle.get('track_id', 0)), '
collision_position_ne': collision_position_ne.copy(), 'own_prediction_ne': np.
asarray(obstacle.get('own_prediction_ne', [np.nan, np.nan]), dtype=float).
reshape(2), 'tcpa_s': float(obstacle.get('tcpa_s', np.nan)), '
predicted_separation_m': float(obstacle.get('dcpa_m', np.nan)), 'collision_level'
: float(obstacle.get('collision_level', np.nan))})
    return visuals

def apf_default_side_from_obstacle(self, obs_pos_body):
    body_angle_rad = float(np.arctan2(obs_pos_body[1], obs_pos_body[0]))
    if abs(wrap_angle(body_angle_rad)) > self.
apf_activation_front_half_angle_rad:

```

```

        return 0.0
    if abs(body_angle_rad) <= np.deg2rad(5.0):
        if self.left_clearance_m > self.right_clearance_m + 0.05:
            return 1.0
        if self.right_clearance_m > self.left_clearance_m + 0.05:
            return -1.0
        return -1.0
    return -1.0 if body_angle_rad > 0.0 else 1.0

def apf_obstacle_in_priority_front_sector(self, obstacle):
    obs_pos_body = np.asarray(obstacle.get('centre_body', [np.nan, np.nan]),
dtype=float).reshape(2)
    if not np.isfinite(obs_pos_body).all():
        return False
    body_angle_rad = float(np.arctan2(obs_pos_body[1], obs_pos_body[0]))
    return abs(wrap_angle(body_angle_rad)) <= self.
apf_priority_front_half_angle_rad

def apf_lock_side(self, requested_side, obstacle_level):
    now_s = float(self.timefromstart) if self.timefromstart is not None else 0.0
    obstacle_close = np.isfinite(obstacle_level) and obstacle_level <= self.
apf_side_lock_exit_level
    if requested_side == 0.0:
        if self.apf_side_lock_sign != 0.0 and (obstacle_close or now_s < self.
apf_side_lock_until_s):
            self.apf_side_lock_active = True
            return self.apf_side_lock_sign
        self.apf_side_lock_sign = 0.0
        self.apf_side_lock_active = False
        return 0.0
    requested_side = float(np.sign(requested_side))
    if self.apf_side_lock_sign != 0.0 and (obstacle_close or now_s < self.
apf_side_lock_until_s):
        self.apf_side_lock_active = True
        return self.apf_side_lock_sign
    self.apf_side_lock_sign = requested_side
    self.apf_side_lock_until_s = now_s + self.apf_side_lock_s
    self.apf_side_lock_active = True
    return self.apf_side_lock_sign

def refresh_apf_side_lock(self, nearest_forward_level=np.inf):
    if self.apf_side_lock_sign == 0.0:
        self.apf_side_lock_active = False
        return False
    now_s = float(self.timefromstart) if self.timefromstart is not None else 0.0
    obstacle_close = np.isfinite(nearest_forward_level) and
nearest_forward_level <= self.apf_side_lock_exit_level
    if obstacle_close or now_s < self.apf_side_lock_until_s:
        self.apf_side_lock_active = True
        return True
    self.apf_side_lock_sign = 0.0
    self.apf_side_lock_active = False
    return False

def route_progress_and_point(self, lookahead_m=0.0):
    current_ne = np.array([float(self.North), float(self.East)], dtype=float)
    if self.route_path_length_m < 1e-09:
        return (0.0, self.goal_ne.copy())

```



```

        along_m = float(np.dot(current_ne - self.start_ne, self.route_path_unit_ne))
        closest_along_m = float(np.clip(along_m, 0.0, self.route_path_length_m))
        target_along_m = float(np.clip(along_m + lookahead_m, 0.0, self.
route_path_length_m))
        target_ne = self.start_ne + target_along_m * self.route_path_unit_ne
        return (closest_along_m, target_ne)

def goal_distance_m(self):
    current_ne = np.array([float(self.North), float(self.East)], dtype=float)
    return float(np.linalg.norm(self.goal_ne - current_ne))

def final_approach_active(self, final_distance_m=None):
    if final_distance_m is None:
        final_distance_m = self.goal_distance_m()
    if not np.isfinite(final_distance_m):
        return False
    if final_distance_m <= self.final_approach_distance_m:
        return True
    if self.route_path_length_m < 1e-09:
        return True
    current_ne = np.array([float(self.North), float(self.East)], dtype=float)
    along_m = float(np.dot(current_ne - self.start_ne, self.route_path_unit_ne))
    return along_m >= self.route_path_length_m - self.final_approach_distance_m

def apf_path_attraction_body(self):
    path_vec = self.goal_ne - self.start_ne
    path_len_sq = float(np.dot(path_vec, path_vec))
    if path_len_sq < 1e-09:
        return np.zeros(2, dtype=float)
    current_ne = np.array([float(self.North), float(self.East)], dtype=float)
    ratio = float(np.clip(np.dot(current_ne - self.start_ne, path_vec) /
path_len_sq, 0.0, 1.0))
    closest_ne = self.start_ne + ratio * path_vec
    to_path_body = self.earth_point_to_body(closest_ne)
    distance_m = float(np.linalg.norm(to_path_body))
    if distance_m < self.apf_path_threshold_m or distance_m < 1e-06:
        return np.zeros(2, dtype=float)
    return self.apf_path_gain * to_path_body

def apf_goal_attraction_body(self, target_body):
    target_body = np.asarray(target_body, dtype=float).reshape(2)
    distance_m = float(np.linalg.norm(target_body))
    if distance_m < 1e-06:
        return np.zeros(2, dtype=float)
    magnitude = self.apf_goal_gain * min(distance_m, self.
apf_attraction_saturation_m)
    return magnitude * target_body / distance_m

def apf_avoidance_needed(self):
    virtual_obstacles = self.update_apf_virtual_obstacles()
    for obstacle in self.lidar_obstacles + virtual_obstacles:
        obs_pos_body = np.asarray(obstacle.get('centre_body', [np.nan, np.nan]),
dtype=float).reshape(2)
        if not np.isfinite(obs_pos_body).all():
            continue
        angle_rad = abs(wrap_angle(float(np.arctan2(obs_pos_body[1],
obs_pos_body[0]))))

```



```

        if not bool(obstacle.get('virtual', False)) and angle_rad > self.
apf_activation_front_half_angle_rad:
            continue
            if self.apf_repulsion_for_obstacle(obstacle, None, None)[1]:
                return True
            return False

def compute_apf_control(self, t, u_track):
    active_params = self.apf_head_on_params
    final_approach = self.final_approach_active()
    if final_approach:
        target_ne = self.goal_ne.copy()
    else:
        _, target_ne = self.route_progress_and_point(float(active_params.get('
route_lookahead_m', self.apf_route_lookahead_m)))
        target_body = self.earth_point_to_body(target_ne)
        goal_body = self.earth_point_to_body(self.goal_ne)
        path_force = np.zeros(2, dtype=float) if final_approach else self.
apf_path_attraction_body()
        attractive_force = self.apf_goal_attraction_body(goal_body) + path_force
        force_body = attractive_force.copy()
        repulsive_force = np.zeros(2, dtype=float)
        own_vel_body = self.current_velocity_body()
        any_repulsion = False
        clearance_offset_m = 0.0
        self.reset_apf_diagnostics()
        self.apf_target_ne = target_ne.copy()
        self.apf_attractive_force_body = attractive_force
        virtual_obstacles = self.update_apf_virtual_obstacles()
        obstacles = self.lidar_obstacles + virtual_obstacles
        priority_obstacles = []
        secondary_obstacles = []
        for obstacle in obstacles:
            if self.apf_obstacle_in_priority_front_sector(obstacle):
                priority_obstacles.append(obstacle)
            else:
                secondary_obstacles.append(obstacle)
        for obstacle in priority_obstacles:
            repulsion, active, obstacle_params = self.apf_repulsion_for_obstacle(
obstacle, target_body, own_vel_body)
            repulsive_force += repulsion
            force_body += repulsion
            any_repulsion = any_repulsion or active
            if active:
                active_params = obstacle_params
                clearance_offset_m = max(clearance_offset_m, self.
apf_direction_clearance_m(obstacle, params=obstacle_params))
            if not any_repulsion:
                for obstacle in secondary_obstacles:
                    repulsion, active, obstacle_params = self.apf_repulsion_for_obstacle
(obstacle, target_body, own_vel_body)
                    repulsive_force += repulsion
                    force_body += repulsion
                    any_repulsion = any_repulsion or active
                    if active:
                        active_params = obstacle_params
                        clearance_offset_m = max(clearance_offset_m, self.
apf_direction_clearance_m(obstacle, params=obstacle_params))

```

```

        if any_repulsion and self.apf_side_lock_sign != 0.0:
            route_normal_left_ne = np.array([self.route_path_unit_ne[1], -self.
route_path_unit_ne[0]], dtype=float)
            offset_target_ne = target_ne + self.apf_side_lock_sign * max(
clearance_offset_m, self.obstacle_min_pc2_m) * route_normal_left_ne
            offset_target_body = self.earth_point_to_body(offset_target_ne)
            offset_distance = float(np.linalg.norm(offset_target_body))
            if offset_distance > 1e-06:
                offset_force = self.apf_clearance_gain * min(offset_distance, self.
apf_attraction_saturation_m) * offset_target_body / offset_distance
                force_body += offset_force
                attractive_force += offset_force
                self.apf_target_ne = offset_target_ne.copy()
            self.apf_attractive_force_body = attractive_force
            force_norm = float(np.linalg.norm(force_body))
            if not np.isfinite(force_norm) or force_norm < 1e-06:
                force_body = np.array([0.001, 0.0], dtype=float)
                force_norm = float(np.linalg.norm(force_body))
            self.apf_force_body = force_body
            self.apf_repulsive_force_body = repulsive_force
            steering_force = force_body.copy()
            if any_repulsion and steering_force[0] <= 0.0:
                side_sign = float(np.sign(steering_force[1]))
                if side_sign == 0.0:
                    side_sign = self.apf_side_lock_sign
                if side_sign == 0.0:
                    if self.left_clearance_m > self.right_clearance_m + 0.05:
                        side_sign = 1.0
                    else:
                        side_sign = -1.0
                lateral_mag = max(abs(float(steering_force[1])), 0.5 * abs(float(
force_body[0])), 0.2)
                steering_force[1] = side_sign * lateral_mag
                steering_force[0] = max(0.25 * lateral_mag, 0.05)
            self.apf_steering_force_body = steering_force
            now_s = float(self.timefromstart) if self.timefromstart is not None else 0.0
            self.apf_visual_hold_until_s = now_s + self.apf_visual_hold_s
            force_angle = wrap_angle(float(np.arctan2(steering_force[1], steering_force
[0])))
            force_angle = float(np.clip(force_angle, -self.apf_heading_step_limit_rad,
self.apf_heading_step_limit_rad))
            u_cmd = Vector(2)
            u_cmd[1, 0] = np.clip(-self.apf_heading_gain * force_angle / max(self.lastdt
, 0.001), -self.w_max, self.w_max)
            u_cmd[0, 0] = float(np.clip(float(active_params.get('
constant_descent_speed_m_s', self.apf_constant_descent_speed_m_s)), 0.0, self.
v_max))
            if any_repulsion or self.apf_colreg_active or self.apf_side_lock_active:
                if self.apf_colreg_active:
                    self.navigation_mode = 'apf_colreg'
                else:
                    self.navigation_mode = 'apf_avoid'
            else:
                self.navigation_mode = 'apf_track'
        return u_cmd

```

A.6 Overtaking encounter controller

```
"""
Copyright (c) 2025 The uos_sess6072_build Authors.
Authors: Blair Thornton, Alec O'Loughlin, Miquel Massot
All rights reserved.
Licensed under the BSD 3-Clause License.
See LICENSE.md file in the project root for full license information.
"""

import numpy as np
import time
from colreg_apf import constrain_velocity_to_axis, smooth_undirected_axis,
    straight_line_cpa
from math_sess6072 import Vector
from math_sess6072 import Vector
G = 2

def Vector(dim):
    return np.zeros((dim, 1), dtype=float)

def wrap_angle(a):
    return (a + np.pi) % (2 * np.pi) - np.pi

class LaptopController:

    def earth_vector_to_body(self, vector_ne):
        pose = np.asarray(self.p_robot, dtype=float).reshape(-1)
        yaw = pose[2]
        c = np.cos(yaw)
        s = np.sin(yaw)
        vector_ne = np.asarray(vector_ne, dtype=float).reshape(2)
        return np.array([c * vector_ne[0] + s * vector_ne[1], -s * vector_ne[0] + c
            * vector_ne[1]])

    def earth_point_to_body(self, point_ne):
        pose = np.asarray(self.p_robot, dtype=float).reshape(-1)
        return self.earth_vector_to_body(np.asarray(point_ne, dtype=float).reshape
            (2) - pose[0:2])

    def reset_apf_diagnostics(self, clear_visual=True):
        if clear_visual:
            self.apf_force_body = np.zeros(2, dtype=float)
            self.apf_repulsive_force_body = np.zeros(2, dtype=float)
            self.apf_attractive_force_body = np.zeros(2, dtype=float)
            self.apf_steering_force_body = np.zeros(2, dtype=float)
            self.apf_target_ne = np.array([np.nan, np.nan], dtype=float)
            self.apf_virtual_obstacles = []
            self.apf_visual_hold_until_s = 0.0
        self.apf_active_profile_name = 'crossing'
        self.apf_encounter_mode = 'none'
        self.apf_colreg_rule = 'none'
        self.apf_avoidance_side_sign = 0.0
        self.apf_colreg_dcpa_m = np.nan
        self.apf_colreg_tcpa_s = np.nan
        self.apf_colreg_active = False

    def apf_visual_hold_active(self):
        now_s = float(self.timefromstart) if self.timefromstart is not None else 0.0
```

```

        return now_s < self.apf_visual_hold_until_s

def current_velocity_body(self):
    vel_ne = np.asarray(self.v_robot[0:2], dtype=float).reshape(2)
    vel_body = self.earth_vector_to_body(vel_ne)
    if not np.isfinite(vel_body).all():
        return np.zeros(2, dtype=float)
    return vel_body

def obstacle_ekf_process_noise(self, dt):
    dt = max(float(dt), 0.001)
    q = self.obstacle_ekf_accel_std_m_s2 ** 2
    dt2 = dt * dt
    dt3 = dt2 * dt
    dt4 = dt2 * dt2
    return q * np.array([[0.25 * dt4, 0.0, 0.5 * dt3, 0.0], [0.0, 0.25 * dt4,
0.0, 0.5 * dt3], [0.5 * dt3, 0.0, dt2, 0.0], [0.0, 0.5 * dt3, 0.0, dt2]], dtype=
float)

def obstacle_ekf_predict(self, state, covariance, dt):
    dt = max(float(dt), 0.0)
    state = np.asarray(state, dtype=float).reshape(4)
    covariance = np.asarray(covariance, dtype=float).reshape(4, 4)
    F = np.array([[1.0, 0.0, dt, 0.0], [0.0, 1.0, 0.0, dt], [0.0, 0.0, 1.0,
0.0], [0.0, 0.0, 0.0, 1.0]], dtype=float)
    predicted_state = F @ state
    predicted_covariance = F @ covariance @ F.T + self.
obstacle_ekf_process_noise(dt)
    return (predicted_state, predicted_covariance)

def obstacle_measurement_covariance(self, measurement_ne, reference_pose=None):
    measurement_ne = np.asarray(measurement_ne, dtype=float).reshape(2)
    if reference_pose is None:
        reference_pose = getattr(self, 'lidar_reference_pose', None)
    if reference_pose is None:
        reference_pose = np.asarray(self.p_robot, dtype=float).reshape(3)
    else:
        reference_pose = np.asarray(reference_pose, dtype=float).reshape(3)
    base_variance = float(self.obstacle_ekf_measurement_std_m) ** 2
    yaw_variance = 0.0
    sigma = getattr(self, 'Sigma', None)
    if sigma is not None:
        sigma = np.asarray(sigma, dtype=float)
        if sigma.ndim == 2 and sigma.shape[0] > G and (sigma.shape[1] > G):
            yaw_variance = max(float(sigma[G, G]), 0.0)
    yaw_rate_rad_s = getattr(self, 'sensed_imu_yaw_rate_rad_s', None)
    if yaw_rate_rad_s is None or not np.isfinite(yaw_rate_rad_s):
        velocity = np.asarray(getattr(self, 'v_robot', np.zeros((3, 1))), dtype=
float).reshape(-1)
        yaw_rate_rad_s = velocity[2] if len(velocity) > 2 else 0.0
    timing_std_s = max(float(getattr(self, 'lidar_time_sync_std_s', 0.02)),
float(getattr(self, 'lidar_scan_time_s', 0.0)) / np.sqrt(12.0))
    yaw_variance += (float(yaw_rate_rad_s) * timing_std_s) ** 2
    relative_ne = measurement_ne - reference_pose[0:2]
    yaw_jacobian = np.array([-relative_ne[1], relative_ne[0]], dtype=float)
    return base_variance * np.eye(2, dtype=float) + yaw_variance * np.outer(
yaw_jacobian, yaw_jacobian)

```

```

def obstacle_ekf_update(self, state, covariance, measurement_ne,
measurement_covariance=None):
    state = np.asarray(state, dtype=float).reshape(4)
    covariance = np.asarray(covariance, dtype=float).reshape(4, 4)
    measurement_ne = np.asarray(measurement_ne, dtype=float).reshape(2)
    H = np.array([[1.0, 0.0, 0.0, 0.0], [0.0, 1.0, 0.0, 0.0]], dtype=float)
    R_obs = self.obstacle_measurement_covariance(measurement_ne) if
measurement_covariance is None else np.asarray(measurement_covariance, dtype=
float).reshape(2, 2)
    innovation = measurement_ne - H @ state
    innovation_covariance = H @ covariance @ H.T + R_obs
    try:
        kalman_gain = covariance @ H.T @ np.linalg.inv(innovation_covariance)
    except np.linalg.LinAlgError:
        kalman_gain = covariance @ H.T @ np.linalg.pinv(innovation_covariance)
    corrected_state = state + kalman_gain @ innovation
    identity = np.eye(4, dtype=float)
    correction = identity - kalman_gain @ H
    corrected_covariance = correction @ covariance @ correction.T + kalman_gain
@ R_obs @ kalman_gain.T
    return (corrected_state, corrected_covariance)

def obstacle_track_motion_is_stable(self, track):
    sample_count = int(track.get('stats_sample_count', 0))
    hit_count = int(track.get('hit_count', 0))
    if sample_count < self.obstacle_prediction_min_samples or hit_count < self.
obstacle_prediction_min_hits or int(track.get('miss_count', 0)) > 0:
        return False
    samples = track.get('motion_window', [])
    if len(samples) < 2:
        return False
    first_stamp = float(samples[0].get('stamp_s', 0.0))
    last_stamp = float(samples[-1].get('stamp_s', first_stamp))
    if last_stamp - first_stamp < self.obstacle_prediction_min_time_span_s:
        return False
    speed_m_s = float(track.get('speed_mean_m_s', 0.0))
    was_stable = bool(track.get('motion_stable', False))
    speed_threshold = self.apf_dynamic_exit_speed_threshold_m_s if was_stable
else self.apf_dynamic_speed_threshold_m_s
    if not np.isfinite(speed_m_s) or speed_m_s < speed_threshold:
        return False
    return True

def obstacle_track_prediction_accel_ne(self, track):
    if not bool(track.get('motion_stable', False)) or int(track.get('
stats_sample_count', 0)) < self.obstacle_prediction_accel_min_samples:
        return np.zeros(2, dtype=float)
    accel_ne = np.asarray(track.get('accel_ne', [0.0, 0.0]), dtype=float).
reshape(2)
    accel_var_ne = np.asarray(track.get('accel_var_ne', [np.inf, np.inf]), dtype
=float).reshape(2)
    if not np.isfinite(accel_ne).all() or not np.isfinite(accel_var_ne).all():
        return np.zeros(2, dtype=float)
    accel_std_m_s2 = float(np.sqrt(max(float(np.max(accel_var_ne)), 0.0)))
    if accel_std_m_s2 > self.obstacle_prediction_max_accel_std_m_s2:
        return np.zeros(2, dtype=float)
    return accel_ne

```

```

def obstacle_track_prediction_ne(self, track):
    if not self.obstacle_ekf_prediction_enabled:
        return np.empty((0, 2), dtype=float)
    state = np.asarray(track.get('state', [np.nan, np.nan, 0.0, 0.0]), dtype=
float).reshape(4)
    if not np.isfinite(state).all():
        return np.empty((0, 2), dtype=float)
    velocity_ne = state[2:4].copy()
    accel_ne = self.obstacle_track_prediction_accel_ne(track)
    step_s = max(float(self.obstacle_prediction_step_s), 0.001)
    horizon_s = float(self.obstacle_prediction_horizon_s)
    if not np.isfinite(horizon_s) or horizon_s <= 0.0:
        return np.empty((0, 2), dtype=float)
    times = np.arange(0.0, horizon_s, step_s, dtype=float)
    times = np.r_[times, horizon_s]
    return np.column_stack([state[0] + velocity_ne[0] * times + 0.5 * accel_ne
[0] * times * times, state[1] + velocity_ne[1] * times + 0.5 * accel_ne[1] *
times * times])

def sync_obstacle_track_fields(self, track):
    state = np.asarray(track.get('state', [np.nan, np.nan, 0.0, 0.0]), dtype=
float).reshape(4)
    track['pos_ne'] = state[0:2].copy()
    track['vel_ne'] = state[2:4].copy()
    pc1_m = float(track.get('pc1_m', self.obstacle_min_pc1_m))
    pc2_m = float(track.get('pc2_m', self.obstacle_min_pc2_m))
    if not np.isfinite(pc1_m) or pc1_m <= 0.0:
        pc1_m = self.obstacle_min_pc1_m
    if not np.isfinite(pc2_m) or pc2_m <= 0.0:
        pc2_m = self.obstacle_min_pc2_m
    track['pc1_m'] = max(pc1_m, self.obstacle_min_pc1_m)
    track['pc2_m'] = max(min(pc2_m, track['pc1_m']), self.obstacle_min_pc2_m)
    track['radius_m'] = 0.5 * track['pc1_m']
    track['equivalent_radius_m'] = track['radius_m']
    length_axis_ne = np.asarray(track.get('length_axis_ne', [1.0, 0.0]), dtype=
float).reshape(2)
    axis_norm = float(np.linalg.norm(length_axis_ne))
    track['length_axis_ne'] = length_axis_ne / axis_norm if np.isfinite(
length_axis_ne).all() and axis_norm >= 1e-06 else np.array([1.0, 0.0], dtype=
float)
    if bool(track.get('motion_stable', False)) and track['pc1_m'] >= self.
obstacle_axis_min_aspect_ratio * track['pc2_m']:
        state[2:4] = constrain_velocity_to_axis(state[2:4], track['
length_axis_ne'], self.obstacle_axis_max_velocity_gap_rad)
        track['state'] = state
        track['vel_ne'] = state[2:4].copy()
        ekf_velocity_ne = track['vel_ne'].copy()
        speed_m_s = float(np.linalg.norm(ekf_velocity_ne))
        track['speed_m_s'] = speed_m_s
        heading_hold_speed_m_s = float(getattr(self, '
obstacle_heading_hold_speed_m_s', 0.03))
        if speed_m_s >= heading_hold_speed_m_s:
            heading_rad = float(np.arctan2(ekf_velocity_ne[1], ekf_velocity_ne[0]))
            track['heading_rad'] = heading_rad
            track['heading_deg'] = float(np.rad2deg(heading_rad))
            track['heading_axis_ne'] = ekf_velocity_ne / speed_m_s
        else:
            previous_heading_rad = float(track.get('heading_rad', np.nan))

```

```

        if not np.isfinite(previous_heading_rad):
            previous_heading_rad = float(np.arctan2(track['length_axis_ne'][1],
track['length_axis_ne'][0]))
            track['heading_rad'] = previous_heading_rad
            track['heading_deg'] = float(np.rad2deg(previous_heading_rad))
            track['heading_axis_ne'] = np.array([np.cos(previous_heading_rad), np.
sin(previous_heading_rad)])
        if not self.obstacle_ekf_prediction_enabled:
            track['prediction_model'] = 'disabled'
            track['prediction_ne'] = np.empty((0, 2), dtype=float)
            return
        if np.linalg.norm(self.obstacle_track_prediction_accel_ne(track)) > 0.0:
            track['prediction_model'] = 'sliding_window_acceleration'
        elif bool(track.get('motion_stable', False)):
            track['prediction_model'] = 'stable_constant_velocity'
        else:
            track['prediction_model'] = 'ekf_constant_velocity'
        track['prediction_ne'] = self.obstacle_track_prediction_ne(track)

def make_obstacle_track(self, detection_ne, stamp_s, pc1_m=np.nan, pc2_m=np.nan,
length_axis_ne=None):
    detection_ne = np.asarray(detection_ne, dtype=float).reshape(2)
    pc1_m = float(pc1_m) if np.isfinite(pc1_m) and pc1_m > 0.0 else self.
obstacle_min_pc1_m
    pc2_m = float(pc2_m) if np.isfinite(pc2_m) and pc2_m > 0.0 else self.
obstacle_min_pc2_m
    length_axis_ne = np.asarray([1.0, 0.0] if length_axis_ne is None else
length_axis_ne, dtype=float).reshape(2)
    covariance = np.diag([self.obstacle_ekf_initial_position_std_m ** 2, self.
obstacle_ekf_initial_position_std_m ** 2, self.
obstacle_ekf_initial_velocity_std_m_s ** 2, self.
obstacle_ekf_initial_velocity_std_m_s ** 2])
    track = {'id': self.apf_next_track_id, 'state': np.array([detection_ne[0],
detection_ne[1], 0.0, 0.0], dtype=float), 'covariance': covariance, 'stamp_s':
float(stamp_s), 'last_seen_s': float(stamp_s), 'hit_count': 1, 'miss_count': 0, '
history_ne': [], 'lidar_history_ne': [], 'motion_window': [], 'pc1_m': pc1_m, '
pc2_m': pc2_m, 'length_axis_ne': length_axis_ne}
    self.sync_obstacle_track_fields(track)
    self.append_obstacle_track_history(track, pc1_m=pc1_m, pc2_m=pc2_m,
length_axis_ne=length_axis_ne, detection_ne=detection_ne, stamp_s=stamp_s)
    self.apf_next_track_id += 1
    return track

def make_obstacle_track_candidate(self, detection_ne, stamp_s, pc1_m=np.nan,
pc2_m=np.nan, length_axis_ne=None):
    detection_ne = np.asarray(detection_ne, dtype=float).reshape(2)
    length_axis_ne = np.asarray([1.0, 0.0] if length_axis_ne is None else
length_axis_ne, dtype=float).reshape(2)
    pc1_m = float(pc1_m) if np.isfinite(pc1_m) and pc1_m > 0.0 else self.
obstacle_min_pc1_m
    pc2_m = float(pc2_m) if np.isfinite(pc2_m) and pc2_m > 0.0 else self.
obstacle_min_pc2_m
    return {'centre_ne': detection_ne.copy(), 'stamp_s': float(stamp_s), '
last_seen_s': float(stamp_s), 'hit_count': 1, 'pc1_m': pc1_m, 'pc2_m': pc2_m, '
length_axis_ne': length_axis_ne}

def predict_obstacle_track_to_time(self, track, stamp_s):
    now = float(stamp_s)

```



```

        dt = max(now - float(track.get('stamp_s', now)), 0.0)
        state, covariance = self.obstacle_ekf_predict(track.get('state', np.r_[track.
get('pos_ne', [np.nan, np.nan]), track.get('vel_ne', [0.0, 0.0])]), track.get('
covariance', np.eye(4, dtype=float)), dt)
        track['state'] = state
        track['covariance'] = covariance
        track['stamp_s'] = now
        self.sync_obstacle_track_fields(track)

def append_obstacle_track_history(self, track, pc1_m=np.nan, pc2_m=np.nan,
length_axis_ne=None, detection_ne=None, stamp_s=None):
    stamp_s = float(stamp_s if stamp_s is not None else track.get('stamp_s',
time.time()))
    pos_ne = np.asarray(track['pos_ne'], dtype=float).reshape(2).copy()
    vel_ne = np.asarray(track.get('vel_ne', [0.0, 0.0]), dtype=float).reshape(2).
copy()
    if not np.isfinite(vel_ne).all():
        vel_ne = np.zeros(2, dtype=float)
    pc1_m = float(pc1_m) if np.isfinite(pc1_m) and pc1_m > 0.0 else float(track.
get('pc1_m', self.obstacle_min_pc1_m))
    pc2_m = float(pc2_m) if np.isfinite(pc2_m) and pc2_m > 0.0 else float(track.
get('pc2_m', self.obstacle_min_pc2_m))
    track['pc1_m'] = max(pc1_m, self.obstacle_min_pc1_m)
    track['pc2_m'] = max(min(pc2_m, track['pc1_m']), self.obstacle_min_pc2_m)
    if length_axis_ne is not None:
        length_axis_ne = np.asarray(length_axis_ne, dtype=float).reshape(2)
        axis_norm = float(np.linalg.norm(length_axis_ne))
        if np.isfinite(length_axis_ne).all() and axis_norm >= 1e-06:
            length_axis_ne = length_axis_ne / axis_norm
            previous_axis = np.asarray(track.get('length_axis_ne',
length_axis_ne), dtype=float).reshape(2)
            if float(np.dot(length_axis_ne, previous_axis)) < 0.0:
                length_axis_ne = -length_axis_ne
            track['length_axis_ne'] = smooth_undirected_axis(previous_axis,
length_axis_ne, self.obstacle_axis_smoothing_alpha)
        history = track.setdefault('history_ne', [])
        history.append(pos_ne)
        if len(history) > self.obstacle_history_len:
            del history[:-self.obstacle_history_len]
        if detection_ne is not None:
            detection_ne = np.asarray(detection_ne, dtype=float).reshape(2)
            if np.isfinite(detection_ne).all():
                lidar_history = track.setdefault('lidar_history_ne', [])
                lidar_history.append(detection_ne.copy())
                if len(lidar_history) > self.obstacle_history_len:
                    del lidar_history[:-self.obstacle_history_len]
            speed_m_s = float(np.linalg.norm(vel_ne))
            heading_rad = float(np.arctan2(vel_ne[1], vel_ne[0])) if speed_m_s >= self.
obstacle_heading_hold_speed_m_s else float(track.get('heading_rad', np.nan))
            motion_window = track.setdefault('motion_window', [])
            motion_window.append({'stamp_s': stamp_s, 'pos_ne': pos_ne, 'vel_ne': vel_ne
, 'speed_m_s': speed_m_s, 'heading_rad': heading_rad, 'pc1_m': track['pc1_m'], '
pc2_m': track['pc2_m']})
            cutoff_s = stamp_s - max(float(self.obstacle_stats_window_s), 0.0)
            track['motion_window'] = [sample for sample in motion_window if float(sample.
get('stamp_s', stamp_s)) >= cutoff_s][-self.obstacle_history_len:]
            self.update_obstacle_track_statistics(track)

```



```

def update_obstacle_track_statistics(self, track):
    samples = track.get('motion_window', [])
    samples = [sample for sample in samples if np.isfinite(np.asarray(sample.get(
('pos_ne', [np.nan, np.nan]), dtype=float)).all()))
    track['stats_sample_count'] = int(len(samples))
    if not samples:
        track['velocity_mean_ne'] = np.asarray(track.get('vel_ne', [0.0, 0.0]),
dtype=float).reshape(2)
        track['velocity_var_ne'] = np.zeros(2, dtype=float)
        track['speed_mean_m_s'] = float(np.linalg.norm(track['velocity_mean_ne'
]))
        track['speed_var_m2_s2'] = 0.0
        track['heading_mean_rad'] = np.nan
        track['heading_var_rad2'] = np.nan
        track['heading_circular_variance'] = np.nan
        track['pc1_mean_m'] = track.get('pc1_m', self.obstacle_min_pc1_m)
        track['pc2_mean_m'] = track.get('pc2_m', self.obstacle_min_pc2_m)
        track['pc1_var_m2'] = 0.0
        track['pc2_var_m2'] = 0.0
        track['accel_ne'] = np.zeros(2, dtype=float)
        track['accel_var_ne'] = np.zeros(2, dtype=float)
        track['motion_stable'] = False
        self.sync_obstacle_track_fields(track)
        return
    velocities = np.asarray([sample['vel_ne'] for sample in samples], dtype=
float)
    finite_vel = np.isfinite(velocities).all(axis=1)
    velocities = velocities[finite_vel]
    if len(velocities) > 0:
        track['velocity_mean_ne'] = np.mean(velocities, axis=0)
    else:
        track['velocity_mean_ne'] = np.asarray(track.get('vel_ne', [0.0, 0.0]),
dtype=float).reshape(2)
    if len(velocities) > 0:
        track['velocity_var_ne'] = np.var(velocities, axis=0)
    else:
        track['velocity_var_ne'] = np.zeros(2, dtype=float)
    speeds = np.asarray([sample['speed_m_s'] for sample in samples], dtype=float
)
    speeds = speeds[np.isfinite(speeds)]
    fitted_speed_m_s = float(np.linalg.norm(track['velocity_mean_ne']))
    if len(speeds) > 0:
        track['speed_mean_m_s'] = fitted_speed_m_s
        track['speed_var_m2_s2'] = float(np.var(speeds))
    else:
        track['speed_mean_m_s'] = fitted_speed_m_s
        track['speed_var_m2_s2'] = 0.0
    headings = np.asarray([sample['heading_rad'] for sample in samples], dtype=
float)
    headings = headings[np.isfinite(headings)]
    if len(headings) > 0:
        sin_mean = float(np.mean(np.sin(headings)))
        cos_mean = float(np.mean(np.cos(headings)))
        heading_mean = float(np.arctan2(sin_mean, cos_mean))
        heading_error = wrap_angle(headings - heading_mean)
        resultant_length = float(np.hypot(sin_mean, cos_mean))
        track['heading_mean_rad'] = heading_mean
        track['heading_var_rad2'] = float(np.var(heading_error))

```

```

        track['heading_circular_variance'] = float(1.0 - np.clip(
resultant_length, 0.0, 1.0))
    else:
        track['heading_mean_rad'] = np.nan
        track['heading_var_rad2'] = np.nan
        track['heading_circular_variance'] = np.nan
    pc1_values = np.asarray([sample['pc1_m'] for sample in samples], dtype=float
)
    pc2_values = np.asarray([sample['pc2_m'] for sample in samples], dtype=float
)
    pc1_values = pc1_values[np.isfinite(pc1_values) & (pc1_values > 0.0)]
    pc2_values = pc2_values[np.isfinite(pc2_values) & (pc2_values > 0.0)]
    track['pc1_mean_m'] = max(float(np.mean(pc1_values)), self.
obstacle_min_pc1_m) if len(pc1_values) > 0 else float(track.get('pc1_m', self.
obstacle_min_pc1_m))
    track['pc2_mean_m'] = max(float(np.mean(pc2_values)), self.
obstacle_min_pc2_m) if len(pc2_values) > 0 else float(track.get('pc2_m', self.
obstacle_min_pc2_m))
    track['pc2_mean_m'] = min(track['pc2_mean_m'], track['pc1_mean_m'])
    track['pc1_var_m2'] = float(np.var(pc1_values)) if len(pc1_values) > 0 else
0.0
    track['pc2_var_m2'] = float(np.var(pc2_values)) if len(pc2_values) > 0 else
0.0
    track['pc1_m'] = track['pc1_mean_m']
    track['pc2_m'] = track['pc2_mean_m']
    velocity_samples = [sample for sample in samples if np.isfinite(np.asarray(
sample.get('vel_ne', [np.nan, np.nan]), dtype=float)).all())]
    velocity_samples.sort(key=lambda sample: float(sample.get('stamp_s', 0.0)))
    accels = []
    for prev_sample, next_sample in zip(velocity_samples[:-1], velocity_samples
[1:]):
        dt = float(next_sample.get('stamp_s', 0.0)) - float(prev_sample.get('
stamp_s', 0.0))
        if dt <= 0.001:
            continue
        prev_vel = np.asarray(prev_sample['vel_ne'], dtype=float).reshape(2)
        next_vel = np.asarray(next_sample['vel_ne'], dtype=float).reshape(2)
        accels.append((next_vel - prev_vel) / dt)
    if accels:
        accel_ne = np.mean(np.asarray(accels, dtype=float), axis=0)
        accel_norm = float(np.linalg.norm(accel_ne))
        if accel_norm > self.obstacle_max_accel_m_s2:
            accel_ne *= self.obstacle_max_accel_m_s2 / max(accel_norm, 1e-06)
        track['accel_ne'] = accel_ne
        track['accel_var_ne'] = np.var(np.asarray(accels, dtype=float), axis=0)
    else:
        track['accel_ne'] = np.zeros(2, dtype=float)
        track['accel_var_ne'] = np.zeros(2, dtype=float)
    track['motion_stable'] = self.obstacle_track_motion_is_stable(track)
    self.sync_obstacle_track_fields(track)

def annotate_lidar_obstacle_with_track(self, obstacle, track):
    prediction_ne = np.asarray(track.get('prediction_ne', np.empty((0, 2))),
dtype=float)
    obstacle['track_id'] = int(track['id'])
    obstacle['velocity_ne'] = np.asarray(track['vel_ne'], dtype=float).reshape
(2).tolist()

```

```

        obstacle['velocity_mean_ne'] = np.asarray(track.get('velocity_mean_ne',
track['vel_ne']), dtype=float).reshape(2).tolist()
        obstacle['velocity_var_ne'] = np.asarray(track.get('velocity_var_ne', [0.0,
0.0]), dtype=float).reshape(2).tolist()
        obstacle['accel_ne'] = np.asarray(track.get('accel_ne', [0.0, 0.0]), dtype=
float).reshape(2).tolist()
        obstacle['speed_m_s'] = float(track.get('speed_m_s', 0.0))
        obstacle['speed_mean_m_s'] = float(track.get('speed_mean_m_s', obstacle['
speed_m_s']))
        obstacle['speed_var_m2_s2'] = float(track.get('speed_var_m2_s2', 0.0))
        obstacle['heading_rad'] = float(track.get('heading_rad', np.nan))
        obstacle['heading_deg'] = float(track.get('heading_deg', np.nan))
        obstacle['heading_mean_rad'] = float(track.get('heading_mean_rad', np.nan))
        obstacle['heading_var_rad2'] = float(track.get('heading_var_rad2', np.nan))
        obstacle['pc1_m'] = float(track.get('pc1_mean_m', track.get('pc1_m', self.
obstacle_min_pc1_m)))
        obstacle['pc2_m'] = float(track.get('pc2_mean_m', track.get('pc2_m', self.
obstacle_min_pc2_m)))
        obstacle['pc1_var_m2'] = float(track.get('pc1_var_m2', 0.0))
        obstacle['pc2_var_m2'] = float(track.get('pc2_var_m2', 0.0))
        obstacle['length_axis_ne'] = np.asarray(track.get('heading_axis_ne', track.
get('length_axis_ne', [1.0, 0.0])), dtype=float).reshape(2).tolist()
        obstacle['equivalent_radius_m'] = float(track.get('equivalent_radius_m',
self.obstacle_min_equivalent_radius_m))
        obstacle['stats_sample_count'] = int(track.get('stats_sample_count', 0))
        obstacle['motion_stable'] = bool(track.get('motion_stable', False))
        obstacle['prediction_model'] = track.get('prediction_model', '
ekf_constant_velocity')
        obstacle['predicted_trajectory_ne'] = prediction_ne.tolist()
        lidar_points_ne = np.asarray(obstacle.get('points_ne', []), dtype=float)
        if lidar_points_ne.ndim == 2 and lidar_points_ne.shape[1] == 2:
            track['lidar_points_ne'] = lidar_points_ne.tolist()

def prune_obstacle_tracks(self, now):
    self.apf_obstacle_tracks = [track for track in self.apf_obstacle_tracks if
now - float(track.get('last_seen_s', track.get('stamp_s', now))) <= self.
apf_track_timeout_s and int(track.get('miss_count', 0)) <= 5]

def update_apf_obstacle_tracks(self, stamp_s):
    if not self.obstacle_ekf_prediction_enabled:
        self.apf_obstacle_tracks = []
        self.apf_virtual_obstacles = []
        tracked_fields = {'track_id', 'velocity_ne', 'velocity_mean_ne', '
velocity_var_ne', 'accel_ne', 'speed_m_s', 'speed_mean_m_s', 'speed_var_m2_s2', '
heading_rad', 'heading_deg', 'heading_mean_rad', 'heading_var_rad2', 'pc1_m', '
pc2_m', 'pc1_var_m2', 'pc2_var_m2', 'stats_sample_count', 'motion_stable', '
prediction_model', 'predicted_trajectory_ne'}
        for obstacle in self.lidar_obstacles:
            for field in tracked_fields:
                obstacle.pop(field, None)
        return
    now = float(stamp_s if stamp_s is not None else time.time())
    confirmation_hits = max(int(getattr(self, 'obstacle_track_confirmation_hits'
, 2)), 2)
    detections = []
    for obstacle_index, obstacle in enumerate(self.lidar_obstacles):
        centre_ne = np.asarray(obstacle.get('centre_ne', [np.nan, np.nan]),
dtype=float).reshape(2)

```

```

        if np.isfinite(centre_ne).all():
            pc1_m = float(obstacle.get('pc1_m', self.obstacle_min_pc1_m))
            pc2_m = float(obstacle.get('pc2_m', self.obstacle_min_pc2_m))
            length_axis_ne = np.asarray(obstacle.get('length_axis_ne', [1.0,
0.0])), dtype=float).reshape(2)
            measurement_covariance = np.asarray(obstacle.get('
measurement_covariance', self.obstacle_measurement_covariance(centre_ne)), dtype
=float).reshape(2, 2)
            detections.append((obstacle_index, centre_ne, pc1_m, pc2_m,
length_axis_ne, measurement_covariance))
        if not detections:
            self.apf_obstacle_track_candidates = [candidate for candidate in self.
apf_obstacle_track_candidates if now - float(candidate.get('last_seen_s',
candidate.get('stamp_s', now))) <= self.apf_track_timeout_s]
            for track in self.apf_obstacle_tracks:
                self.predict_obstacle_track_to_time(track, now)
                track['miss_count'] = int(track.get('miss_count', 0)) + 1
            self.prune_obstacle_tracks(now)
            return
        detection_positions = np.asarray([detection for _, detection, _, _, _ in
detections], dtype=float)
        predicted_tracks = []
        candidates = []
        matched_candidates = set()
        promoted_candidates = set()
        for track_index, track in enumerate(self.apf_obstacle_tracks):
            dt = max(now - float(track.get('stamp_s', now)), 0.0)
            predicted_state, predicted_covariance = self.obstacle_ekf_predict(track.
get('state', np.r_[track.get('pos_ne', [np.nan, np.nan]), track.get('vel_ne',
[0.0, 0.0])])), track.get('covariance', np.eye(4, dtype=float)), dt)
            predicted_tracks.append((predicted_state, predicted_covariance))
            predicted_pos = predicted_state[0:2]
            for detection_index, detection in enumerate(detection_positions):
                distance = float(np.linalg.norm(detection - predicted_pos))
                candidates.append((distance, track_index, detection_index))
            candidates.sort(key=lambda item: item[0])
            assigned_tracks = set()
            assigned_detections = set()
            detection_track = {}
            for distance, track_index, detection_index in candidates:
                if distance > self.apf_track_association_m:
                    break
                if track_index in assigned_tracks or detection_index in
assigned_detections:
                    continue
                track = self.apf_obstacle_tracks[track_index]
                detection = detection_positions[detection_index]
                predicted_state, predicted_covariance = predicted_tracks[track_index]
                corrected_state, corrected_covariance = self.obstacle_ekf_update(
predicted_state, predicted_covariance, detection, detections[detection_index
][5])
                track['state'] = corrected_state
                track['covariance'] = corrected_covariance
                track['stamp_s'] = now
                track['last_seen_s'] = now
                track['hit_count'] = int(track.get('hit_count', 0)) + 1
                track['miss_count'] = 0
                self.sync_obstacle_track_fields(track)

```

```

        self.append_obstacle_track_history(track, pc1_m=detections[
detection_index][2], pc2_m=detections[detection_index][3], length_axis_ne=
detections[detection_index][4], detection_ne=detection, stamp_s=now)
        assigned_tracks.add(track_index)
        assigned_detections.add(detection_index)
        detection_track[detection_index] = track
    for track_index, track in enumerate(self.apf_obstacle_tracks):
        if track_index not in assigned_tracks:
            predicted_state, predicted_covariance = predicted_tracks[track_index
]

            track['state'] = predicted_state
            track['covariance'] = predicted_covariance
            track['stamp_s'] = now
            track['miss_count'] = int(track.get('miss_count', 0)) + 1
            self.sync_obstacle_track_fields(track)
    for detection_index, detection in enumerate(detections):
        if detection_index in assigned_detections:
            continue
        _, detection_ne, pc1_m, pc2_m, length_axis_ne, _ = detection
        best_candidate_index = None
        best_candidate_distance = np.inf
        for candidate_index, candidate in enumerate(self.
apf_obstacle_track_candidates):
            if candidate_index in matched_candidates:
                continue
            candidate_pos = np.asarray(candidate.get('centre_ne', [np.nan, np.
nan]), dtype=float).reshape(2)
            if not np.isfinite(candidate_pos).all():
                continue
            distance = float(np.linalg.norm(detection_ne - candidate_pos))
            if distance < best_candidate_distance:
                best_candidate_distance = distance
                best_candidate_index = candidate_index
        if best_candidate_index is not None and best_candidate_distance <= self.
apf_track_association_m:
            candidate = self.apf_obstacle_track_candidates[best_candidate_index]
            candidate['centre_ne'] = detection_ne.copy()
            candidate['last_seen_s'] = now
            candidate['hit_count'] = int(candidate.get('hit_count', 0)) + 1
            candidate['pc1_m'] = pc1_m
            candidate['pc2_m'] = pc2_m
            candidate['length_axis_ne'] = length_axis_ne
            matched_candidates.add(best_candidate_index)
            if int(candidate['hit_count']) >= confirmation_hits:
                track = self.make_obstacle_track(detection_ne, now, pc1_m=pc1_m,
pc2_m=pc2_m, length_axis_ne=length_axis_ne)
                self.apf_obstacle_tracks.append(track)
                assigned_detections.add(detection_index)
                detection_track[detection_index] = track
                promoted_candidates.add(best_candidate_index)
            continue
        self.apf_obstacle_track_candidates.append(self.
make_obstacle_track_candidate(detection_ne, now, pc1_m=pc1_m, pc2_m=pc2_m,
length_axis_ne=length_axis_ne))
    for detection_index, track in detection_track.items():
        obstacle_index, _, _, _, _ = detections[detection_index]
        self.annotate_lidar_obstacle_with_track(self.lidar_obstacles[
obstacle_index], track)

```

```

        self.apf_obstacle_track_candidates = [candidate for candidate_index,
        candidate in enumerate(self.apf_obstacle_track_candidates) if candidate_index
        not in promoted_candidates if now - float(candidate.get('last_seen_s', candidate.
        get('stamp_s', now))) <= self.apf_track_timeout_s]
        self.prune_obstacle_tracks(now)

    def apf_track_for_obstacle(self, obstacle):
        if not self.obstacle_ekf_prediction_enabled:
            return None
        centre_ne = np.asarray(obstacle.get('centre_ne', [np.nan, np.nan]), dtype=
        float).reshape(2)
        if not np.isfinite(centre_ne).all():
            return None
        now = float(self.latest_lidar_received_s if self.latest_lidar_received_s is
        not None else time.time())
        best_track = None
        best_distance = np.inf
        for track in self.apf_obstacle_tracks:
            if now - float(track.get('last_seen_s', track.get('stamp_s', now))) >
            self.apf_track_timeout_s:
                continue
            predicted_state, _ = self.obstacle_ekf_predict(track.get('state', np.r_[
            track.get('pos_ne', [np.nan, np.nan]), track.get('vel_ne', [0.0, 0.0]])], track.
            get('covariance', np.eye(4, dtype=float)), max(now - float(track.get('stamp_s',
            now)), 0.0))
            predicted_pos = predicted_state[0:2]
            distance = float(np.linalg.norm(centre_ne - predicted_pos))
            if distance < best_distance:
                best_distance = distance
                best_track = track
            if best_distance <= max(self.apf_track_association_m * 1.5, self.
            lidar_dbscan_eps_m * 2.0):
                return best_track
        return None

    def apf_build_encounter_params(self, **overrides):
        params = {'cluster_range_enabled': bool(self.apf_cluster_range_enabled), '
        risk_pc_scale': float(self.apf_risk_pc_scale), 'avoidance_pc_scale': float(self.
        apf_avoidance_pc_scale), 'direction_pc_scale': float(self.apf_direction_pc_scale
        ), 'virtual_pc_scale': float(self.apf_virtual_pc_scale), 'repulsive_gain': float
        (self.apf_repulsive_gain), 'route_lookahead_m': float(self.apf_route_lookahead_m
        ), 'collision_horizon_s': float(self.apf_collision_horizon_s), '
        constant_descent_speed_m_s': float(self.apf_constant_descent_speed_m_s), '
        dynamic_speed_threshold_m_s': float(self.apf_dynamic_speed_threshold_m_s)}
        params.update(overrides)
        return params

    def apf_profile_name_for_encounter(self, encounter):
        if encounter == 'overtaking':
            return 'overtaking'
        if encounter == 'head_on':
            return 'head_on'
        return 'crossing'

    def apf_params_for_encounter(self, encounter=None):
        profile_name = self.apf_profile_name_for_encounter(self.apf_encounter_mode
        if encounter is None else encounter)
        if profile_name == 'overtaking':

```



```

        return self.apf_overtaking_params
    if profile_name == 'head_on':
        return self.apf_head_on_params
    return self.apf_crossing_params

def apf_cpa_metrics(self, obs_pos_body, obs_vel_body, own_vel_body):
    if not self.obstacle_ekf_prediction_enabled:
        return (np.nan, np.nan)
    tcpa_s, dcpa_m, _, _ = straight_line_cpa([0.0, 0.0], own_vel_body,
obs_pos_body, obs_vel_body)
    return (tcpa_s, dcpa_m)

def apf_pass_astern_side_from_velocity(self, obs_vel_body):
    obs_vel_body = np.asarray(obs_vel_body, dtype=float).reshape(2)
    if not np.isfinite(obs_vel_body).all():
        return 0.0
    if float(np.linalg.norm(obs_vel_body)) < self.
apf_dynamic_speed_threshold_m_s:
        return 0.0
    lateral_speed = float(obs_vel_body[1])
    if abs(lateral_speed) < self.apf_dynamic_speed_threshold_m_s:
        return 0.0
    return float(np.sign(lateral_speed))

def apf_classify_encounter(self, obs_pos_body, obs_vel_body, own_vel_body):
    obs_pos_body = np.asarray(obs_pos_body, dtype=float).reshape(2)
    obs_vel_body = np.asarray(obs_vel_body, dtype=float).reshape(2)
    own_vel_body = np.asarray(own_vel_body, dtype=float).reshape(2)
    body_angle_rad = float(np.arctan2(obs_pos_body[1], obs_pos_body[0]))
    bearing_starboard_deg = float(np.rad2deg(wrap_angle(-body_angle_rad)))
    obs_speed = float(np.linalg.norm(obs_vel_body))
    own_speed = float(np.linalg.norm(own_vel_body))
    dynamic_obstacle = obs_speed >= self.apf_dynamic_speed_threshold_m_s
    relative_heading_deg = np.nan
    if 'overtaking' in str(getattr(self, 'webots_environment', '')).lower():
        return ('overtaking', -1.0, 'COLREG Rule 13: overtake on starboard side'
)
    if dynamic_obstacle:
        relative_heading_deg = abs(float(np.rad2deg(wrap_angle(np.arctan2(
obs_vel_body[1], obs_vel_body[0])))))
        if np.isfinite(relative_heading_deg) and abs(bearing_starboard_deg) <= 22.5
and (relative_heading_deg >= 157.5):
            return ('head_on', -1.0, 'COLREG Rule 14: alter to starboard')
        if np.isfinite(relative_heading_deg) and abs(bearing_starboard_deg) <= 67.5
and (relative_heading_deg <= 67.5) and (own_speed > obs_speed + self.
apf_dynamic_speed_threshold_m_s):
            return ('overtaking', -1.0, 'COLREG Rule 13: overtake on starboard side'
)
        pass_astern_side = self.apf_pass_astern_side_from_velocity(obs_vel_body)
        if dynamic_obstacle and 0.0 < bearing_starboard_deg <= 112.5:
            requested_side = pass_astern_side if pass_astern_side != 0.0 else -1.0
            return ('crossing_from_starboard', requested_side, 'COLREG Rule 15: give
way, pass astern')
        if dynamic_obstacle and -112.5 <= bearing_starboard_deg < 0.0:
            return ('crossing_from_port', 0.0, 'COLREG Rule 17: stand on')
        return ('static_obstacle', 0.0, 'none')

def apf_default_side_from_obstacle(self, obs_pos_body):

```

```

        body_angle_rad = float(np.arctan2(obs_pos_body[1], obs_pos_body[0]))
        if abs(wrap_angle(body_angle_rad)) > self.
apf_activation_front_half_angle_rad:
    return 0.0
    if abs(body_angle_rad) <= np.deg2rad(5.0):
        if self.left_clearance_m > self.right_clearance_m + 0.05:
            return 1.0
        if self.right_clearance_m > self.left_clearance_m + 0.05:
            return -1.0
        return -1.0
    return -1.0 if body_angle_rad > 0.0 else 1.0

def apf_obstacle_in_priority_front_sector(self, obstacle):
    obs_pos_body = np.asarray(obstacle.get('centre_body', [np.nan, np.nan]),
dtype=float).reshape(2)
    if not np.isfinite(obs_pos_body).all():
        return False
    body_angle_rad = float(np.arctan2(obs_pos_body[1], obs_pos_body[0]))
    return abs(wrap_angle(body_angle_rad)) <= self.
apf_priority_front_half_angle_rad

def apf_lock_side(self, requested_side, obstacle_level):
    now_s = float(self.timefromstart) if self.timefromstart is not None else 0.0
    obstacle_close = np.isfinite(obstacle_level) and obstacle_level <= self.
apf_side_lock_exit_level
    if requested_side == 0.0:
        if self.apf_side_lock_sign != 0.0 and (obstacle_close or now_s < self.
apf_side_lock_until_s):
            self.apf_side_lock_active = True
            return self.apf_side_lock_sign
        self.apf_side_lock_sign = 0.0
        self.apf_side_lock_active = False
        return 0.0
    requested_side = float(np.sign(requested_side))
    if self.apf_side_lock_sign != 0.0 and (obstacle_close or now_s < self.
apf_side_lock_until_s):
        self.apf_side_lock_active = True
        return self.apf_side_lock_sign
    self.apf_side_lock_sign = requested_side
    self.apf_side_lock_until_s = now_s + self.apf_side_lock_s
    self.apf_side_lock_active = True
    return self.apf_side_lock_sign

def refresh_apf_side_lock(self, nearest_forward_level=np.inf):
    if self.apf_side_lock_sign == 0.0:
        self.apf_side_lock_active = False
        return False
    now_s = float(self.timefromstart) if self.timefromstart is not None else 0.0
    obstacle_close = np.isfinite(nearest_forward_level) and
nearest_forward_level <= self.apf_side_lock_exit_level
    if obstacle_close or now_s < self.apf_side_lock_until_s:
        self.apf_side_lock_active = True
        return True
    self.apf_side_lock_sign = 0.0
    self.apf_side_lock_active = False
    return False

def route_progress_and_point(self, lookahead_m=0.0):

```



```

        current_ne = np.array([float(self.North), float(self.East)], dtype=float)
        if self.route_path_length_m < 1e-09:
            return (0.0, self.goal_ne.copy())
        along_m = float(np.dot(current_ne - self.start_ne, self.route_path_unit_ne))
        closest_along_m = float(np.clip(along_m, 0.0, self.route_path_length_m))
        target_along_m = float(np.clip(along_m + lookahead_m, 0.0, self.
route_path_length_m))
        target_ne = self.start_ne + target_along_m * self.route_path_unit_ne
        return (closest_along_m, target_ne)

def goal_distance_m(self):
    current_ne = np.array([float(self.North), float(self.East)], dtype=float)
    return float(np.linalg.norm(self.goal_ne - current_ne))

def final_approach_active(self, final_distance_m=None):
    if final_distance_m is None:
        final_distance_m = self.goal_distance_m()
    if not np.isfinite(final_distance_m):
        return False
    if final_distance_m <= self.final_approach_distance_m:
        return True
    if self.route_path_length_m < 1e-09:
        return True
    current_ne = np.array([float(self.North), float(self.East)], dtype=float)
    along_m = float(np.dot(current_ne - self.start_ne, self.route_path_unit_ne))
    return along_m >= self.route_path_length_m - self.final_approach_distance_m

def apf_path_attraction_body(self):
    path_vec = self.goal_ne - self.start_ne
    path_len_sq = float(np.dot(path_vec, path_vec))
    if path_len_sq < 1e-09:
        return np.zeros(2, dtype=float)
    current_ne = np.array([float(self.North), float(self.East)], dtype=float)
    ratio = float(np.clip(np.dot(current_ne - self.start_ne, path_vec) /
path_len_sq, 0.0, 1.0))
    closest_ne = self.start_ne + ratio * path_vec
    to_path_body = self.earth_point_to_body(closest_ne)
    distance_m = float(np.linalg.norm(to_path_body))
    if distance_m < self.apf_path_threshold_m or distance_m < 1e-06:
        return np.zeros(2, dtype=float)
    return self.apf_path_gain * to_path_body

def apf_goal_attraction_body(self, target_body):
    target_body = np.asarray(target_body, dtype=float).reshape(2)
    distance_m = float(np.linalg.norm(target_body))
    if distance_m < 1e-06:
        return np.zeros(2, dtype=float)
    magnitude = self.apf_goal_gain * min(distance_m, self.
apf_attraction_saturation_m)
    return magnitude * target_body / distance_m

def apf_primary_encounter_mode(self):
    own_vel_body = self.current_velocity_body()
    obstacles = self.lidar_obstacles + self.update_apf_virtual_obstacles()
    saw_crossing = False
    for obstacle in obstacles:
        obs_pos_body = np.asarray(obstacle.get('centre_body', [np.nan, np.nan]),
dtype=float).reshape(2)

```

```

        if not np.isfinite(obs_pos_body).all():
            continue
        angle_rad = abs(wrap_angle(float(np.arctan2(obs_pos_body[1],
obs_pos_body[0]))))
        if not bool(obstacle.get('virtual', False)) and angle_rad > self.
apf_activation_front_half_angle_rad:
            continue
        if bool(obstacle.get('virtual', False)):
            encounter = str(obstacle.get('encounter_mode', 'crossing'))
        else:
            track = self.apf_track_for_obstacle(obstacle)
            if track is None or not self.obstacle_track_motion_is_stable(track):
                continue
            track_velocity_ne = np.asarray(track.get('vel_ne', [np.nan, np.nan])
, dtype=float).reshape(2)
            if not np.isfinite(track_velocity_ne).all():
                continue
            obs_vel_body = self.earth_vector_to_body(track_velocity_ne)
            encounter, _, _ = self.apf_classify_encounter(obs_pos_body,
obs_vel_body, own_vel_body)
            if encounter == 'head_on':
                return 'head_on'
            if encounter == 'overtaking':
                return 'overtaking'
            if encounter in {'crossing_from_starboard', 'crossing_from_port'}:
                saw_crossing = True
            return 'crossing' if saw_crossing else 'crossing'

def apf_avoidance_needed(self):
    virtual_obstacles = self.update_apf_virtual_obstacles()
    for obstacle in self.lidar_obstacles + virtual_obstacles:
        obs_pos_body = np.asarray(obstacle.get('centre_body', [np.nan, np.nan]),
dtype=float).reshape(2)
        if not np.isfinite(obs_pos_body).all():
            continue
        angle_rad = abs(wrap_angle(float(np.arctan2(obs_pos_body[1],
obs_pos_body[0]))))
        if not bool(obstacle.get('virtual', False)) and angle_rad > self.
apf_activation_front_half_angle_rad:
            continue
        if self.apf_repulsion_for_obstacle(obstacle, None, None)[1]:
            return True
    return False

def apf_overtaking_close_turn_command(self, encounter,
nearest_obstacle_distance_m, force_angle, surge_speed):
    overtaking_world = 'overtaking' in str(getattr(self, 'webots_environment', ''
')).lower()
    if encounter not in {'overtaking', 'static_obstacle', 'none'} and (not
overtaking_world) or nearest_obstacle_distance_m > self.
apf_overtaking_close_distance_m:
        return (force_angle, surge_speed)
    force_angle = max(abs(float(force_angle)), self.
apf_overtaking_close_turn_angle_rad)
    heading_offset = abs(wrap_angle(float(self.Yaw) - self.route_heading_rad))
    if heading_offset < self.apf_overtaking_turn_release_angle_rad:
        surge_speed = min(surge_speed, self.apf_overtaking_close_turn_speed_m_s)
    return (force_angle, surge_speed)

```

```

def compute_apf_control(self, t, u_track):
    primary_encounter = 'overtaking'
    active_params = self.apf_overtaking_params
    final_approach = self.final_approach_active()
    if final_approach:
        target_ne = self.goal_ne.copy()
    else:
        _, target_ne = self.route_progress_and_point(float(active_params.get('
route_lookahead_m', self.apf_route_lookahead_m)))
        target_body = self.earth_point_to_body(target_ne)
        goal_body = self.earth_point_to_body(self.goal_ne)
        path_force = np.zeros(2, dtype=float) if final_approach else self.
apf_path_attraction_body()
        attractive_force = self.apf_goal_attraction_body(goal_body) + path_force
        force_body = attractive_force.copy()
        repulsive_force = np.zeros(2, dtype=float)
        own_vel_body = self.current_velocity_body()
        any_repulsion = False
        clearance_offset_m = 0.0
        self.reset_apf_diagnostics()
        self.apf_target_ne = target_ne.copy()
        self.apf_attractive_force_body = attractive_force
        virtual_obstacles = self.update_apf_virtual_obstacles()
        obstacles = self.lidar_obstacles + virtual_obstacles
        nearest_obstacle_distance_m = np.inf
        priority_obstacles = []
        secondary_obstacles = []
        for obstacle in obstacles:
            obs_pos_body = np.asarray(obstacle.get('centre_body', [np.nan, np.nan]),
dtype=float).reshape(2)
            if not bool(obstacle.get('virtual', False)) and np.isfinite(obs_pos_body
).all() and (obs_pos_body[0] > 0.0) and (abs(float(np.arctan2(obs_pos_body[1],
obs_pos_body[0]))) <= np.deg2rad(22.5)):
                nearest_obstacle_distance_m = min(nearest_obstacle_distance_m, float
(np.linalg.norm(obs_pos_body)))
                if self.apf_obstacle_in_priority_front_sector(obstacle):
                    priority_obstacles.append(obstacle)
                else:
                    secondary_obstacles.append(obstacle)
        for obstacle in priority_obstacles:
            repulsion, active, obstacle_params = self.apf_repulsion_for_obstacle(
obstacle, target_body, own_vel_body)
            repulsive_force += repulsion
            force_body += repulsion
            any_repulsion = any_repulsion or active
            if active:
                active_params = obstacle_params
                clearance_offset_m = max(clearance_offset_m, self.
apf_direction_clearance_m(obstacle, params=obstacle_params))
            if not any_repulsion:
                for obstacle in secondary_obstacles:
                    repulsion, active, obstacle_params = self.apf_repulsion_for_obstacle
(obstacle, target_body, own_vel_body)
                    repulsive_force += repulsion
                    force_body += repulsion
                    any_repulsion = any_repulsion or active
                    if active:

```

```

        active_params = obstacle_params
        clearance_offset_m = max(clearance_offset_m, self.
apf_direction_clearance_m(obstacle, params=obstacle_params))
        if any_repulsion and self.apf_side_lock_sign != 0.0:
            route_normal_left_ne = np.array([self.route_path_unit_ne[1], -self.
route_path_unit_ne[0]], dtype=float)
            offset_target_ne = target_ne + self.apf_side_lock_sign * max(
clearance_offset_m, self.obstacle_min_pc2_m) * route_normal_left_ne
            offset_target_body = self.earth_point_to_body(offset_target_ne)
            offset_distance = float(np.linalg.norm(offset_target_body))
            if offset_distance > 1e-06:
                offset_force = self.apf_clearance_gain * min(offset_distance, self.
apf_attraction_saturation_m) * offset_target_body / offset_distance
                force_body += offset_force
                attractive_force += offset_force
                self.apf_target_ne = offset_target_ne.copy()
            self.apf_attractive_force_body = attractive_force
            force_norm = float(np.linalg.norm(force_body))
            if not np.isfinite(force_norm) or force_norm < 1e-06:
                force_body = np.array([0.001, 0.0], dtype=float)
                force_norm = float(np.linalg.norm(force_body))
            self.apf_force_body = force_body
            self.apf_repulsive_force_body = repulsive_force
            steering_force = force_body.copy()
            if any_repulsion and steering_force[0] <= 0.0:
                side_sign = float(np.sign(steering_force[1]))
                if side_sign == 0.0:
                    side_sign = self.apf_side_lock_sign
                if side_sign == 0.0:
                    if self.left_clearance_m > self.right_clearance_m + 0.05:
                        side_sign = 1.0
                    else:
                        side_sign = -1.0
                lateral_mag = max(abs(float(steering_force[1])), 0.5 * abs(float(
force_body[0])), 0.2)
                steering_force[1] = side_sign * lateral_mag
                steering_force[0] = max(0.25 * lateral_mag, 0.05)
            self.apf_steering_force_body = steering_force
            now_s = float(self.timefromstart) if self.timefromstart is not None else 0.0
            self.apf_visual_hold_until_s = now_s + self.apf_visual_hold_s
            force_angle = wrap_angle(float(np.arctan2(steering_force[1], steering_force
[0])))
            force_angle = float(np.clip(force_angle, -self.apf_heading_step_limit_rad,
self.apf_heading_step_limit_rad))
            surge_speed = float(active_params.get('constant_descent_speed_m_s', self.
apf_constant_descent_speed_m_s))
            force_angle, surge_speed = self.apf_overtaking_close_turn_command(
primary_encounter, nearest_obstacle_distance_m, force_angle, surge_speed)
            u_cmd = Vector(2)
            u_cmd[1, 0] = np.clip(-self.apf_heading_gain * force_angle / max(self.lastdt
, 0.001), -self.w_max, self.w_max)
            u_cmd[0, 0] = float(np.clip(surge_speed, 0.0, self.v_max))
            if any_repulsion or self.apf_colreg_active or self.apf_side_lock_active:
                if self.apf_colreg_active:
                    self.navigation_mode = 'apf_colreg'
                else:
                    self.navigation_mode = 'apf_avoid'
            else:

```

```
        self.navigation_mode = 'apf_track'  
    return u_cmd
```